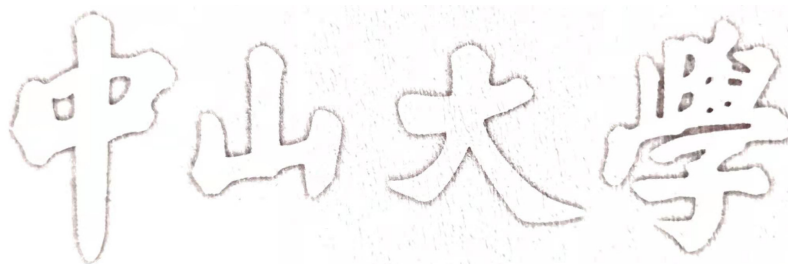


密级: 公开

编号: [学号]



工程 硕士专业学位论文

基于并行计算的金融大规模数据异常检测系统
研究与实现

Research and Implementation of a
Parallel-Computing-Based Anomaly Detection
System for Large-Scale Financial Data

学位申请人: 黄裕文

导师姓名及职称: 黄聃 教授

专业、领域名称: 计算机技术

2026 年 8 月 16 日

中山大学硕士学位论文

基于并行计算的金融大规模数据异常检测系统研究与实现

Research and Implementation of a
Parallel-Computing-Based Anomaly
Detection System for Large-Scale
Financial Data

专业: 计算机技术
学位申请人: 黄裕文
指导教师: 黄聃 教授

论文答辩委员会主席: _____
成员: _____

二〇二六年八月

原创性及使用授权声明

论文原创性声明

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

日期： 年 月 日

学位论文使用授权声明

本人完全了解中山大学有关保留、使用学位论文的规定，即：学校有权保留学位论文并向国家主管部门或其指定机构送交论文的电子版和纸质版；有权将学位论文用于非赢利目的的少量复制并允许论文进入学校图书馆、院系资料室被查阅；有权将学位论文的内容编入有关数据库进行检索；可以采用复印、缩印或其他方法保存学位论文；可以为存在馆际合作关系的兄弟高校用户提供文献传递服务和交换服务。

保密论文保密期满后，适用本声明。

学位论文作者签名：

日期： 年 月 日

导师签名：

日期： 年 月 日

摘要

金融大规模异常检测不仅要从极低异常比例的数据中识别高风险对象，还要满足近实时场景对吞吐和时延的要求。随着样本量由百万级向千万级、亿级扩展，系统瓶颈逐渐集中到四个环节：全量异常评分开销高，候选召回在外存访问条件下面临吞吐约束，CPU 与 GPU 之间的数据交换会放大阶段边界成本，多 GPU 并行还受到结果归并、共享 I/O 路径和硬件拓扑的影响。基于这一判断，本文将研究重点放在大规模金融异常检测的系统组织与执行效率上，目标是在尽量保持检测效果的同时降低端到端开销，并提高单机多 GPU 条件下的扩展能力。

针对上述问题，本文设计并实现 PyTOD-FlashANNS 协同异常检测系统 FlashTOD。系统采用“候选召回—异常评分”的两阶段处理方式：FlashANNS 先从大规模样本中生成规模受控的候选集合，PyTOD 再在候选范围内执行张量化异常评分，并输出异常分数和 Top-k 风险对象。通过候选预算限制后续评分范围，FlashTOD 将全量高代价评分转化为候选集上的精细评分，使检测效果与系统开销之间形成可调节的折中。本文并不提出新的异常检测理论模型，而是重点解决候选召回与异常评分在同一执行链路中的接口衔接和数据组织问题。

单 GPU 场景下，本文围绕候选召回到异常评分之间的热路径进行数据路径优化。针对查询准备、候选重组和评分输入过程中频繁的 CPU-GPU 往返与阶段同步，构建 GPU 主导的执行链路，使查询向量、候选 ID、局部距离和评分中间张量尽量在设备侧连续传递，并通过异步 I/O 与计算重叠、缓冲区复用等方式减少等待。多 GPU 场景则采用数据并行与索引复制的 replicated 架构，使每张 GPU 独立完成本地候选召回和异常评分，CPU 主要负责请求调度和小规模结果汇总；对于需要跨卡协同的场景，进一步结合设备侧归并、通信和拓扑感知 I/O 控制，缓解共享路径竞争对吞吐和尾延迟的影响。

实验采用约 1M 规模的真实或半真实金融数据验证有效性，并使用 10M、50M 和 100M 高保真合成数据测试吞吐、延迟与扩展趋势。在本文设定的数据规模、硬件平台和候选预算条件下，完整融合方案的 PR-AUC 为 0.906，接近 PyTOD 全量评分的 0.932；QPS 由 2700 提升至 4280，增幅约 58.5%。单卡完整优化方案将 QPS 从 3180 提高到 4550，同时把 p99 延迟由 67.0 ms 降至 45.5 ms。replicated 路径下，1、2、4 GPU 的 QPS 分别为 4550、7780 和 13620，4 GPU 吞吐约为单 GPU 的 2.99 倍。

实验结果说明, FlashTOD 在保持较好检测效果的同时, 能够有效压缩单 GPU 执行链路中的数据传输与同步开销, 并在多 GPU 条件下获得可验证的吞吐扩展收益。相关结论应限定在本文给定的数据规模、候选预算和硬件平台范围内; 更复杂的真实生产负载以及多机分布式场景, 仍需进一步验证。

关键词: 异常检测, 候选召回, FlashANNS, GPU 并行计算, 金融风控

ABSTRACT

Large-scale outlier detection for financial risk control must identify a small fraction of high-risk objects from extremely imbalanced data, while meeting near-real-time throughput and tail-latency requirements. As the sample size grows from millions to tens or hundreds of millions, the bottleneck gradually concentrates on four stages: full-scan anomaly scoring is expensive, candidate retrieval becomes throughput-limited under out-of-core access, CPU–GPU data exchange amplifies stage-boundary overhead, and multi-GPU parallelism is constrained by result merging, shared I/O paths, and hardware topology. Motivated by these observations, this thesis focuses on system organization and execution efficiency for large-scale financial outlier detection, aiming to reduce end-to-end cost with minimal effectiveness loss and to improve scalability on a single machine with multiple GPUs.

To address the above challenges, this thesis designs and implements *FlashTOD*, a collaborative outlier detection system integrating PyTOD and FlashANNS. FlashTOD adopts a two-stage pipeline of “candidate retrieval–anomaly scoring”: FlashANNS first produces a size-bounded candidate set from large-scale samples, and PyTOD then performs tensorized anomaly scoring within this candidate set and outputs anomaly scores and Top- k risky objects. By constraining the scoring scope via a candidate budget, FlashTOD converts expensive full-scan scoring into refined scoring over candidates, providing a tunable trade-off between effectiveness and system cost. Rather than proposing a new theoretical model, the key contribution lies in interface alignment and data organization along a unified execution chain.

On a single GPU, we optimize the hot path between candidate retrieval and anomaly scoring. To reduce frequent CPU–GPU round trips and stage synchronization during query preparation, candidate regrouping, and scoring input construction, we build a GPU-led execution chain in which query vectors, candidate IDs, local distances, and intermediate scoring tensors are passed on-device as continuously as possible. We further overlap asynchronous I/O with computation and reuse buffers to reduce waiting. On multiple GPUs, FlashTOD adopts a replicated (data-parallel, index-replicated) architecture so that each GPU independently completes local retrieval and scoring, while the CPU mainly

performs request scheduling and lightweight result aggregation. For workloads requiring cross-device cooperation, we additionally combine device-side merging, communication, and topology-aware I/O control to mitigate throughput and tail-latency degradation caused by shared-path contention.

Experiments use real or semi-real financial data at around 1M scale for effectiveness evaluation, and high-fidelity synthetic data at 10M, 50M, and 100M scales for throughput, latency, and scalability stress tests. Under the evaluated data scales, hardware platform, and candidate budgets, the full fusion scheme achieves a PR-AUC of 0.906, close to the 0.932 of the PyTOD full-scoring baseline. QPS increases from 2700 to 4280 (+58.5%). The full single-GPU optimization increases QPS from 3180 to 4550 and reduces p99 latency from 67.0 ms to 45.5 ms. Along the replicated path, the QPS values on 1, 2, and 4 GPUs are 4550, 7780, and 13620, with 4 GPUs reaching about 2.99× the 1-GPU throughput. These results indicate that FlashTOD preserves competitive effectiveness while compressing data-movement/synchronization overhead on a single GPU and achieving measurable throughput scaling on multiple GPUs, with conclusions bounded by the evaluated conditions.

Keywords: Outlier Detection, Candidate Retrieval, FlashANNS, GPU Parallel Computing, Financial Risk Control

目 录

摘 要.....	I
ABSTRACT	III
符号和缩略语说明.....	VIII
插图清单.....	IX
附表清单.....	XI
第一章 绪论.....	1
1.1 研究背景与意义.....	1
1.2 本文研究内容.....	3
1.3 研究目标与技术路线.....	5
1.4 本文主要工作与技术贡献.....	6
1.5 论文组织结构.....	6
第二章 相关研究工作与关键技术基础.....	8
2.1 异常检测方法研究现状.....	8
2.2 TOD/PyTOD 的 GPU 张量化执行基础.....	9
2.3 FlashANNS 近似最近邻候选召回前端.....	11
2.4 GPU 数据处理、FAISS-GPU 与接口支撑.....	12
2.5 多 GPU 调度与在线推理启示.....	13
2.6 本章小结.....	14
第三章 FlashTOD 检测模型与数据集准备.....	15
3.1 金融异常检测任务建模.....	15
3.2 FlashTOD 系统整体架构.....	16
3.3 实验数据集选取与构建.....	19
3.4 数据预处理与特征构造.....	22
3.5 PyTOD 异常评分模块设计.....	23
3.6 FlashANNS 候选召回前端设计.....	25
3.7 FlashTOD 执行流程.....	27
3.8 基础实验与初步验证.....	28

3.9 本章小结	29
第四章 单卡场景下的系统设计与数据路径优化	30
4.1 单卡系统需求与总体架构	30
4.2 单卡场景下的瓶颈分析	31
4.3 GPU 主导的端到端执行链路设计	34
4.4 候选召回前端的数据路径组织与工程接入	35
4.5 单 GPU 执行链路的工程增强	38
4.6 单 GPU 实验与性能分析	38
4.7 本章小结	41
第五章 多 GPU 并行扩展与归并优化	42
5.1 多 GPU 扩展需求分析	42
5.2 多 GPU 扩展策略设计原则	43
5.3 数据并行与索引复制的多 GPU 执行架构	44
5.4 CPU 控制面调度与负载均衡	45
5.5 跨卡归并场景下的 GPU 侧归并与 NCCL 通信	47
5.6 拓扑感知 I/O 通道绑定与准入控制	47
5.7 多 GPU 扩展实验与结果分析	48
5.8 本章小结	51
第六章 综合实验设计与结果分析	52
6.1 实验环境与软硬件配置	52
6.2 数据集与任务设置	54
6.3 评价指标与比较原则	55
6.4 核心结果展示	57
6.5 结果讨论	62
6.6 局限性分析	63
6.7 本章小结	64
第七章 全文总结与展望	65
7.1 全文总结	65
7.2 未来工作展望	66
参考文献	68
附录 A 实验脚本与日志归档说明	71

附录 B 图表补充说明	74
附录 C 缩写、变量与配置对照	77
在学期间完成的相关学术成果	78
致 谢	79

符号和缩略语说明

ANN	Approximate Nearest Neighbor, 近似最近邻
ANNS	Approximate Nearest Neighbor Search, 近似最近邻搜索
AUC	Area Under Curve, 曲线下面积
BTO	Basic Tensor Operations, 基础张量算子
CPU	Central Processing Unit, 中央处理器
DMA	Direct Memory Access, 直接内存访问
FO	Functional Operations, 功能算子
GDS	GPUDirect Storage, GPU 直连存储访问机制
GPU	Graphics Processing Unit, 图形处理器
H2D / D2H	Host-to-Device / Device-to-Host, 主机到设备 / 设备到主机传输
I/O	Input/Output, 输入/输出
KNN	K-Nearest Neighbors, k 近邻
NIC	Network Interface Card, 网络接口卡
NUMA	Non-Uniform Memory Access, 非一致内存访问
PCIe	Peripheral Component Interconnect Express, 外设组件高速互连
PR-AUC	Precision-Recall Area Under Curve, 精确率—召回率曲线下面积
QPS	Queries Per Second, 每秒查询数
RMM	RAPIDS Memory Manager, RAPIDS 内存管理器
ROC-AUC	Receiver Operating Characteristic Area Under Curve, 受试者工作特征曲线下面积
SSD	Solid-State Drive, 固态硬盘
Top- k overlap	近似排序结果与基线排序结果的前 k 重合度
adaptive_ratio	多 GPU 动态分流比例策略
bounded batching	在时延约束下受控执行的微批处理策略
rerank	对粗排候选执行更高精度精排计算的步骤

插图清单

图 1-1	金融异常检测业务场景总览图（本文绘制）	1
图 1-2	数据规模增长与系统瓶颈迁移示意图（本文绘制）	2
图 1-3	金融大规模异常检测系统标准处理流程图（本文根据金融异常检测流程整理绘制）	3
图 1-4	全文研究内容与章节逻辑关系图（本文绘制）	7
图 2-1	TOD 系统总览图（根据文献 ^[7] 整理）	10
图 2-2	典型异常检测算法到张量算子的映射（根据文献 ^[7] 整理）	10
图 2-3	FlashANNS 异步 I/O—计算流水线原理（根据文献 ^[8] 整理）	11
图 2-4	RAPIDS 在本文系统中的数据处理位置（本文绘制）	12
图 2-5	FAISS-GPU 的 CPU/GPU 互操作路径（根据 FAISS-GPU 文档与本文系统接口整理）	13
图 3-1	金融异常检测任务建模图（本文绘制）	16
图 3-2	FlashTOD 系统概念图（本文绘制）	18
图 3-3	PyTOD 异常评分模块流程图（本文绘制）	24
图 3-4	FlashANNS 候选召回执行流程图（本文绘制）	26
图 3-5	模块间中间结果传递示意图（本文绘制）	27
图 3-6	不同数据集上的候选召回—候选评分初步有效性验证结果（实验数据绘制）	29
图 4-1	单卡异常检测系统总体架构图（本文绘制）	31
图 4-2	原始链路与 GPU 主导链路对比图（本文绘制）	33
图 4-3	外存 ANNS 异步 I/O—计算流水线与本文候选召回前端接入方式（根据文献 ^[8] 和本文系统实现整理）	36
图 4-4	1M 数据规模下单卡各阶段耗时分解（实验数据绘制）	39
图 4-5	10M 数据规模下单卡各阶段耗时分解（实验数据绘制）	40
图 4-6	单卡执行链路递进式消融结果（实验数据绘制；该图为递进式消融，非完全独立的单因素消融）	40
图 5-1	数据并行与索引复制多 GPU 架构（本文绘制）	45
图 5-2	CPU 控制面调度与微批示意（本文绘制）	46
图 5-3	拓扑感知 I/O 通道示意（本文绘制）	48

图 5-4	多 GPU 扩展趋势与共享路径瓶颈分析（实验数据绘制）	49
图 5-5	平均延迟与 p99 延迟变化（实验数据绘制）	49
图 5-6	I/O 通道配置对扩展性的影响（实验数据绘制）	50
图 6-1	实验平台拓扑示意图（本文根据实验平台配置绘制）	54
图 6-2	数据规模层级与实验类型关系图（本文绘制）	55
图 6-3	指标体系结构图（本文绘制）	57
图 6-4	端到端比较方案的检测效果与吞吐率（实验数据绘制）	59
图 6-5	端到端比较方案的吞吐率（实验数据绘制）	60
图 6-6	FlashTOD 效果与性能折中（实验数据绘制）	60
图 6-7	单卡优化与多 GPU 扩展收益对比（实验数据绘制）	61
图 6-8	不同规模下系统表现趋势（实验数据绘制）	61

附表清单

表 1-1 核心问题、处理方式、对应章节与验证指标 5

表 1-2 本文技术路线的实现层次 6

表 2-1 各类异常检测方法复杂度与系统适配性比较 9

表 2-2 已有技术能力边界及与本文关系 14

表 3-1 实验数据集总体信息 20

表 3-2 数据集属性与实验用途说明 21

表 3-3 异常评分指标与算子映射 24

表 3-4 候选召回模块主要参数 26

表 4-1 单卡主要瓶颈与对应优化项 33

表 5-1 FlashTOD 多 GPU 部署适用边界 44

表 5-2 多 GPU 扩展结果汇总 50

表 6-1 实验平台与软件环境配置 53

表 6-2 对比方案、工程基线与内部执行路径设置 56

表 6-3 端到端方案的检测效果与吞吐比较 59

表 6-4 FlashTOD 单 GPU 性能收益的路径来源 59

第一章 绪论

1.1 研究背景与意义

金融异常检测面对的对象并不抽象。它可能是一笔金额和地域组合异常的交易，也可能是账户在短时间窗内突然改变的行为序列，或由设备指纹、登录地点和交易对手共同构成的高风险模式。高频交易流持续进入系统后，风险告警必须在有限时间内完成；吞吐不足会造成请求堆积，尾延迟过高则可能使告警错过处置窗口。因而，检测效果与执行效率是同一业务约束的两个方面，而不是先后独立优化的目标^[1-2]。

图 1-1 概括了本文所关注的业务环境。单笔交易特征、账户时间窗统计、设备与地域信息以及关系结构特征需要在进入检测链路前形成一致的输入表达；异常分数和风险排序又要及时返回上层业务。这里不展开金融行业的一般流程，关键约束只有两个：输入持续增长，告警时间预算有限。前者抬高计算与存储压力，后者限制了系统依靠离线批处理消化峰值流量的空间^[3-4]。

数据规模较小时，主要矛盾通常是 CPU 上的距离计算、排序和聚合耗时。将评分算子迁移到 GPU 后，瓶颈并没有消失，而是转移了。全量距离或近邻计算仍可能产生远大于输入矩阵的中间结果；查询向量和候选结果跨越 CPU-GPU 边界时会产生 H2D、D2H 传输；候选 ID、距离与排序信息若在主机侧重新组织，还会增加同步点。索引无法完全驻留显存时，SSD 访问开始进入关键路径。扩展到多

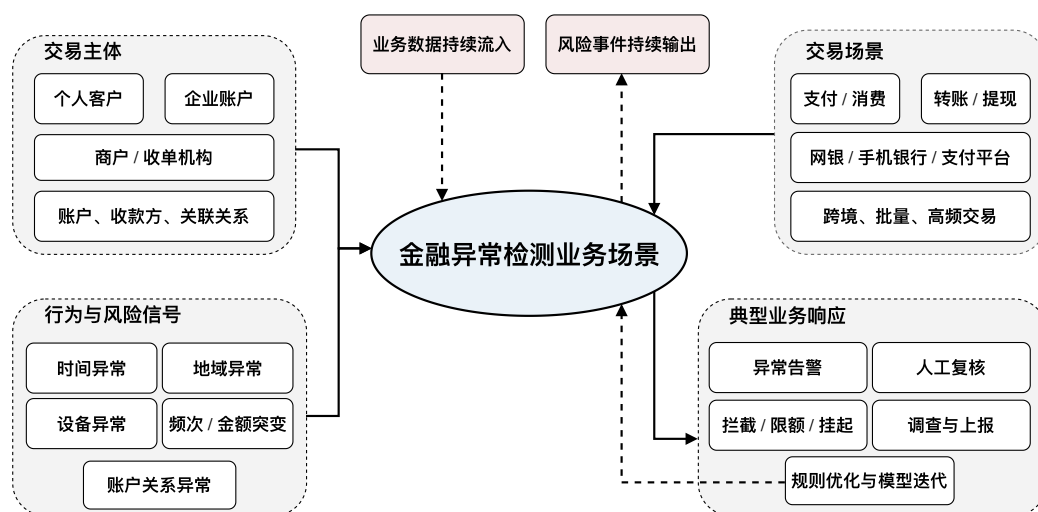


图 1-1 金融异常检测业务场景总览图（本文绘制）

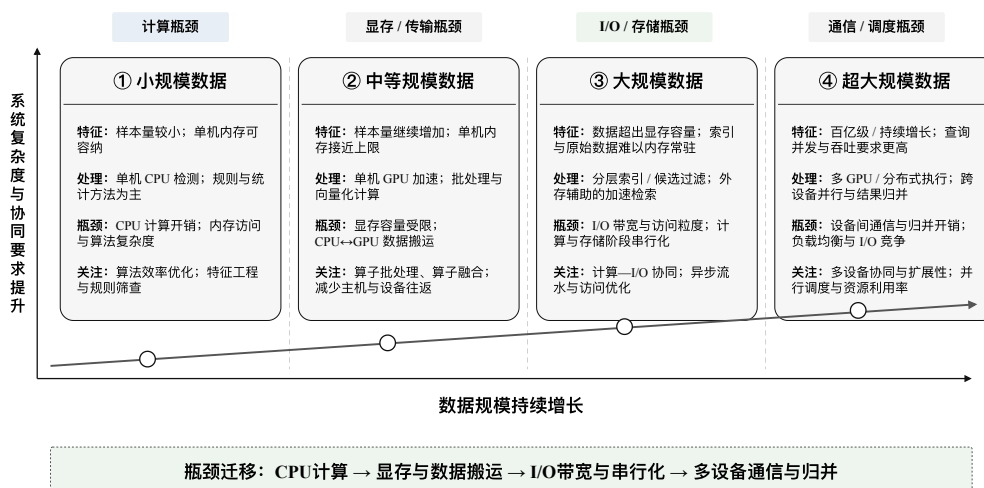


图 1-2 数据规模增长与系统瓶颈迁移示意图（本文绘制）

GPU 后，结果归并以及 PCIe、NUMA 和共享 I/O 竞争又会限制新增计算资源的利用率。

图 1-2 描述的是这种主导因素的变化，而不是若干彼此独立的性能问题。小规模阶段缩短 CPU 算子时间即可得到明显收益；当评分已经 GPU 化，系统开始受数据驻留位置和模块边界影响；当单卡链路被压缩，多 GPU 调度、跨卡归并与共享 SSD 路径才成为新的上限。只优化某个检测器或检索算子，无法保证端到端吞吐同步提高^[5-6]。

全量评分成本过高，使两阶段处理成为一种可行的系统选择：先通过近似最近邻（Approximate Nearest Neighbor, ANN）检索得到规模受控的候选集合，再在候选集合上执行较精细的异常评分。候选召回压缩了评分范围，但也引入了新的取舍。候选预算过小可能遗漏与异常判断有关的邻域，预算过大又会把计算压力重新交给评分模块。更重要的是，近似最近邻搜索与张量化评分的简单串接并不会自动形成低开销链路。候选结果若返回 CPU 打包，再通过 H2D 送入 GPU 评分，局部算子的加速仍可能被阶段边界抵消。

本文据此构建 FlashTOD。系统以 PyTOD 作为候选集合上的张量化异常评分模块^[7]，以已有 FlashANNS 系统的外存 ANNS 能力作为大规模候选召回前端^[8]。单 GPU 部分研究查询、候选和评分中间张量如何连续留在设备侧；多 GPU 部分研究查询调度、结果归并和共享 I/O 约束。本文讨论的是这些已有能力在金融大规模异常检测链路中的协同组织与实现边界，不将其表述为新的异常检测理论模型，也不把当前实验结果外推为对全部金融风控部署条件的普遍结论。

1.2 本文研究内容

1.2.1 本文研究对象与系统处理链路

图 1-3展示了金融异常检测所处的完整业务流程，其中包含数据接入、特征处理、风险识别、告警以及后续处置。该图用于交代系统上下文，并不表示本文实现了银行生产风控的全部环节。真实生产链路还可能包含规则引擎、人工审核、工单流转、拦截或冻结、额度调整以及模型在线更新闭环，这些环节依赖机构规则、权限体系和持续运营机制，不属于本文实现范围。

本文从数据预处理完成后的向量化输入开始，重点覆盖候选召回、候选组织、PyTOD 异常评分、风险排序和必要结果输出。原始交易记录、账户时间窗行为、设备与地域特征或图关系统计在进入系统前被整理为固定维度向量；候选召回前端返回候选 ID、距离和索引映射信息；评分模块在候选集合上生成异常分数与排序；系统只把最终小规模结果和必要日志交给后续业务。这里的“结果输出”不包含规则决策、人工审核和业务处置本身。

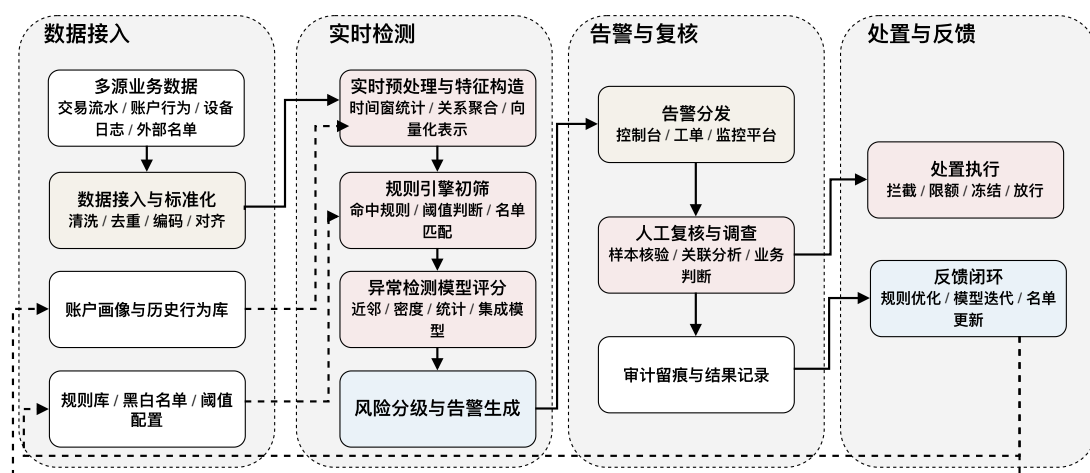


图 1-3 金融大规模异常检测系统标准处理流程图（本文根据金融异常检测流程整理绘制）

1.2.2 本文关注的核心系统问题

上述研究范围内有四个直接影响端到端表现的问题。它们分别落在评分、召回、模块边界和多设备扩展阶段，后文的系统设计与实验验证均围绕这四类问题展开。

异常评分阶段的高复杂度计算问题 距离型和密度型异常检测依赖近邻计算，常见评分过程包含距离计算、排序、局部密度估计与邻域聚合^[1,7]。复杂度压力并不只来自输入数据。输入矩阵本身可能仍能装入显存，但若直接构造全量距离矩阵，

存储量会达到 $O(n^2)$ ；朴素实现中的成对差分张量还会随特征维度继续放大。因而，问题 1 不是泛指“计算量较大”，而是如何避免在全量样本上物化高代价中间结果，并把可张量化评分限制在候选预算内。

候选召回阶段的存储访问与吞吐瓶颈问题 索引能够驻留显存时，召回开销主要来自近邻搜索与候选维护；索引超过显存容量后，SSD 读页进入关键路径。此时搜索性能不再只由 ANN 算法决定，I/O 访问粒度、在途请求数量以及 Fetch、Compute 和候选 Update 能否重叠都会影响吞吐。候选预算同样受到两端约束：预算过小可能降低异常覆盖和 Recall@k，过大则增加 PyTOD 的距离、topk 与聚合成本。问题 2 研究的是候选质量和外存访问代价如何共同受控，而不是单独追求检索吞吐^[5,8]。

CPU-GPU 数据传输与阶段同步问题 候选召回返回的不只是 candidate ID，还包括 distance、local rank 以及 reference/index mapping。若这些字段回到 CPU 后由主机侧 pack_candidates 重新组织，执行路径会变为“GPU 检索 → D2H → CPU 候选重组 → H2D → GPU 评分”。问题出现在阶段边界。一次检索即使足够快，两轮数据搬运、对象转换和同步仍会抬高端到端时延。问题 3 因此聚焦设备侧候选表达和连续执行，而不是把 CPU-GPU 通信概括为无法定位的附加成本^[3-4]。

多 GPU 扩展中的结果聚合与共享 I/O 瓶颈问题 索引复制 (replicated) 使每张 GPU 保留完整索引并独立处理查询，设备间依赖较少，适合吞吐优先且索引可以复制的条件；索引分片 (sharded) 有利于容量扩展，却需要合并各卡局部结果。无论采用哪种组织方式，多张 GPU 都可能竞争同一 PCIe 交换路径、NUMA 远端内存或 SSD 通道。设备数量增加后，通信、归并和 I/O 排队可能同时增长，所以吞吐不会自然线性扩展。问题 4 研究不同部署路径的适用条件及其代价，不预先声称所有多 GPU 瓶颈已经被解决^[6,9]。

表 1-1 给出四个问题与后续章节的对应关系。表中指标均来自现有实验设计，没有新增实验或数值。

1.2.3 本文研究边界与主线

FlashTOD 不是新的异常检测理论模型。本文保留 PyTOD 的评分能力，并研究它与候选召回前端之间的系统接口、数据驻留和执行顺序。FlashANNS 同样属于已有 ANNS 系统；本文使用其候选检索能力，不把异步 I/O 机制或索引算法归为本文原创。RAPIDS、FAISS-GPU、页锁定内存和 NCCL 也只在 FlashTOD 的组织、适配和实现中发挥作用。

表 1-1 核心问题、处理方式、对应章节与验证指标

核心问题	本文主要处理方式	对应章节	主要验证指标
异常评分复杂度	候选召回压缩评分范围, PyTOD 在候选集合上执行张量化评分	第三章、第六章	PR-AUC、Recall@ k 、QPS
外存候选召回开销	接入 FlashANNs 异步 I/O—计算流水线, 联动控制候选预算	第三章、第四章	检索阶段耗时、QPS、I/O 通道配置影响
CPU-GPU 往返	GPU 主导数据路径, 设备侧组织 ID、distance、rank 与索引映射	第四章、第六章	查询准备与候选组织耗时、QPS、p99 延迟
多 GPU 扩展损耗	replicated 主方案、设备侧归并兜底路径与拓扑感知 I/O 控制	第五章、第六章	QPS、p99 延迟、资源利用率、扩展倍数

当前系统范围是单机单 GPU 与单机多 GPU。多机分布式索引、跨机通信和集群调度没有进入本文验证。实验使用公开真实或半真实金融数据检验两阶段链路的可用性, 并使用高保真合成数据观察大规模吞吐、时延和扩展趋势; 这些输入不能等同于银行生产流量。现有外部基线覆盖也不完备, 结论只适用于本文的数据、硬件、候选预算和实现路径。全文的主线因而限定为三部分: FlashTOD 协同框架、单 GPU 数据路径优化和单机多 GPU 扩展。

1.3 研究目标与技术路线

本文的目标是在既定检测方法和硬件条件下, 降低候选召回—异常评分链路的系统开销, 并检验该链路在单机 GPU 环境中的扩展边界。检测效果、QPS 和尾延迟需要共同观察, 但本文不把“可在实验环境运行”直接等同于生产可部署。

评分范围是第一个需要压缩的对象。第三章先把金融数据整理为统一向量输入, 以 FlashANNs 产生候选集合, 再由 PyTOD 完成候选集上的张量化评分, 对应问题 1 和问题 2。该阶段建立功能链路, 并通过 PR-AUC、Recall@ k 等指标检查候选预算是否造成不可接受的效果损失。

候选集合生成后, 新的瓶颈出现在模块边界。第四章把查询表示、candidate ID、distance、local rank、索引映射和评分中间张量尽量保持在设备侧, 并利用异步 I/O、双缓冲和必要的页锁定内存减少等待。这里处理的是问题 2 和问题 3。RAPIDS 与 FAISS-GPU 只承担数据后端和前期接口验证角色, 相关技术背景留在第二章说明。

当单卡链路得到压缩后, 多 GPU 扩展才具有实际意义。第五章比较 replicated 主方案、容量受限时的设备侧归并路径以及共享 I/O 约束, 对应问题 4; 第六章在统一实验口径下汇总检测效果、阶段耗时、QPS、p99 延迟和资源利用率。表 1-2 概括了这条路线的实现层次。

表 1-2 本文技术路线的实现层次

技术路线环节	核心任务	对应章节
统一建模与向量输入	对齐表格型、图金融数据与两阶段接口	第三章
候选召回与张量化评分	控制候选预算，建立 FlashANNS—PyTOD 功能链路	第三章
单 GPU 数据路径	压缩 H2D/D2H、候选重组和 I/O 串行等待	第四章
单机多 GPU 扩展	组织 replicated、设备侧归并和拓扑感知 I/O 路径	第五章
统一实验验证	检验效果、吞吐、时延、资源利用率与扩展边界	第六章

1.4 本文主要工作与技术贡献

本文完成的工作沿三条主线组织。候选接口、RAPIDS 数据处理、FAISS-GPU 验证以及 NCCL 通信不是独立的理论贡献，而是相应主线中的工程组成部分。

1. FlashTOD 协同框架。在金融大规模异常检测的统一向量输入上，本文将已有 FlashANNS 候选检索能力和 PyTOD 张量化评分组织为两阶段执行链路，并明确查询向量、候选 ID、distance、local rank、reference/index mapping 与评分张量之间的接口。贡献落在系统中的组织、适配、实现与验证，不涉及重新提出 PyTOD 或 FlashANNS。
2. 单 GPU 数据路径优化。本文针对“GPU 检索—CPU 重组—GPU 评分”的折返路径，设计 GPU 主导执行链路，使候选结果和评分中间张量尽量保持为设备侧对象，并结合异步 stream、双缓冲、页锁定内存及 RAPIDS 数据后端降低必要边界开销。FAISS-GPU 用于前期 device/host pointer 路径和候选接口验证，不作为最终外存召回主路径。
3. 单机多 GPU 扩展。本文在索引可复制时采用 replicated 查询级并行，以 CPU 控制面完成请求调度；容量受限时讨论设备侧归并和 NCCL 通信，并用 topology-aware I/O 约束共享 PCIe、NUMA 与 SSD 路径。该部分贡献是这些机制在 FlashTOD 中的部署组织和实验验证，不把通用多 GPU 技术本身归为原创。

1.5 论文组织结构

全文从四个核心系统问题展开。第三章先完成 FlashTOD 的任务建模、候选召回和评分链路，主要处理问题 1 和问题 2；第四章进入单 GPU 热路径，继续处理

外存召回以及 CPU-GPU 阶段边界，对应问题 2 和问题 3；第五章讨论 replicated、设备侧归并和共享 I/O，对应问题 4。第六章把四类问题放回统一实验口径下验证，第七章总结结论与研究边界。图 1-4给出了问题与章节之间的逻辑关系，它强调研究推进顺序，不重复图 1-3的业务流程或表 1-2的实现层次。

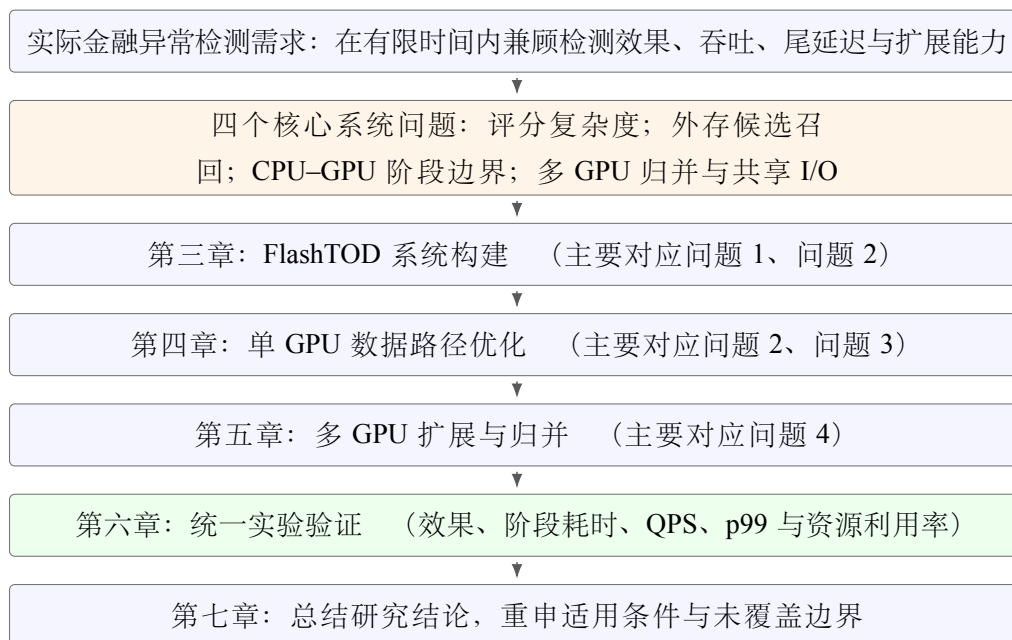


图 1-4 全文研究内容与章节逻辑关系图（本文绘制）

第二章 相关研究工作与关键技术基础

本章沿 FlashTOD 的执行链路梳理相关工作，不按地域拆分。讨论的重点不是罗列技术名词，而是判断已有方法解决了哪一段问题、能够提供什么能力，以及这些能力接入候选召回—异常评分链路后还留下哪些系统边界。

2.1 异常检测方法研究现状

异常检测方法通常包括距离型、密度型、统计型、集成型和深度学习型路线^[1,10-11]。在金融数据上，单笔交易偏离、账户时间窗行为变化和多实体关系异常的表现形式不同，方法选择也不能只看离线精度。评分计算复杂度、近邻依赖、GPU 批处理适配性、可解释性和在线维护代价都会影响系统能否进入近实时链路^[2,12-13]。

k NN、LOF 和 ABOD 等方法依赖距离或邻域结构。 k NN 需要计算并筛选近邻，LOF 还要在局部邻域上比较密度，ABOD 则引入样本间角度关系。这类评分与偏离正常行为的业务解释较为接近，`cdist`、`topk` 和局部聚合也便于组织为 GPU 批处理；代价是近邻计算会随样本规模迅速增长^[7,14-15]。输入向量可能不大，全量距离矩阵和成对差分张量却可能先耗尽显存。候选召回因此不仅是检索加速手段，也是在进入这些评分方法前控制中间结果规模的系统约束。

HBOS、COPOD 和 ECOD 等统计型方法通常不需要维护完整近邻结构，单次评分较轻，适合快速基线或辅助筛查。它们的代价同样明确：边缘分布、直方图或分位统计难以覆盖所有关系型和局部结构异常，对特征预处理与分布假设也更敏感。密度型方法处在两者之间，局部解释较直观，但近邻构造和邻域聚合仍然昂贵。表 2-1 比较的是这些取舍，而不是给出统一的优劣排序。

集成方法通过多个检测器或子空间模型提高稳健性，却会叠加推理成本与结果组织复杂度。深度模型能够表达非线性、时序和图关系模式，但训练、部署和在线解释成本更高；金融图异常检测还受到标签不平衡、噪声联系和数据构建方式影响^[16-18]。这些方法适合不同任务，不能仅因模型复杂就认定更适合高吞吐部署。

本文不沿“最先进异常检测模型竞争”的路线展开。研究对象是系统协同：选择能够张量化的异常评分，把它放在规模受控的候选集合上运行，并观察效果与系统开销之间的折中。该边界保留了评分方法的可替换性，也避免将 FlashTOD 的系统收益解释为某个新检测模型的贡献。

表 2-1 各类异常检测方法复杂度与系统适配性比较

方法类别	代表方法	计算与部署取舍	可解释性	与候选召回的衔接
距离型	<i>k</i> NN、ABOD、COF	依赖距离和排序，GPU 批处理适配较好，但全量中间结果开销高	强	候选集合可直接限制距离计算范围
密度型	LOF、LOCI	需要近邻结构与局部聚合，邻域组织成本随规模增长	较强	可在候选邻域上评分，但依赖候选覆盖
统计型	HBOS、COPOD、ECOD	单次评分较轻，对预处理和分布表达较敏感	中等	可直接用于候选评分或快速基线
集成型	特征装袋、检测器集成	稳健性较强，多个评分器会增加推理和结果组织开销	中等	能够衔接，但候选集上仍需执行多个检测器
深度学习型	Autoencoder、GNN、时序模型	表达能力强，训练、部署和在线维护成本较高	较弱	需要额外模型接口，不是本文评分主线

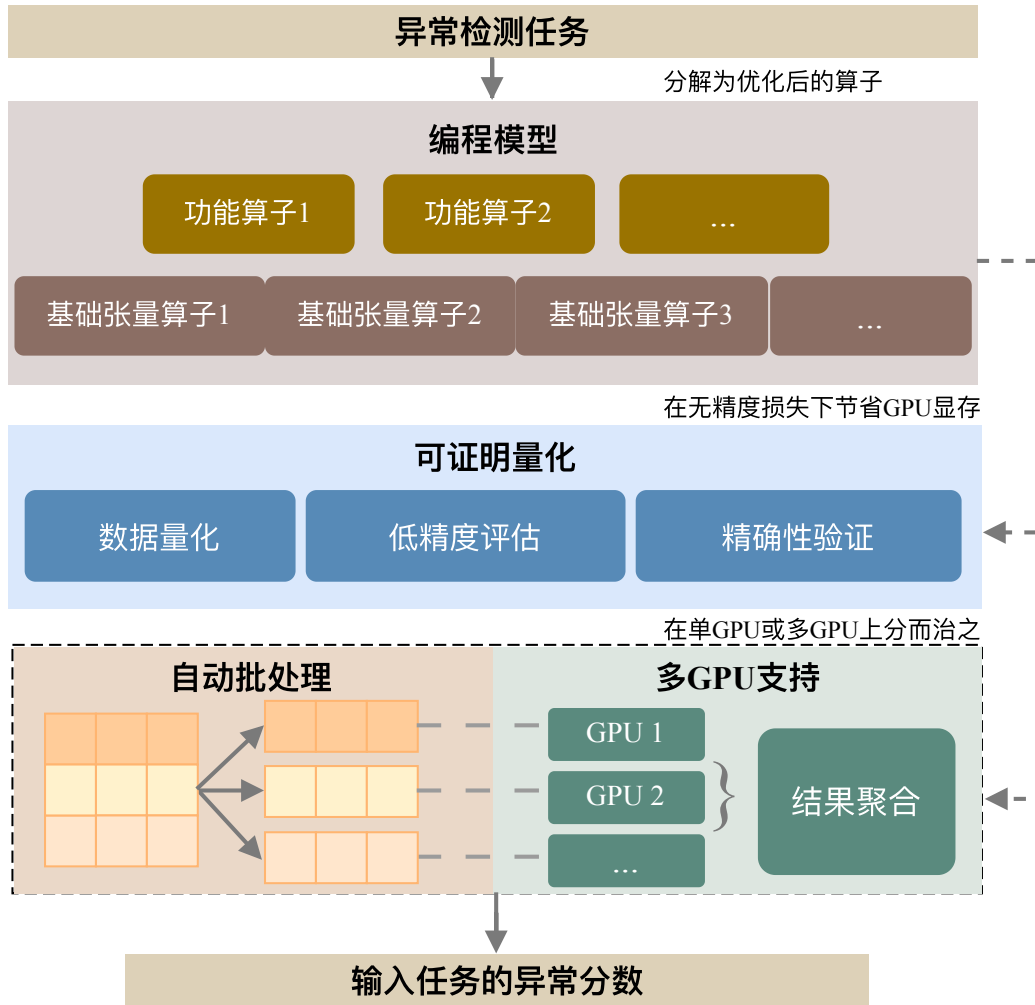
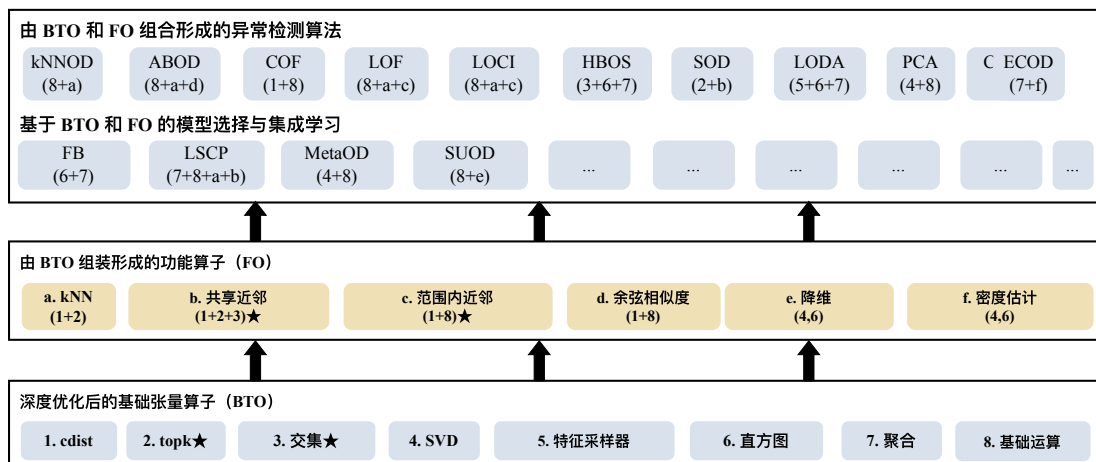
2.2 TOD/PyTOD 的 GPU 张量化执行基础

TOD 解决的是异常评分如何统一映射到 GPU。它把算法拆为基础张量算子 (Basic Tensor Operations, BTO) 和由基础算子组成的功能算子 (Functional Operations, FO)，再用这些算子表达具体检测器^[7]。以 *k*NN 类评分为例，一条典型算子链可以写成 `cdist` $\rightarrow$ `topk` $\rightarrow$ `local aggregation` $\rightarrow$ `anomaly score`。距离、近邻筛选和聚合共享统一张量表示后，不再需要为每个检测器单独维护一套 GPU 实现。

图 2-1 展示 TOD 的分层结构，图 2-2 给出典型算法到算子的映射。自动批处理把超出显存预算的评分任务拆为可执行批次；可证明量化在其约束内压缩部分算子开销；多 GPU 支持允许批处理任务分配到多个设备。这些机制提高的是评分阶段的可执行性。它们并不改变 *k*NN、LOF 等方法对距离和邻域的基本依赖，也不能取消候选预算与检测效果之间的取舍^[7]。

PyTOD 将上述张量化思想提供为可复用的软件接口。本文使用它承接候选集合上的距离、排序和聚合评分，并保留评分函数可替换的空间。早期 GPU 异常检测工作已经证明特定检测器或特征选择步骤可以并行化^[19-20]；TOD/PyTOD 的差别在于提供较统一的算子抽象和批处理执行基础。

它的能力边界也很清楚。PyTOD 不负责从亿级或外存索引中产生候选，不管

图 2-1 TOD 系统总览图（根据文献^[7]整理）图 2-2 典型异常检测算法到张量算子的映射（根据文献^[7]整理）

理 SSD—GPU 访问路径，也不定义候选 ID、distance、local rank 和索引映射如何跨模块传递。在线请求调度与多 GPU 共享 I/O 同样不属于评分框架本身。FlashTOD 因此把 PyTOD 放在候选召回之后：利用其 GPU 评分能力，但不要求它承担前端检索和端到端系统调度。

2.3 FlashANNS 近似最近邻候选召回前端

FlashANNS 是近似最近邻搜索（Approximate Nearest Neighbor Search, ANNS）系统，不是异常检测算法。它在高维向量索引中返回近邻候选，并不输出异常分数。本文把这项能力放在两阶段链路前端：FlashANNS 生成局部候选集合，PyTOD 再根据距离、密度或邻域统计产生异常评分^[8]。

当索引无法完全驻留显存时，搜索过程需要在 SSD 访问与 GPU 计算之间协调。FlashANNS 采用 dependency-relaxed asynchronous I/O，将数据 Fetch、距离 Compute 和候选 Update 交叠推进，并利用 warp 级并发组织细粒度搜索任务。图 2-3 展示了这条流水线。其价值不在于消除 SSD 延迟，而在于让部分已就绪计算不必等待全部读页完成，从而减少 GPU 的串行等待^[5,8]。

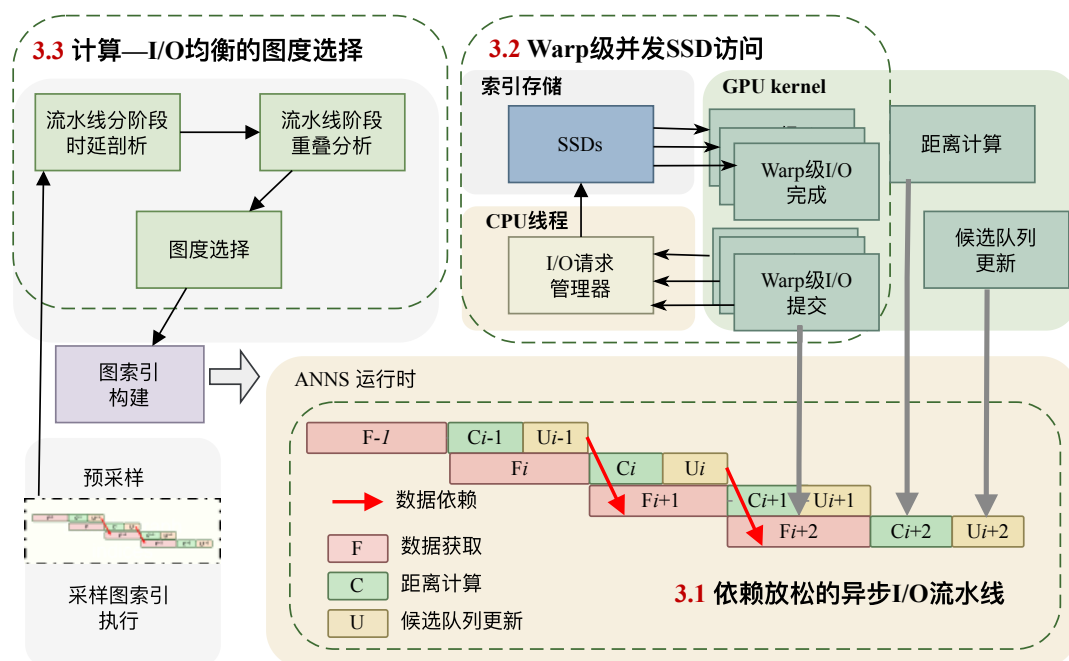


图 2-3 FlashANNS 异步 I/O—计算流水线原理（根据文献^[8]整理）

这一能力适合作为 FlashTOD 的候选召回前端，但有三项限制。其一，FlashANNS 只返回近邻候选，异常分数仍由评分模块计算。其二，召回质量依赖搜索深度和候选预算；预算过小可能漏掉评分所需邻域，预算过大会增加后续计

算。其三，检索内部的高吞吐不等于端到端高吞吐。若 candidate ID、distance、local rank 和 reference mapping 仍需 D2H 回到 CPU 重组，再经 H2D 送入 PyTOD，阶段边界会重新暴露数据搬运和同步成本。

本文不改写 FlashANNS 的索引或搜索算法。后续工作的切入点是接口组织：使前端输出尽可能保持为设备侧候选缓冲，并与评分张量的布局、批次和生命周期对齐。第四章的数据路径优化由此获得明确问题来源。

2.4 GPU 数据处理、FAISS-GPU 与接口支撑

RAPIDS 服务于 GPU 侧数据处理。cuDF 用于字段筛选、类型转换、特征拼接和窗口统计，RMM 为设备内存池及必要的页锁定主机缓冲提供资源管理支持^[21-23]。在 FlashTOD 中，这些能力主要用于向量输入准备和候选中间结果组织，不承担异常评分。RAPIDS 也不是自动的“全 GPU”保证：兼容模式未覆盖的操作、Python 对象转换或显式主机接口都可能触发 CPU fallback，继而产生额外搬运。第四章需要根据实际执行路径检查这些边界。

FAISS-GPU 提供成熟的 GPU ANN API 和资源管理方式，适合验证查询输入与候选输出接口^[24-25]。同一 GPU 上的 device pointer 可以直接进入索引，避免额外 H2D；host pointer 或跨设备输入则会触发相应数据搬运。其候选 ID、distance 返回格式以及 StandardGpuResources 的 scratch memory 管理，使本文能够在前期确认“检索输出如何进入 PyTOD”这一接口，而不必先依赖完整外存链路。

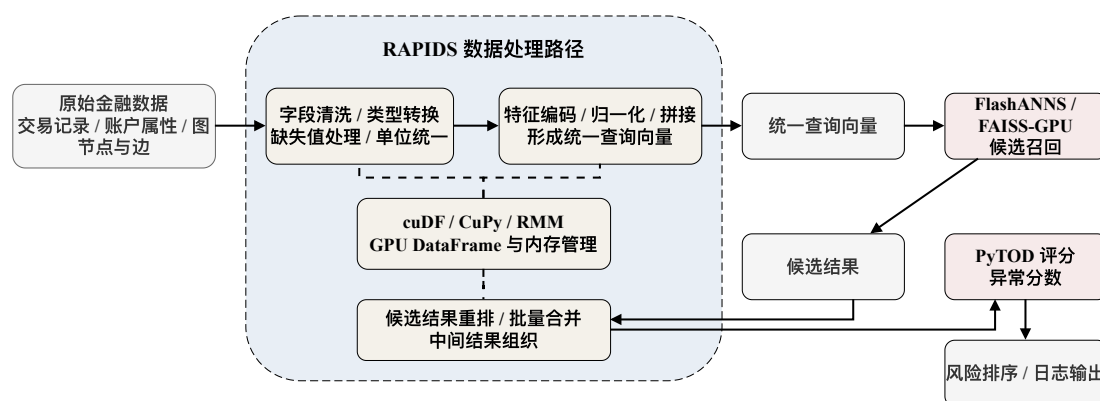


图 2-4 RAPIDS 在本文系统中的数据处理位置（本文绘制）

三者的边界可以简要概括。RAPIDS 负责数据处理和内存后端，FAISS-GPU 负责前期 ANN 接口及 device/host pointer 路径验证，FlashANNS 负责完整 FlashTOD 中的大规模外存候选召回。FAISS-GPU 不是最终外存主路径，FlashANNS 也不替代 RAPIDS 的表格数据处理。把这一区分集中说明后，后续章节只需引用相应角

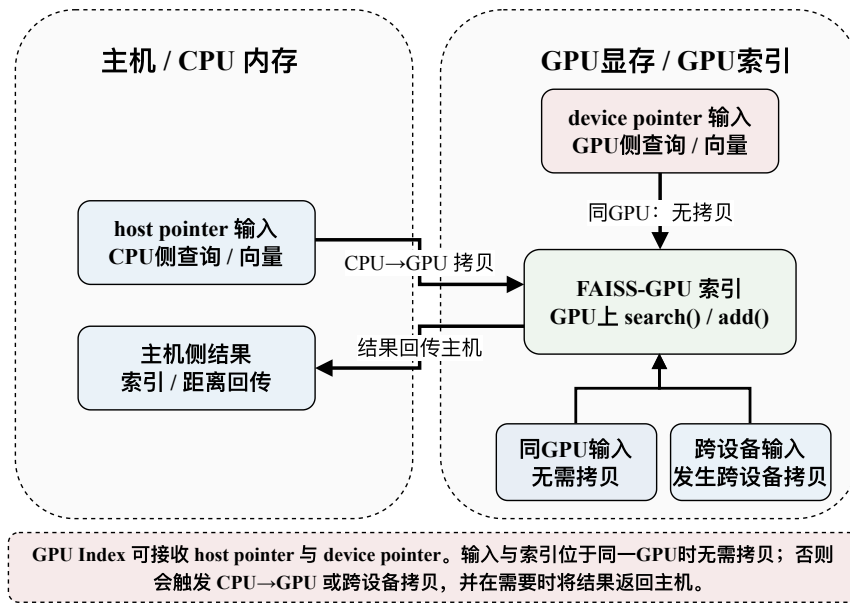


图 2-5 FAISS-GPU 的 CPU/GPU 互操作路径（根据 FAISS-GPU 文档与本文系统接口整理）

色，不再重复论证。

2.5 多 GPU 调度与在线推理启示

多 GPU 设计要先在 replicated 与 sharded 之间取舍。replicated 在每张 GPU 上保留完整索引，把不同查询分发给不同设备。查询可以沿用单卡闭环，跨卡通信少，适合索引可复制且吞吐优先的条件；代价是每张卡都要承担索引副本和评分缓冲的显存占用。sharded 将索引分布到多张 GPU，容量扩展更强，但单个查询可能需要访问多个分片并归并局部结果。第五章因此把 replicated 作为主方案，把 sharded 相关归并放在容量受限时讨论，而不是认为两者在所有条件下等价^[9,26]。

归并位置决定 CPU 是否重新进入数据平面。CPU aggregation 实现直接，但各卡局部候选或分数需要 D2H，主机排序后还可能发生 H2D 或额外同步。GPU merge 能保留设备侧结果并减少主机参与，代价是设备间通信、缓冲区管理和 CUDA stream 协调。NCCL 操作可异步入队到指定 stream；同一 ncclGroupStart/End() 中混用多个 stream 可能引入全局同步点^[27]。因此，GPU merge 是容量受限路径下的工程选择，不是没有成本的替代方案。

计算资源增加后，共享 I/O 可能成为上限。多张 GPU 若共享 PCIe switch、NUMA 远端路径或同一 SSD 通道，在途请求会相互排队；GDS 可以减少 CPU bounce buffer，但实际传输仍受 GPU、存储设备与 PCIe 拓扑关系影响^[6,28-29]。卡数增加不会自动得到线性吞吐：索引复制占用容量，分片引入归并，CPU 控制面仍需调度请求，共

享路径还会抬高 p99 延迟。第五章的 topology-aware I/O 和准入控制正是针对这一限制。

表 2-2 比较的是文献与成熟技术组件的能力边界，不是实验结果表。TOD/PyTOD、FAISS-GPU 和 FlashANNS 分别对应 GPU 异常评分、GPU ANN 检索接口和外存 GPU ANNS；FlashTOD 则研究这些能力组合后的候选组织、设备驻留和端到端执行问题。第六章另行区分已有评分方法参考、本文构造的工程组合基线以及内部执行路径对照，不将本表各行直接当作同口径实验方案。

表 2-2 已有技术能力边界及与本文关系

技术或系统	主要研究对象	异常评分	候选召回	GPU	外存 ANNS	数据路径协同	多 GPU	与本文关系
传统 CPU 异常检测实现	CPU 异常评分	支持	非统一	否	否	主机内执行	非统一	作为能力参照，本文未新增 PyOD 实验
TOD/PyTOD ^[7]	GPU 张量化异常评分	支持	否	支持	否	评分内部	支持	第六章以现有全量评分结果作为评分参考
FAISS-GPU ^[24-25]	GPU ANN 检索与接口	否	支持	支持	否	host/device pointer 接口	可扩展	作为本文工程组合基线的召回组件
FlashANNS ^[8]	外存 GPU ANNS	否	支持	支持	支持	检索内部流水线	不作为能力前提	本文完整方案的候选召回前端
本文 Flash-TOD	端到端系统协同	经 PyTOD	经 FlashANNS	支持	支持	研究重点	单机多 GPU	本文完整方案，受数据、硬件和候选预算限制

2.6 本章小结

现有工作已经分别提供了 GPU 异常评分、大规模 ANNS、GPU 数据处理和多 GPU 通信能力。它们针对的是不同层面的问题：PyTOD 不产生外存候选，FlashANNS 不计算异常分数，RAPIDS 不定义检测逻辑，FAISS-GPU 在本文中只验证接口，NCCL 也不会自动决定合理的归并路径。把这些边界说清楚，并不意味着已有技术能力不足，而是说明单项能力不会自行组成低开销的端到端系统。

本文要补充的研究缺口，是候选召回与异常评分在统一数据路径中的系统协同。第三章据此建立 FlashTOD 的任务模型、模块接口和基础执行流程；第四章再处理功能联通之后仍然存在的候选重组、H2D/D2H 与 I/O 等待问题。

第三章 FlashTOD 检测模型与数据集准备

3.1 金融异常检测任务建模

金融大规模异常检测任务的本质，是在高维、多源、强时序的交易与行为数据中，快速识别少量偏离正常模式的高风险样本。与一般的二分类任务不同，这类任务不仅受到标签稀缺、样本极度不平衡等问题影响，还直接受制于系统吞吐、响应时延和设备资源约束。从工程实现角度看，任务建模不能只停留在输入标签和输出分数的概念层面，还必须明确原始金融数据如何进入系统、哪些对象进入候选召回、评分阶段对输入提出什么接口要求，以及输出结果如何支持后续排序、复核与追踪。

从输入形式上看，本文将待检测对象抽象为一个由样本集合 $\mathbf{X} = \{x_1, x_2, \dots, x_n\}$ 组成的金融行为数据集，其中每个样本 x_i 对应一条交易记录、一个账户时间窗行为摘要或一个节点行为表示。每个样本由连续型数值特征、离散型类别特征和时间上下文特征组成，在预处理阶段被统一映射为固定维度向量表示 $v_i \in \mathbb{R}^d$ 。对于图结构金融数据，还需将节点特征与边关系经时间窗统计、邻域聚合或嵌入映射后转换为向量化表示，从而与表格型交易数据形成统一输入接口^[30]。这一抽象为候选召回模块和异常评分模块提供一致的向量接口，使交易级异常、账户级异常与图节点级异常能够进入同一条执行链路。

从系统目标上看，本文的异常检测任务并非直接对全量数据执行精确评分，而是采用“候选召回—异常评分”两阶段处理框架。这样建模的直接原因在于，全量评分在大规模数据条件下会同时放大距离计算、邻域组织和中间结果存储开销，而候选召回可以先把评分对象压缩到规模可控的局部邻域，再由 PyTOD 异常评分模块完成更细粒度的异常判断。第一阶段利用高吞吐近似最近邻检索在大规模样本中快速找到与查询样本最相关的候选集合 C_i ；第二阶段利用 PyTOD 在候选集合上执行张量化异常评分，生成异常分数 s_i 。形式上，可将系统目标写为

$$C_i = \mathcal{R}(v_i, \mathcal{I}), \quad s_i = \mathcal{F}(v_i, C_i), \quad (3-1)$$

式 (3-1) 中， $\mathcal{R}$ 表示候选召回过程， $\mathcal{I}$ 表示候选索引， $\mathcal{F}$ 表示异常评分函数。由此可见，系统最终性能不再由单一评分公式决定，而是由候选召回质量、候选规模控制和评分阶段执行效率共同决定。这也是本文把两阶段结构作为系统建模基础，而不是把它视为单纯实现细节的原因^[7]。

从输出形式上看，系统可产生三类结果：其一是每个样本的异常分数，用于后续阈值判断或人工复核排序；其二是 Top- k 风险样本集合，用于构造重点审查名单；其三是与异常结果相关的候选邻域、关键特征统计及日志信息，用于系统追踪和后续解释分析。在金融风控场景中，这三类结果分别对应在线筛查、风险排序和审计追踪三种实际需求。

图 3-1 对这一统一任务建模过程进行了整体展示。

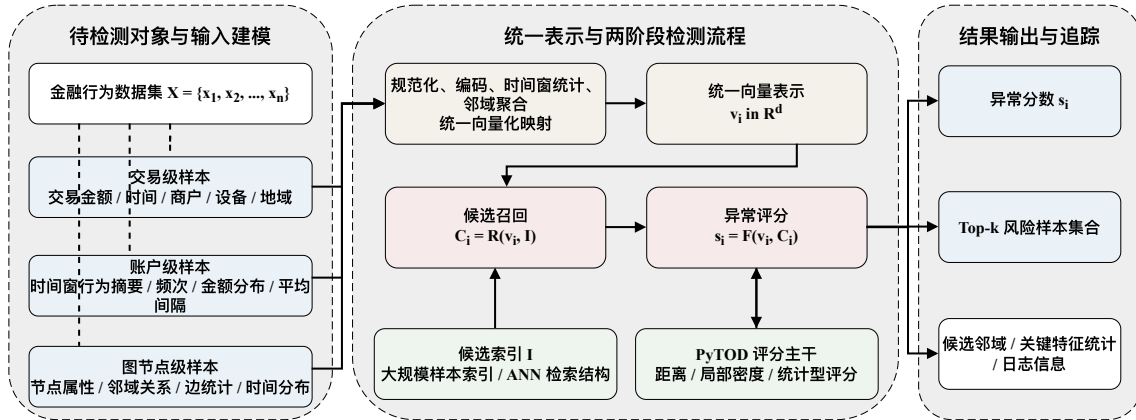


图 3-1 金融异常检测任务建模图（本文绘制）

在该统一建模下，单笔交易记录、账户时间窗摘要和图节点邻域信息不再对应三套割裂接口，而是统一映射为固定维度向量 $v_i \in \mathbb{R}^d$ 。候选召回与 PyTOD 评分据此共享输入，输出也自然收束为异常分数、Top- k 风险对象以及必要的候选邻域和日志信息。

3.2 FlashTOD 系统整体架构

在向量化表示—候选召回—异常评分—结果输出与追踪这一处理链路上，本文选择由 PyTOD 与 FlashANNS 协同构成的异常检测系统作为具体研究对象，并将其命名为 FlashTOD，其中 Flash 取自候选召回前端 FlashANNS，TOD 对应 PyTOD 所沿用的 TOD 路线。PyTOD 将异常检测过程抽象为可复用的张量算子，适合作为异常评分模块；FlashANNS 本身是高吞吐 ANNS 系统，在本文中被组织为候选召回前端，如图 3-2 所示。二者的角色分工相对清晰，但真正落到工程实现时，阶段边界上的开销并不会因为模块职责明确而自动消失。若查询特征先在 CPU 上构造，再传输至 GPU 检索，而候选结果又回到 CPU 重新组织后再送回 GPU 评分，那么瓶颈会很快转移到 CPU↔GPU 数据搬运与阶段同步上；进一步地，在多 GPU 场景中，若将多个 GPU 的局部结果重新集中到 CPU 归并，CPU 还可能再次成为系统瓶颈。

FlashTOD 的总体结构可以划分为输入与特征构造、候选召回、异常评分、GPU 执行支撑、多 GPU 扩展和结果输出几个层次。系统重点不在于重新定义异常检测模型，而在于组织 PyTOD 与 FlashANNS 之间的数据流、候选接口和执行边界，使候选召回与异常评分能够在统一系统链路中协同工作。需要说明的是，图 3-2 中 FAISS-GPU 表示前期 ANN 接口和设备侧检索路径的初步验证组件；完整 FlashTOD 路径中的主要候选召回前端为 FlashANNS。

在输入层，系统接收金融交易记录、账户行为摘要、设备信息以及关系特征等多源数据；在预处理层，这些异构输入经过缺失值处理、类别编码、时间窗统计、图关系聚合和归一化后，被统一映射为固定维度向量表示 $v_i \in \mathbb{R}^d$ 。这一统一向量既是候选召回模块的检索输入，也是后续 PyTOD 评分阶段的基础对象，从而避免为不同数据来源分别维护割裂的执行接口。

在候选召回层，完整 FlashTOD 路径以 FlashANNS 作为主要候选召回前端，面向大规模外存近似最近邻检索提供高吞吐候选生成路径；FAISS-GPU 则主要用于前期验证显存内或局部索引场景下的候选召回接口、设备侧输入输出路径以及候选 ID 与距离返回格式。候选召回层输出候选 ID、局部距离、排序信息以及后续评分所需的数据引用。在异常评分层，PyTOD 接收查询向量与候选集合，在候选范围内完成距离计算、局部密度或统计聚合等张量化异常评分，并生成异常分数。

在数据路径层，FlashTOD 尽量让查询表示、候选缓冲和评分中间张量保持 GPU 常驻，只在原始输入、必要日志和最终结果等边界通过页锁定内存（pinned memory）与主机交互，以压缩 CPU-GPU 数据往返和阶段同步开销。在多 GPU 扩展层，系统优先采用索引复制（replicated）架构，使每张 GPU 独立完成本地候选召回与异常评分闭环；当索引容量或部署条件限制完整复制时，再进入设备侧归并（GPU merge）或主机侧聚合（CPU aggregation）等兜底路径。最终输出层则面向风险排序、异常分数、前 k 个高风险对象（Top- k ）、日志记录与反馈接口，为后续实验评估和工程监控提供统一结果。

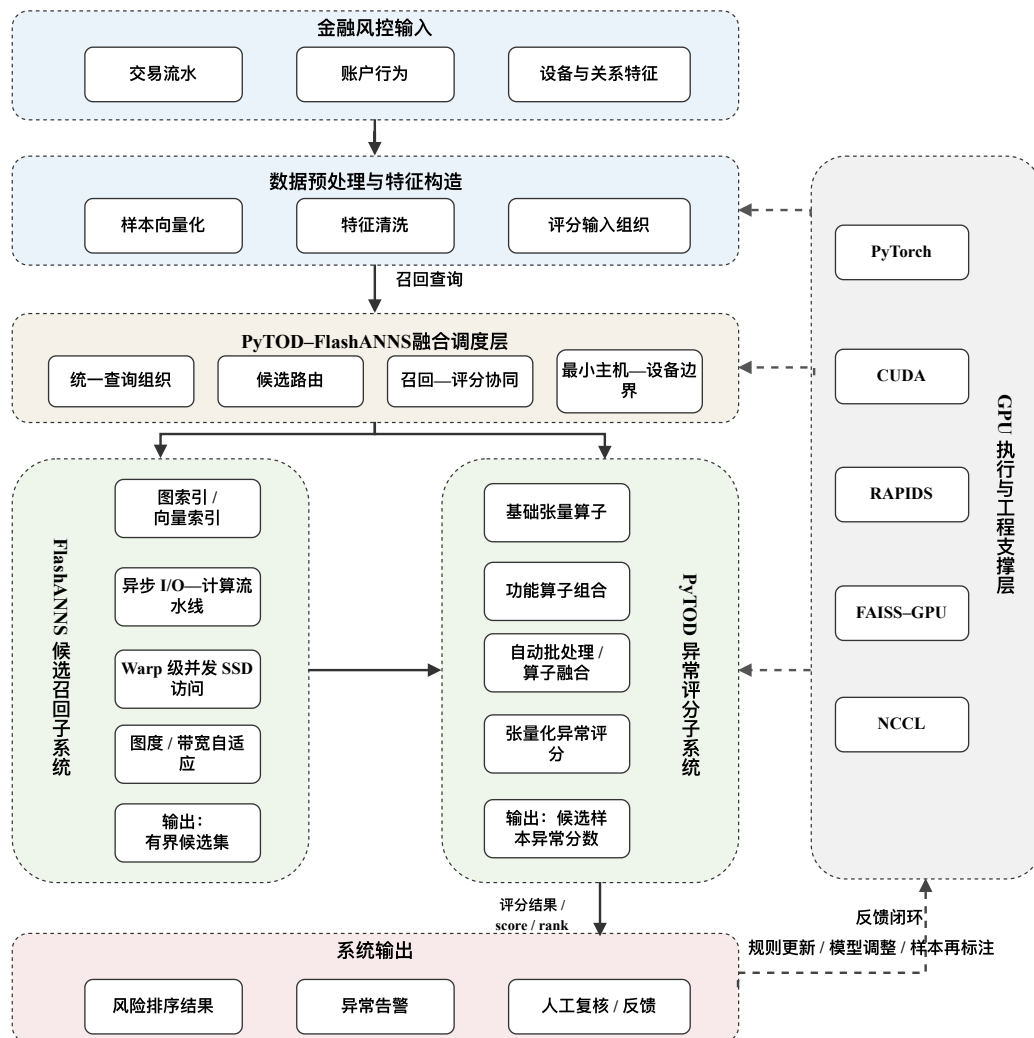


图 3-2 FlashTOD 系统概念图（本文绘制）

3.3 实验数据集选取与构建

本文的数据集安排并不是把公开数据和合成数据简单并列，而是围绕全文实验口径逐层展开。真实或半真实金融数据承担现实场景下的有效性验证；1M 规模合成数据作为桥接层，用于把真实数据上的方法可用性与后续系统实验接到同一条规模链路上；10M 与 50M 规模合成数据主要对应单卡性能实验；多 GPU 扩展与压力测试则继续延伸到 100M 规模^[31-32]。下文若无特殊说明，M 均表示样本数量级，1M、10M、50M 和 100M 分别约对应 100 万、1000 万、5000 万和 1 亿条样本。这样安排的目的，是让真实口径、单卡性能口径和多 GPU 扩展口径之间能够平滑过渡，而不是让某一个数据集同时承担全部验证任务。

本文所称“真实或半真实金融数据”，主要指 IEEE-CIS、Credit Card Fraud Detection、DGraphFin 等公开金融相关数据集。这类数据通常来源于真实业务或真实任务背景，但已经经过脱敏、采样、匿名化、特征变换或竞赛化处理，仍保留金融异常检测中的高维稀疏、类别不平衡、行为偏离和关系结构等典型特征，却不能等同于金融机构生产系统中的在线原始流量。相应地，AMLSim / IBM AML 的 10M、50M、100M 数据主要用于构造可控规模下的压力测试和扩展性验证，适合观察系统吞吐、延迟、I/O 路径和多 GPU 扩展趋势，但不应直接解释为真实生产负载的等比例替代。

在真实口径的有效性验证中，本文主要使用三类公开金融数据。第一类为 IEEE-CIS 欺诈检测数据集（IEEE-CIS Fraud Detection），该数据集包含交易表与身份表两部分，特征维度高、缺失值复杂，较接近现实支付风控场景，适合检验系统在高维结构化金融数据上的向量化表示、候选召回与评分链路是否能够稳定工作^[33]。第二类为 Credit Card Fraud Detection 数据集，该数据集虽规模相对较小，但具有经典性和广泛使用基础，适合提供一组较稳定的对照验证口径^[34-35]。第三类为 DGraphFin 金融图数据集（DGraphFin），其包含大规模节点与边关系以及金融行为特征，主要用于检验图金融数据在经过向量化映射后，能否进入本文统一的候选召回—异常评分链路^[30,36-37]。在具体实验组织时，本文将真实数据实验控制在与 1M 合成数据相近的输入量级，以保证有效性验证与后续桥接实验之间的口径连续。

在 1M 规模桥接实验中，本文采用基于 IBM 反洗钱数据与模拟体系（IBM AML / AMLSim）构建的高保真合成金融交易数据。IBM AML-Data 明确以洗钱与异常金融交易为目标，提供带标签的合成金融交易数据集；AMLSim 则能够根据预定义行为规则、账户关系和交易模式生成大规模、可控分布的银行交易数据^[31-32]。选择 1M 合成规模的原因，不在于它本身构成压力测试，而在于它能够在保留金

融语境和结构一致性的同时，把真实集验证与更大规模系统实验接到同一数据生成体系上。

在更大规模的性能与扩展实验中，本文继续沿用 IBM AML / AMLSim 生成的高保真合成数据。10M 与 50M 主要用于单卡性能验证和中大规模趋势观察；多 GPU 路径则在 10M、50M 与 100M 三个层级上继续展开，其中 10M 用于索引复制主方案的起始扩展口径，50M 用于中间规模观察，100M 用于更强的容量与 I/O 压力场景。与简单随机生成数据相比，IBM AML / AMLSim 体系保留了更强的金融语境和结构一致性，因此可作为系统实验的统一规模底座。

不同数据集在全文中的角色由此也更加清晰：IEEE-CIS、Credit Card Fraud Detection 和 DGraphFin 对应真实或半真实数据口径下的有效性验证；1M 合成数据承担真实集与系统实验之间的桥接；10M、50M、100M 合成数据则分别进入单卡性能、多 GPU 扩展和超大规模压力测试。为便于说明不同数据集在全文中的基本属性，表 3-1 汇总了数据来源与实验规模口径。

表 3-1 实验数据集总体信息

数据集	来源类型	数据形态	实验规模口径	标签形式	在本文中的用途
IEEE-CIS Fraud Detection	真实/半真实公开金融数据	结构化交易表 + 身份表，高维表格特征、缺失值较复杂	约 1M 级输入口径	二分类欺诈标签	真实场景下的主有效性验证，检验系统在高维金融表格数据上的可用性
Credit Card Fraud Detection	真实/半真实公开金融数据	结构化信用卡交易表格，经典欺诈检测基准	按统一实验口径组织到约 1M 级输入	二分类欺诈标签	作为经典基准数据集，用于对照验证两阶段框架的有效性
DGraphFin	真实/半真实公开金融图数据	图节点、边关系及金融行为特征	约 1M 级输入口径	节点/行为异常标签	验证图金融数据在向量化映射后进入统一候选召回—异常评分链路的可行性
IBM AML / AMLSim (1M)	高保真合成金融交易数据	合成账户—交易网络与表格化交易记录	1M	异常交易/可疑行为标签	真实集与系统实验之间的桥接规模，用于统一后续单卡与多 GPU 实验口径
IBM AML / AMLSim (10M)	高保真合成金融交易数据	合成账户—交易网络与表格化交易记录	10M	异常交易/可疑行为标签	单卡性能验证主规模，同时作为多 GPU 索引复制扩展起始规模
IBM AML / AMLSim (50M)	高保真合成金融交易数据	合成账户—交易网络与表格化交易记录	50M	异常交易/可疑行为标签	单卡中大规模性能验证与多 GPU 中间规模扩展观察
IBM AML / AMLSim (100M)	高保真合成金融交易数据	合成账户—交易网络与表格化交易记录	100M	异常交易/可疑行为标签	多 GPU 归并、I/O 路径与超大规模压力测试

数据集名称口径说明：需要说明的是，表中公开数据集名称、实验导出文件名和预处理后向量维度并不完全等同。部分实验文件来自公开数据集的预处理导出结果，部分文件来自 AMLSim 或金融合成数据生成流程。为避免将原始字段维度与

表 3-2 数据集属性与实验用途说明

数据集	数据来源与属性	规模口径	原始特征说明	统一表示口径	本文用途
IEEE-CIS Fraud Detection	公开金融欺诈任务数据，经竞赛化与特征脱敏处理	约 1M 级输入口径	交易表与身份表合并后约 434 列，去除 ID 与标签后的字段数随预处理变化	预处理后映射为固定维度向量 $v_i \in \mathbb{R}^d$	真实或半真实口径下的高维表格有效性验证
Credit Card Fraud Detection	公开信用卡交易欺诈数据，特征经匿名化变换	约 1M 级输入口径	30 个输入特征，包括匿名化主成分、交易时间与金额	预处理后映射为固定维度向量 $v_i \in \mathbb{R}^d$	经典金融欺诈基准上的对照验证
DGraphFin	公开金融图任务数据，包含节点、边关系与行为特征	约 1M 级输入口径	17 维节点原始属性，并结合边关系与邻域行为统计	节点及邻域信息聚合为固定维度向量 $v_i \in \mathbb{R}^d$	图金融数据向量化后进入统一链路的有效性验证
IBM AML / AMLSim (1M)	高保真合成金融交易数据，标签与异常比例随生成配置确定	1M	合成账户、交易网络与表格化交易记录	预处理后映射为固定维度向量 $v_i \in \mathbb{R}^d$	真实口径与系统性能实验之间的桥接规模
IBM AML / AMLSim (10M)	高保真合成金融交易数据，主要放大样本规模	10M	合成账户、交易网络与表格化交易记录	预处理后映射为固定维度向量 $v_i \in \mathbb{R}^d$	单卡性能验证与多 GPU 索引复制起始规模
IBM AML / AMLSim (50M)	高保真合成金融交易数据，强化中大规模路径压力	50M	合成账户、交易网络与表格化交易记录	预处理后映射为固定维度向量 $v_i \in \mathbb{R}^d$	单卡中大规模趋势与多 GPU 中间规模观察
IBM AML / AMLSim (100M)	高保真合成金融交易数据，用于更强容量与 I/O 压力	100M	合成账户、交易网络与表格化交易记录	预处理后映射为固定维度向量 $v_i \in \mathbb{R}^d$	多 GPU 归并、共享路径冲突与压力测试

预处理后向量维度混淆，本文在表中分别列出原始数据来源、实验文件名、统一向量维度和实验用途。其中，`fraud / ADBench 13_fraud` 对应 ADBench 中的 `fraud` 导出文件，作为公开金融欺诈数据的预处理实验口径；`campaign / ADBench 5_campaign` 为 ADBench 公开异常检测数据；`fraud_handbook` 来源于金融欺诈样例与合成链路；`amlsim` 对应 IBM AML / AMLSim 生成配置；`finance_seed_200k` 为合成种子或中间调试数据，不作为最终主实验规模。IEEE-CIS 与 DGraphFin 在正文中用于说明真实或半真实金融数据来源和图金融口径，但当前显存估算表只覆盖已经导出的向量文件，因此未单独列入。

说明：公开金融数据集的原始字段维度与进入 FlashTOD 后的统一向量维度并不等同。不同数据集在缺失值、类别编码、数值归一化和图关系聚合方式上存在差异，本文实验在预处理后将其统一映射为固定维度向量 $v_i \in \mathbb{R}^d$ ，作为候选召回和异常评分的共同输入；当前实现中各实验数据的向量维度及全量距离计算显存风险见表 3-2 的补充说明。IBM AML / AMLSim 高保真合成数据主要用于吞吐、延迟和扩展性压力测试，不等同于真实金融机构生产流量。

这种安排将真实有效性验证与系统压力实验分开：IEEE-CIS、Credit Card Fraud Detection 和 DGraphFin 检验两阶段框架在真实或半真实金融数据上的可用性，1M

表 3-2 补充：当前已导出实验文件的向量维度与全量距离计算显存估算

数据集或规模	样本数 n	维度 d	输入矩阵	距离矩阵 n^2	显式差分 n^2d	说明
fraud / ADBench 13_fraud	284807	29	31.5 MiB	302 GiB	8.56 TiB	真实集，全量距离矩阵已超单卡显存
campaign / ADBench 5_campaign	41188	62	9.74 MiB	6.32 GiB	392 GiB	输入很小，但显式成对差分仍不可取
fraud_handbook (1M)	1M	20	76.3 MiB	3.64 TiB	72.8 TiB	合成桥接规模
fraud_handbook (10M)	10M	20	763 MiB	364 TiB	7.11 PiB	单卡与多 GPU 起始性能规模
fraud_handbook (50M)	50M	20	3.73 GiB	8.88 PiB	178 PiB	中大规模压力场景
fraud_handbook (100M)	100M	20	7.45 GiB	35.5 PiB	711 PiB	超大规模压力场景
amlsim (1M)	1M	9	34.3 MiB	3.64 TiB	32.7 TiB	合成桥接规模
amlsim (10M)	10M	9	343 MiB	364 TiB	3.20 PiB	中大规模性能场景
amlsim (100M)	100M	9	3.35 GiB	35.5 PiB	320 PiB	超大规模压力场景
finance_seed_200k	200k	20	15.3 MiB	149 GiB	2.91 TiB	合成种子或中间数据

注：估算均按 float32 计算。输入矩阵按 $4nd$ 字节估算；距离矩阵按 $4n^2$ 字节估算，可视为直接保留全量样本两两距离的最低存储量级；显式差分按 $4n^2d$ 字节估算，用于说明朴素成对差分中间张量的显存风险。实际 PyTOD/TOD 实现会通过批处理、算子组织或候选召回缩小计算范围，本文不将该估算解释为实际运行时固定占用。

IBM AML / AMLSim 衔接真实口径和合成规模，后续更大样本层级则逐步进入单卡性能、多 GPU 扩展和压力测试。

3.4 数据预处理与特征构造

由于不同数据集在字段命名、数据类型、缺失值比例、标签分布以及时间粒度上存在较大差异，原始金融数据无法直接输入候选召回和异常评分模块。在 FlashTOD 中，数据预处理与特征构造的任务，不是为每个数据集分别堆叠一套特征工程，而是建立一套能够同时服务于候选召回和异常评分的统一向量接口。本节将进一步讨论，如何在尽量保留业务语义的前提下，将表格型金融数据和图结构金融数据整理为可检索、可评分、可批处理的向量表示 $v_i \in \mathbb{R}^d$ 。其中， d 由最终预处理脚本确定，并在候选召回与异常评分实验中保持一致。

对于表格型金融数据，预处理过程主要包括缺失值处理、异常字段清洗、数值归一化、类别编码以及时间字段规范化。缺失值处理方面，针对数值型字段采用均值、中位数或业务合理缺省值进行填补，针对类别型字段则引入专门的缺失标识，以避免无效缺失模式被误当作正常类别。异常字段清洗包括删除重复样本、去除明显无意义标识列、统一金额和时间单位等。对于数值型特征，本文采用标准化或归一化处理，降低金额、频率、时差等不同尺度特征之间的量纲影响；对于

类别型字段，则采用 one-hot、频次编码或目标统计前的安全编码方式，以便后续统一映射至数值空间^[33-34]。经过这一轮处理后，表格型交易记录将进入统一向量接口，直接服务于候选召回与评分阶段。

对于时间信息，本文采用滑动时间窗和行为统计相结合的方式构造增强特征。以单个账户、设备或商户为中心，在固定时间窗口内统计交易频次、交易金额分布、平均时间间隔、夜间交易比例、跨地域交易切换次数等行为特征，并将其与原始交易字段拼接形成更完整的查询表示。这样处理的考虑在于，金融异常往往不是由单个字段瞬时决定，而是体现在与历史行为模式的偏离关系上。通过时间窗统计，系统可以在不引入复杂序列模型的前提下，把一部分时序上下文编码到统一向量中，使候选召回与异常评分共享同一组行为特征基础。

对于图金融数据，本文不直接在原始图结构上执行复杂图学习算法，而是先将图节点与边关系转换为节点级向量表示。具体做法包括：对节点原始属性进行规范化与编码；对邻域中边的数量、方向、金额统计和时间分布等进行聚合；将节点本身属性与邻域统计特征拼接形成统一向量。这样处理的重点在于接口统一：DGraphFin 等图金融数据经过映射后，可以与表格型交易数据一起进入同一条候选召回—异常评分链路，而不需要为图数据额外建立一套与系统主流程平行的执行结构^[30]。

本文将数值、时间、类别和图关系特征统一收束为固定维度向量，使不同来源金融数据能够进入同一候选召回与评分接口；字段到派生特征的具体组织见附录 B 补充表 B-1。

3.5 PyTOD 异常评分模块设计

在本文的两阶段异常检测框架中，PyTOD 承担的是候选集合上的异常评分模块角色。它接收的是由候选召回阶段输出的查询向量、候选张量及其索引映射信息，并在这一局部邻域内完成距离计算、近邻排序和异常性聚合，从而生成最终异常分数^[7]。这一用法将 PyTOD 放在系统的精细评分位置，从而把评分阶段的计算边界控制在候选预算范围内。

从执行接口上看，候选集合 C_i 一旦产生，查询样本 v_i 与候选集合中的向量可以直接进入 PyTOD 的计算链路，被组织为批量距离矩阵、局部近邻排序结果与聚合统计结果。PyTOD 评分阶段主要完成四类操作：计算查询样本与候选集合之间的距离；对候选集合按距离进行排序或筛选；对局部距离分布、密度差异或邻域结构执行聚合；根据预定义评分函数输出异常分数。这样组织后，PyTOD 能够成为一个围绕候选集工作的 GPU 评分内核。

在评分指标选择上，为了保证同一套执行路径能够适配不同数据集和业务场景下的异常性定义，本文保留基于距离、局部密度和邻域统计的通用评分接口。本文使用 PyTOD 作为统一评分骨架，同时保留可切换评分函数的能力，避免把固定评分公式直接写死在系统中。

本文对 PyTOD 的使用方式与其原本将大规模全样本直接用于处理异常检测的思路不同。TOD/PyTOD 更强调通过自动批处理和多 GPU 支持缓解全量异常检测的复杂度问题，而本文把它放在候选召回之后，专门负责较小候选集合上的精细评分。需要说明的是，这一调整在没有改变 PyTOD 评分能力的同时，重新限定了它在系统中的职责，使复杂度更多集中在候选召回与数据路径优化这一便于系统优化展开的层面。

图 3-3 给出了 PyTOD 异常评分模块的执行流程，表 3-3 对常用评分方式与张量算子映射进行了归纳。

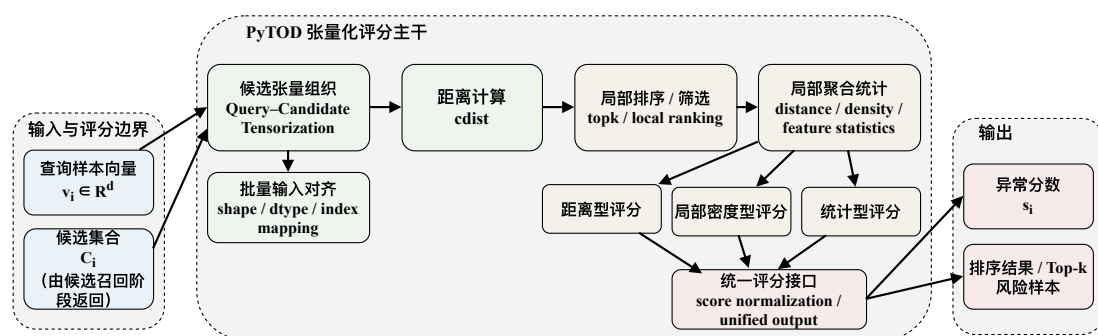


图 3-3 PyTOD 异常评分模块流程图（本文绘制）

表 3-3 异常评分指标与算子映射

评分类型	代表性评分思路	候选集上的计算对象	简化算子链	在本文中的定位
距离型评分	依据查询样本与候选邻域的距离分布刻画异常程度	查询向量与候选集合的局部距离关系	$\text{cdist} \rightarrow \text{topk} \rightarrow \text{聚合}$	主评分方式，直观反映样本偏离程度
局部密度型评分	比较查询样本与邻域样本的局部密度差异	查询样本及其候选邻域的局部密度结构	$\text{cdist} \rightarrow \text{topk} \rightarrow \text{密度比值}$	补充识别相对异常样本
统计型评分	基于候选邻域内特征分布、分位点或边缘偏离程度评分	查询样本在候选邻域中的特征偏离情况	统计/直方图 $\rightarrow$ 聚合	轻量级辅助评分或对照评分方式
融合评分接口	对不同评分结果进行统一输出，形成最终风险排序	距离、密度、统计等候选集评分结果	归一化 $\rightarrow$ 加权输出	体现统一评分模块的可扩展性

3.6 FlashANNS 候选召回前端设计

在 FlashTOD 中, FlashANNS 负责从大规模样本中快速生成与查询样本最相关的候选集合, 为后续精细评分阶段提供规模可控且质量可接受的候选近邻集合^[8]。这意味着, 候选召回模块是评分阶段的前端输入组织模块: 它决定了后续 PyTOD 看到什么样的局部邻域、在多大预算下完成评分, 以及评分阶段需要承担多大计算负担。

在实现演进上, 本文前期利用 FAISS-GPU 较完善的 ANN API 对候选召回接口进行初步验证, 主要用于确认查询向量输入、候选 ID 与距离返回、候选集合传递和 PyTOD 候选评分之间的接口关系。随着系统目标转向大规模外存驻留或超显存索引场景, 完整 FlashTOD 路径进一步以 FlashANNS 作为主要候选召回前端, 以利用其异步 I/O—计算流水线提升大规模候选生成吞吐。

在索引构建阶段, 本文以向量化后的样本表示 v_i 为输入, 构建面向 ANN 检索的候选索引。对于规模较小且全部可驻留在显存内的实验场景, 可直接使用 GPU 端索引完成近似搜索; 对于规模更大或需要考虑外存路径的场景, 则采用 FlashANNS 异步 I/O—计算流水线所支持的高吞吐检索路径。这里的设计重点不是把所有查询都推向最高召回精度, 而是让索引结构和检索路径能够稳定服务于后续评分阶段: 既要让查询以较低延迟定位到局部候选邻域, 又要保留参数调节空间, 使召回质量与系统吞吐之间能够按实验目标进行权衡。

在查询执行阶段, FlashANNS 对输入查询向量 v_i 快速检索并返回一个候选集合 C_i 。在本文系统中, 候选集合不仅包含候选 ID, 还包含必要的距离值、局部排序信息或可支持后续评分的数据引用。之所以要把这些字段一起看作接口设计的一部分, 是因为一旦候选结果还需要在 CPU 侧进行大规模整理、重排和格式转换, 召回阶段获得的吞吐收益就会在模块边界被快速抵消。考虑以上原因, 本文在设计候选召回模块时, 特别强调输出格式与评分模块输入接口的一致性, 即尽量让候选结果以可直接进入 GPU 评分链路的形式表达。

在参数组织方面, 本文并不把候选集合规模、召回近邻数量和索引搜索参数理解为孤立的调参项, 而是把它们视为系统预算约束。候选集合过小, 可能导致真实异常邻域缺失, 从而影响评分质量; 候选集合过大, 则会使后续 PyTOD 评分重新承受高复杂度负担。类似地, 搜索深度过低会使召回不足, 过高则会直接侵蚀系统吞吐。因此, 本文对这些参数的控制逻辑并不是追求极限检索精度, 而是追求对后续异常评分足够好的候选质量, 使候选召回真正服务于系统总体目标, 而不是成为独立优化指标的终点。

图 3-4 展示了候选召回模块的基本执行过程, 表 3-4 总结了关键控制参数。

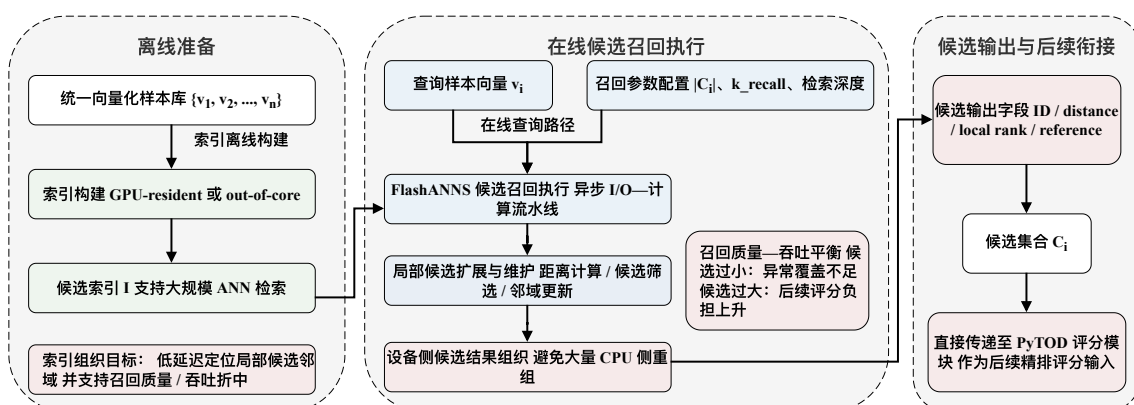


图 3-4 FlashANNS 候选召回执行流程图（本文绘制）

表 3-4 候选召回模块主要参数

参数名称	符号/表示	含义	对系统的影响	设置原则
候选集合规模	$ C_i $	每个查询样本返回的候选样本数	过小会降低异常覆盖率，过大会增加后续评分负担	在召回质量与评分复杂度之间折中设置
近邻返回数量	k_{recall}	候选召回阶段实际返回的近邻数量	决定局部邻域完整性与后续评分的输入范围	与评分阶段使用的局部邻域规模保持协调
检索深度/访问强度参数	nprobe、efSearch 等	控制 ANN 检索阶段访问的索引范围或搜索强度	参数越高通常召回质量更好，但吞吐下降	以满足评分需求为目标，而非追求极限检索精度
索引驻留方式	GPU 常驻 (GPU-resident) / 外存驻留 (out-of-core)	候选索引驻留在显存内或通过外存路径访问	影响检索延迟、I/O 压力与系统扩展能力	小规模优先显存驻留，大规模场景采用外存驻留路径
候选输出字段	ID、distance、rank、reference	候选模块返回给评分模块的主要内容	决定后续 PyTOD 评分接口是否需要额外重组	尽量采用可直接进入 GPU 评分链路的输出格式
查询批大小	batch_size	单次提交到候选召回模块的查询数量	影响设备利用率、时延与流水线连续性	与 GPU 资源、显存容量和时延约束共同确定
精排预算	rerank_budget	进入高精度评分阶段的候选上限	控制精排复杂度与最终检测效果之间的平衡	与候选集合规模联动设置，避免全量重评分

3.7 FlashTOD 执行流程

在完成数据预处理、向量化表示、候选召回模块和异常评分模块设计后，本文进一步将二者整合为统一的执行流程。该流程覆盖从查询输入到异常结果输出的完整链路，其目标是在保持模块职责清晰的同时，把各阶段的输入输出接口明确下来，并尽量减少中间结果的重复复制和格式转换。本文将 FlashTOD 的基础执行流程概括为输入向量化、候选召回、候选张量组织、PyTOD 评分和风险输出五个环节。

具体而言，系统首先接收原始交易记录、账户行为摘要或图节点样本，经由数据预处理与特征构造转换为统一向量表示。随后，向量表示作为查询输入送入候选召回模块：在前期初步验证阶段，该步骤可由 FAISS-GPU 的成熟 ANN API 完成，以验证候选召回与 PyTOD 评分之间的接口；在完整 FlashTOD 路径中，该步骤由 FlashANNs 候选召回前端完成，以支持大规模外存驻留或超显存索引条件下的候选生成。候选集合在 GPU 侧完成必要的组织与重排，形成 PyTOD 可以直接接收的候选张量、距离张量及索引映射结构；PyTOD 在候选集合上执行张量化异常评分，输出异常分数及对应排序结果；最终，系统根据异常分数形成风险样本、日志记录和必要解释信息，并进入后续实验采样或部署监控链路。其中，统一向量接口负责连接异构金融特征与候选召回模块，候选张量组织则负责将候选 ID、距离和排序信息整理为 PyTOD 可直接使用的评分输入。

至此，第三章所完成的主要是功能级衔接：系统知道输入从哪里来、候选如何生成、评分如何执行、结果如何输出，但功能可行并不等于数据路径已经低开销。若候选召回结果仍需频繁回传 CPU 再被整理成评分输入，系统链路就会重新引入高昂的 CPU↔GPU 往返与阶段同步成本；若中间结果能够以 GPU 常驻张量或低开销设备对象表达，则整个候选召回—异常评分过程才更接近统一 GPU 执行链路。因此，第四章的重点不再停留在功能可行性验证，转而继续压缩这条流程在单卡上的数据路径成本。

图 3-5 进一步展示查询向量、候选结果与评分中间张量在模块间的传递方式。

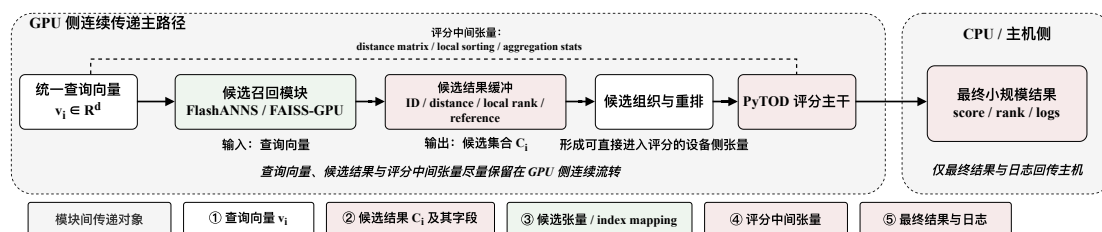


图 3-5 模块间中间结果传递示意图（本文绘制）

图中 FAISS-GPU 表示前期 ANN API 初步验证路径；完整 FlashTOD 路径以

FlashANNS 作为主要候选召回前端。

3.8 基础实验与初步验证

在进入系统级优化和多 GPU 扩展之前，有必要首先验证 FlashTOD 在功能上的正确性，并把真实口径实验与后续 1M 合成桥接实验接到同一条规模链路上。因此，本节的基础实验围绕三个更具体的问题展开：候选召回与异常评分链路是否能够正确联通；在候选集合上执行 PyTOD 评分是否会带来不可接受的检测效果损失；当前初始实现是否已经具备继续进入系统优化阶段的前提。

在功能正确性验证中，实验主要关注查询输入、候选集合生成、候选组织与异常评分输出之间是否存在接口不一致、结果异常或批处理错误。实验安排先以真实或半真实金融数据完成基础联调，再在 1M 合成规模下重复相同口径的检查，以确认候选召回与异常评分接口在跨数据来源条件下仍保持一致。随后，将候选集评分结果与小规模全量评分结果进行比较，以观察候选召回对异常评分口径的影响。实验重点考察精确率—召回率曲线下面积（Precision-Recall Area Under Curve, PR-AUC）、Top- k 召回率（Recall@ k ）和 Top- k 重合度（Top- k overlap）等指标在候选预算受限后的变化，以判断主要排序结果、Top- k 风险样本与整体检测效果是否仍保持在可接受范围内。如果候选集合过小导致异常模式无法被覆盖，则说明候选召回参数仍需调整；如果候选评分与全量评分在主要排序结果上保持较高一致性，则说明两阶段框架具备继续开展系统优化的基础。

基础实验还将初步观察不同数据集下候选召回与异常评分之间的关系。结构化交易数据中的候选召回结果往往对金额、时间和设备行为更敏感，而图金融数据中的候选召回结果则更容易受到邻域统计特征影响。做这一层观察是为了在进入第四章和第五章之前，先确认统一接口和两阶段流程在不同金融数据形态下都能够稳定工作。图 3-6 展示了不同数据集上的候选召回—候选评分初步有效性结果。

本节实验以真实或半真实数据和 1M 合成桥接规模为入口，统一采用固定维度向量表示，并以候选集评分对照小规模全量评分。关注重点是候选预算变化对排序一致性、Top- k 风险样本重合度以及 PR-AUC、Recall@ k 的影响，从而判断两阶段流程是否足以支撑后续系统优化。

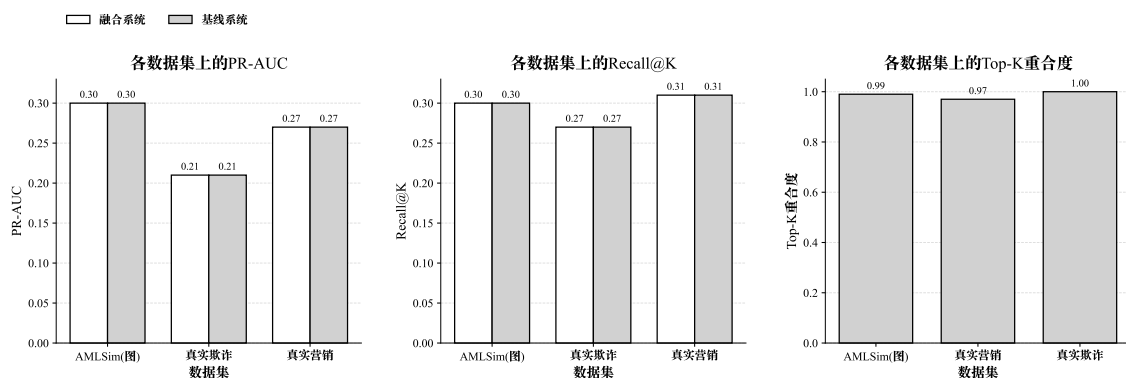


图 3-6 不同数据集上的候选召回—候选评分初步有效性验证结果（实验数据绘制）

3.9 本章小结

本章围绕 FlashTOD 的功能结构，完成了任务建模、数据集选取、预处理与特征构造、异常评分模块设计、候选召回模块设计以及执行流程构建。通过对金融异常检测任务的统一抽象，本文将表格型金融数据与图金融数据共同映射为统一向量接口，并建立了“候选召回—异常评分”的两阶段处理框架。在实验准备上，本章进一步明确了全文统一的数据规模口径，使真实有效性验证、桥接实验、单卡性能分析和多 GPU 压力测试能够沿同一规模链路展开，从而为后续章节中的单卡优化和多 GPU 验证提供一致的数据基础。

本章解决了系统对象如何建立、模块接口如何对齐、基础实验如何起步的问题，但并未真正解决数据路径成本。到本章为止，PyTOD 与 FlashANNS 已经不再是两个孤立模块，而是被放置在一条候选召回—异常评分执行链路中统一考察；然而，这条链路目前主要满足功能可行和接口联通，尚未从系统层面压缩 CPU↔GPU 数据往返与阶段同步开销。因此，第四章将进一步进入单卡优化，重点讨论 GPU 主导执行链路、异步 I/O—计算流水线以及数据路径压缩问题。

第四章 单卡场景下的系统设计与数据路径优化

4.1 单卡系统需求与总体架构

在完成第三章的任务建模、数据准备和 FlashTOD 执行流程设计之后，本节将研究重点进一步转向系统实现层面，即在单 GPU 条件下，如何通过合理的数据路径组织与执行链路压缩，使异常检测系统在保证检测效果的前提下获得更高吞吐率和更低延迟。单卡场景在本文中具有两层作用：一方面，它是多 GPU 扩展的前提，只有单卡链路本身足够高效，多卡扩展才有现实意义；另一方面，它更容易把系统瓶颈完整地暴露出来，尤其是 CPU↔GPU 数据交换、候选结果组织、I/O 等待与算子同步之间的连锁影响。

从系统需求出发，单卡异常检测系统需要同时满足三个目标：在有限显存条件下稳定处理大规模输入，并通过合适的批处理组织支撑候选召回与异常评分连续执行；尽量减少 CPU 对数据平面的直接介入，使查询特征、候选结果与评分中间张量更多停留在 GPU 一侧；在存在外存访问或多阶段流水线的条件下，尽可能重叠 I/O 与 GPU 计算，压缩串行等待时间。由此可见，单卡优化的重点是围绕 GPU 构建一条中间结果传递成本更低的端到端执行链路。

基于这一目标，本文将单卡系统划分为五个核心模块：数据接入与预处理模块、查询向量组织模块、候选召回模块、异常评分模块以及结果输出与日志模块。它们分别承担原始金融记录接入与字段规范化、统一设备侧查询表示构造、候选集合高吞吐返回、候选集上的张量化评分以及最终结果回传与运行时记录等职责。这样的模块划分把热路径和边界路径区分开来：前四个模块构成候选召回到异常评分的核心执行链路，最后一个模块只在必要边界与主机交互。系统总体设计调整之后，能够使系统围绕 GPU 主导路径重新组织它们的衔接关系，避免让这些模块继续沿用“CPU 预处理—GPU 检索—CPU 整理—GPU 评分”的分段模式。

从执行结构上看，本章所研究的单卡系统并不把 ANNS 候选召回前端和 PyTOD 当作两个彼此独立的程序，而是把它们放在同一条执行链路的前后两段：前者在本文系统中先生成候选集合，后者负责精细异常评分，二者之间通过设备侧中间结果表达和统一接口进行衔接。后续各节将围绕这一思路展开，先分析原始链路中的主要瓶颈，再给出 GPU 主导的端到端执行链路；其中，FAISS-GPU 主要用于前期候选召回接口和设备侧路径验证，完整路径接入 FlashANNS 作为主要候选召回前端，单卡优化则围绕候选输出、PyTOD 评分输入、页锁定内存、异步

stream 和双缓冲等机制展开。

图 4-1 展示了单卡系统五个核心模块的连接关系，为后续按数据路径逐层展开分析提供了整体视角。

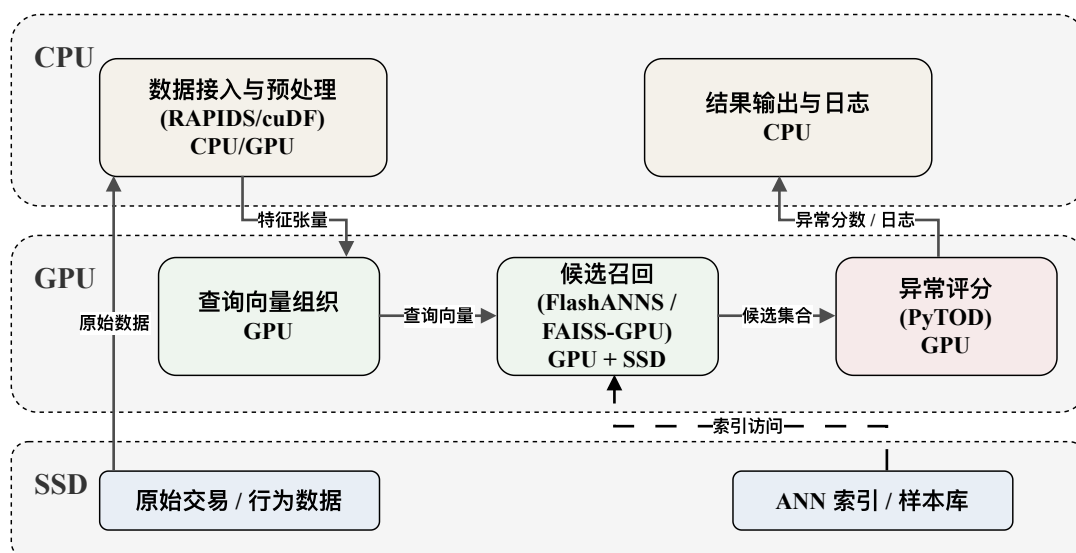


图 4-1 单卡异常检测系统总体架构图（本文绘制）

图中 FAISS-GPU 表示前期候选召回接口和设备侧检索路径的初步验证组件，完整 FlashTOD 单卡路径接入 FlashANNs 作为主要候选召回前端。

4.2 单卡场景下的瓶颈分析

尽管第三章已经完成了 FlashTOD 执行流程的构建，但从系统执行效率看，原始链路仍存在明显瓶颈。问题在于这些阶段的衔接方式仍然保留了典型的“CPU 主导分段执行”特征：查询特征多在 CPU 侧整理，候选结果在 GPU 上产生后又返回 CPU 重组，后续异常评分再重新依赖 GPU。这样一来，原本分散在不同阶段边界上的开销会在 FlashTOD 链路中叠加出现：传输代价增加，同步点变多，等待时间也被层层放大。

首先，主机与设备之间的频繁边界交换会削弱候选召回和异常评分的单点加速收益。GPU 索引既可接收 host 指针，也可接收 device 指针；当输入本来就在与索引相同的 GPU 上时，不发生拷贝且速度最快，否则会发生 CPU→GPU 或跨设备拷贝，结果也会在完成后返回相应位置。这意味着，如果查询特征在 CPU 侧完成拼接和标准化，再逐批传输到 GPU，那么即便 GPU 检索本身很快，整条链路仍要持续承担入口传输成本。若候选结果又在 GPU 侧生成后回到 CPU 被重新整理，再送入 PyTOD 评分，则一次查询实际上会跨越两轮主机—设备边界。这样形成的

折返路径在小规模实验中未必突出，但在百万级输入和高并发场景下会迅速累积，成为系统瓶颈。

其次，候选中间结果的表达方式会继续放大链路成本。候选召回模块返回的通常不仅是候选样本 ID，还可能包含局部距离、索引位置或局部排序信息。若这些结果优先被组织成 CPU 侧更容易解释的数据结构，例如 Python 对象列表、pandas DataFrame 或主机侧稀疏结构，则后续 PyTOD 在 GPU 上评分前还需要重新完成设备侧重建。这样带来的问题不仅是一次额外的数据复制，更在于它打断了 GPU 对中间张量的连续访问与批量组织能力。因此，在单卡系统中，候选结果若不能以设备侧友好的形式表达和传递，评分阶段就很难形成低开销的连续执行链路。

再次，当候选召回需要访问外存或已经存在明显 I/O 压力时，SSD-I/O 与 GPU 计算之间的串行等待会把前两类问题进一步放大。传统外存驻留 (out-of-core) ANN 系统的主要问题之一在于 SSD 访问与 GPU 计算无法有效重叠，系统被迫在“读数据—等数据—算数据”的节奏中推进，从而使高性能 GPU 长时间等待 I/O 完成。而在本文的 FlashTOD 链路中，如果检索内部已经开始重叠 I/O 与计算，但候选召回结束后仍要回到 CPU 完成数据组织和评分前准备，那么被检索阶段隐藏掉的一部分等待会在后续边界重新出现。也正因为这些开销是串联叠加的，单卡优化不能只依赖某一个模块的局部加速，而必须从端到端执行链路出发识别瓶颈。

最后，内存分配与资源组织方式也会在单卡环境下引入隐式开销。StandardGpuResources 会预先保留 scratch memory，以避免频繁 cudaMalloc/cudaFree 导致的同步和性能恶化；页锁定主机内存与内存池机制也有助于稳定传输并减少分配抖动。这说明，即便查询路径和候选结果传递已经更多地留在 GPU 侧，如果系统仍在热路径中频繁申请和释放临时缓冲区，也会因为资源管理本身而造成明显损失。

基于上述分析，本文将单卡场景下的瓶颈概括为四类：查询与候选结果跨边界传递过于频繁、候选中间结果依赖主机侧重组、SSD-I/O 与 GPU 计算之间存在串行等待，以及内存与资源管理带来的隐式同步。这四类问题并不是彼此孤立存在的，它们共同说明单卡系统要获得稳定收益，关键在于压缩整条数据路径。

图 4-2 以并列方式对比了原始 CPU↔GPU 往返链路和优化后的 GPU 主导连续链路，直观说明多次 H2D/D2H 传输和阶段同步如何成为主要瓶颈；表 4-1 则对这些瓶颈的类型与后续优化方向进行了分类。

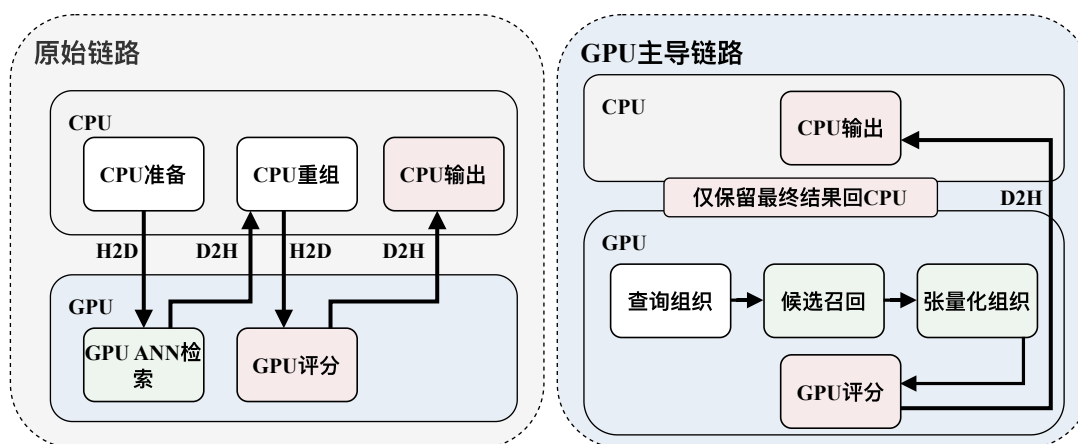


图 4-2 原始链路与 GPU 主导链路对比图（本文绘制）

表 4-1 单卡主要瓶颈与对应优化项

瓶颈类型	发生位置	主要影响	对应优化方案	涉及组件	预期收益
高频 CPU↔GPU 往返	查询准备、候选输出与评分结果边界	放大 H2D/D2H 次数，增加同步等待与尾延迟	GPU 主导执行链路，边界处结合页锁定内存做必要交换	RAPIDS/cuDF PyTOD RMM 页锁定内存	降低跨设备往返频次，压缩尾延迟并提高端到端吞吐
候选重组开销	候选召回到异端评分的接口边界	主机侧 pack_candidates、序列化重建成本高	采用设备常驻候选缓冲（device-resident candidate buffer）和统一张量交接接口	候选召回前端 PyTOD FAISS-GPU 初步接口验证	缩短候选打包阶段，避免 CPU 重新成为数据平面瓶颈
I/O 串行等待	SSD 读页、ANN 图扩展与局部距离计算之间	GPU 空转等待数据，召回吞吐受限	异步 I/O 重叠、双缓冲和流级并行推进	外存 ANNS 前端 NVMe SSD CUDA streams	隐藏部分读页等待，提高候选召回有效吞吐与 GPU 忙碌度
资源分配抖动	临时缓冲区、scratch memory 与主机交换缓冲	频繁分配释放导致延迟抖动和显存碎片	内存池复用、scratch 预分配与微批双缓冲	FAISS GpuResources RMM memory pool CUDA streams	稳定阶段延迟，降低分配开销（allocation overhead），改善资源利用率

4.3 GPU 主导的端到端执行链路设计

针对上一节所分析的瓶颈，本文设计面向单卡场景的 GPU 主导端到端执行链路。这一路径的核心做法是：在接入已有外存 ANNS 前端的基础上，把查询特征组织、候选检索结果传递与异常评分中间张量计算尽量收拢到 GPU 一侧，仅在必要边界通过页锁定内存等机制完成主机交互，从而同时压缩 SSD-I/O 与 GPU 计算之间的串行等待，以及主机—设备边界上的额外搬运。

从总体原则上看，GPU 主导端到端执行链路包含三层含义：查询特征的最终组织尽可能在 GPU 侧完成；候选召回结果以设备侧可直接消费的形式交给异常评分模块；评分阶段产生的中间张量继续在设备侧推进，只有最终分数、排序结果或必要日志信息再回传 CPU。对应到系统行为上，这一路径的真正收益并不是某一次传输被完全消除，而是把原先反复出现的“主机—设备—主机—设备”折返模式压缩为少数必要边界交互。

4.3.1 查询特征在 GPU 侧的驻留与组织

在原始实现中，金融交易数据通常先以表格形式在 CPU 侧完成清洗、归一化和拼接，再整体打包为查询向量传入 GPU。这样组织虽然直观，但会让所有查询都在入口处承担一次完整的 H2D 成本。本文在这一位置的设计决策，是把部分预处理与特征组织前移到 GPU 数据处理后端，使关键查询向量尽可能在设备侧形成最终表示。RAPIDS 的 GPU DataFrame 组件 cuDF，以及与其协同的 GPU 数组计算库 CuPy，提供了批量列运算、张量表达和统计聚合能力，使这一做法在工程上具备可行性^[22]。预期收益也较为直接：查询入口不再承担整批向量的重复搬运，后续候选召回阶段能够更快接收到可检索的设备侧表示。

4.3.2 候选检索结果在 GPU 侧的保持与传递

候选召回阶段结束后，系统面临的关键问题是：候选结果应以何种形式进入异常评分模块。本文在这里的设计决策，是尽量保持候选结果为设备常驻（device-resident）对象，即候选 ID、局部距离和必要索引信息不先回 CPU 再重建，而是在 GPU 上直接组织为 PyTOD 可消费的中间张量。FAISS-GPU 文档指出，同 GPU 上的输入与输出可以避免额外拷贝，且 GpuResources 可通过预分配 scratch memory 降低中间缓冲区分配开销^[25]。沿着这一接口设计，前期验证阶段中 FAISS-GPU 的候选 ID 与距离输出可用于确认 PyTOD 评分输入接口；在完整 FlashTOD 路径中，接入的 ANNS 前端返回候选结果后，系统将其以设备侧对象形式送入 PyTOD 评分阶段，预期收益主要体现在候选打包和阶段切换成本的下降。

4.3.3 异常评分中间张量的 GPU 常驻执行

PyTOD 评分阶段本身具备张量化执行基础，因此在本文设计中，其主要中间表示包括候选距离矩阵、局部排序结果、聚合统计量和最终异常分数。本文在这一位置的选择很明确：中间张量不在计算过程中回到 CPU，而是在 GPU 上连续推进，直到得到最终小规模输出后再按需回传主机。这样做的原因在于，一旦这些对象中途离开 GPU，不仅会增加 D2H 成本，还会破坏 PyTOD 在设备侧连续执行和批量算子复用的优势。预期收益因此不仅表现为传输减少，也表现为评分阶段的局部性和算子复用被更好地保留。

4.3.4 必要条件下的主机侧交互与页锁定内存机制

GPU 主导执行并不意味着 CPU 在单卡系统中完全消失。原始输入读取、部分日志写回、结果落盘以及特定部署场景下的主机接口，仍然要求系统与 CPU 保持交互。因此，这一小节对应的设计决策不是完全取消主机交互，而是把交互严格限制在必要边界，并通过页锁定内存等机制把这些边界开销控制在较低水平。RMM 文档指出，PinnedHostMemoryResource 可提供页锁定主机内存，从而加快 Host↔Device 数据传输^[23]。这类机制的收益主要不是提高某个算子的峰值速度，而是降低必要交互对整条热路径的干扰。

因此，单卡热路径只保留必要的主机边界：查询向量、规范化特征、候选 ID/distance/rank 以及 PyTOD 的距离矩阵、排序和聚合张量尽量在 GPU 侧连续传递；原始输入、最终异常分数和运行日志则以小规模方式回到 CPU。这样既保留系统可观察性，也避免候选到评分之间重新形成主机侧折返。

4.4 候选召回前端的数据路径组织与工程接入

本节进一步讨论候选召回前端如何接入整条单卡热路径。需要先明确的是，FlashANNS 是已有的高吞吐 ANNS 系统，本文并不重新设计其索引结构、搜索算法或异步 I/O 机制，而是将其近似最近邻检索能力组织为 FlashTOD 的候选召回前端。相应地，本节不讨论如何提出 FlashANNS，而关注已有 ANNS 前端返回的候选结果如何以较低边界成本进入 PyTOD 评分模块。

从候选召回前端视角看，系统需要同时处理三类接口：第一，查询向量如何以设备侧对象进入 ANNS 前端；第二，候选 ID、局部距离和排序信息如何保持为 PyTOD 可消费的中间表示；第三，外存访问、GPU 局部计算和评分前准备如何在同一执行链路中衔接。若这些接口仍然依赖 CPU 侧反复打包和重组，即使 ANNS 前端本身吞吐较高，端到端收益也会被阶段边界开销抵消。

4.4.1 外存 ANNS 前端的流水线接入

在大规模或超显存索引场景中，候选召回前端往往需要访问 SSD 等外存介质。已有 FlashANNS 工作通过依赖放松异步流水线（dependency-relaxed asynchronous pipeline）将外存 I/O 与 GPU 计算重叠，从而缓解传统外存驻留检索中的串行等待问题^[8]。本文复用这一能力时，并不把它作为独立模块结果直接呈现，而是将其放入候选召回—候选组织—异常评分的连续链路中，使候选生成阶段的等待尽量不以完整停顿的形式暴露在评分入口前。

具体而言，候选召回前端不再把 SSD-I/O 视为必须先完成的前置步骤，而是将检索中的部分访问与 GPU 侧局部计算、候选维护和评分前准备并行组织。对于 ANNS 前端，这意味着查询在等待部分图节点或向量块从 SSD 到达时，GPU 仍可以继续推进已到达部分的距离计算和候选维护；对于 FlashTOD，则意味着 PyTOD 评分模块能够更连续地接收候选输入，而不是周期性等待整个检索阶段结束。这里的系统贡献在于跨模块衔接：把已有外存 ANNS 前端的流水线能力延伸到后续评分入口，避免局部吞吐收益在候选输出边界被新的 CPU 侧整理开销抵消。

图 4-3 展示了已有 ANNS 前端异步流水线与本文候选召回前端链路的接入方式。

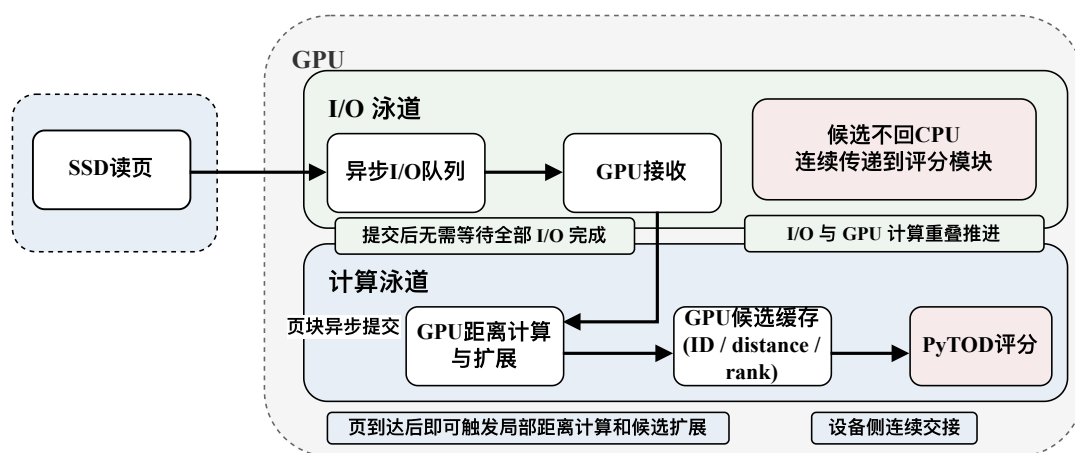


图 4-3 外存 ANNS 异步 I/O—计算流水线与本文候选召回前端接入方式（根据文献^[8]和本文系统实现整理）

这一链路把 SSD 读页等待、页到达后的局部计算和批次切换尽量交叠起来：异步 I/O 队列削弱等待，warp 级并发访问与双缓冲候选缓存维持设备侧连续推进，候选结果则直接进入评分模块，避免重回 CPU 打包。

4.4.2 GPU 数据处理与候选组织

在完成候选召回前端的流水线接入之后，单卡系统仍需要进一步优化 GPU 数据处理与候选组织方式，以减少表格型中间结果处理和设备资源管理所带来的额外开销。为此，本文在原始 PyTorch/CPU 数据路径基础上引入 RAPIDS GPU 数据处理生态，对金融数据预处理、特征拼接、中间结果组织和批量统计环节进行后端重构^[22-23]。RAPIDS 在这里同样不是异常检测算法的一部分，而是用于支撑候选召回前端与评分模块之间的设备侧数据组织。

后端优化围绕热路径展开：字段筛选、类型转换和窗口统计尽量前移到 RAPIDS/cuDF；候选组织、批量拼接和统计聚合转为 GPU DataFrame 或 CuPy 张量表达；临时内存与必要的主机交换缓冲由 RMM 内存池和页锁定主机端（pinned host）资源管理；与 PyTOD、候选召回前端交接时，则优先采用设备侧连续缓冲，避免“DataFrame—Python 对象—Tensor”的反复转换路径^[23]。

RAPIDS 在本文中用于稳定热路径的数据和内存后端，后端重构的直接目标，是缩短 query_prep、pack_candidates 以及检索资源初始化中的对象转换和资源抖动，使候选召回前端返回的结果能够更低成本地进入 PyTOD 评分阶段。

4.4.3 ANNS 检索接口验证与完整路径接入

除了数据处理后端的重构外，ANNS 检索接口本身也需要进一步工程化衔接。候选召回前端并不等价于某一个 ANNS 研究工作本身；它是本文系统中连接统一向量输入和 PyTOD 候选评分输入的接口层。在实现演进中，本文首先利用 FAISS-GPU 较成熟的 GPU ANN API 完成候选召回链路的初步验证。该阶段主要用于确认 GPU 侧查询输入、候选 ID 与距离输出、设备指针路径和 PyTOD 候选评分接口之间的衔接关系。

随后，完整 FlashTOD 路径进一步以 FlashANNS 作为主要候选召回前端，以适应大规模外存驻留或超显存索引条件下的高吞吐候选生成需求。因此，FAISS-GPU 在本文中更接近初步验证和接口参照组件，而 FlashANNS 是本文完整路径中接入的主要 ANNS 前端。本文在接入层上的优化主要体现在三个方面：统一检索输入接口，使查询向量能够以设备侧对象进入候选召回前端；统一候选输出接口，使前期验证阶段 FAISS-GPU 返回的候选结果，以及完整路径中 FlashANNS 返回的候选结果，都能以 GPU 侧张量形式被 PyTOD 消费；统一资源管理策略，通过 FAISS 的 GpuResources 与系统侧缓冲区管理配合，为后续缓冲区申请、设备指针路径和延迟抖动分析提供工程参照^[25]。这些优化的共同目标，是让已有 ANNS 前端更顺畅地嵌入整条热路径，避免把本文工作误解为对某个 ANNS 组件本身的重新提出。

4.5 单 GPU 执行链路的工程增强

在完成数据路径、异步流水线和后端优化后，系统仍然需要若干工程增强机制来保证单卡链路在高负载下保持稳定。这些增强能够补足其稳定性与可观察性，包括异步 CUDA stream 组织、双缓冲与微批执行、显存碎片控制以及日志和性能采样机制。

在流组织方面，本文通过将查询准备、候选检索、评分计算和必要的主机边界交互映射到不同 CUDA stream，使数据传输、检索和评分能够在更大程度上并行推进。在缓冲方面，本文采用双缓冲与微批组织方式：当当前批次进入候选召回与评分阶段时，下一批次的输入组织和部分准备可以并行进行，从而进一步压缩流水线空泡。在显存管理方面，本文通过限制中间张量生命周期、统一缓冲区尺寸以及使用内存池机制减少碎片和反复分配开销。在监控与可观察性方面，本文建立统一日志字段与性能采样机制，记录查询输入规模、候选集合规模、各阶段耗时、GPU 利用率和 Host↔Device 交互次数，为后续章节分析多 GPU 扩展提供统一观测基础。

4.6 单 GPU 实验与性能分析

为了验证前述单卡系统设计与数据路径优化的有效性，本文沿用第三章定义的单卡规模链路组织实验。其中，真实或半真实数据主要用于确认链路可运行和检测效果不失真，1M、10M、50M 三个合成规模主要用于性能、吞吐和压力分析；二者不直接比较绝对吞吐。1M 承担由真实集过渡到系统实验的桥梁角色，10M 与 50M 对应单卡性能分析的主体规模。在这一框架下，本节重点比较四类方案：原始分段执行链路、仅启用候选召回加速的链路、启用 GPU 主导执行链路但不做后端重构的链路，以及本文完整的单卡优化方案。这样的对比口径能够区分不同收益来源：候选召回本身带来的收益、执行链路重构带来的收益，以及后端与工程增强叠加后的整体收益。换言之，这四类方案构成递进式消融实验，用于观察候选召回、数据路径压缩和后端重构在端到端性能中的累积贡献。实验关注的指标包括每秒查询数（Queries Per Second, QPS）、平均延迟与第 99 百分位延迟（p99 延迟）、CPU↔GPU 传输次数与体积、GPU 利用率、I/O 利用率以及各阶段耗时分解。

实验结果显示，单纯依赖候选召回加速虽然能够降低部分检索耗时，但若候选结果仍需回到 CPU 侧整理再进入评分，端到端吞吐提升并不充分。相比之下，当查询特征、候选结果与评分中间张量更多地留在 GPU 一侧后，主机—设备边界交互次数明显减少，系统吞吐率和延迟分位数都表现出更稳定的改善。进一步地，当已有外存 ANNS 前端的异步 I/O—计算流水线与设备侧连续执行链路结合后，候

选召回阶段的等待可以被部分隐藏，评分阶段也更容易连续接收输入，反映在更高的 GPU 利用率上。

从阶段耗时分解来看，优化前系统的主要开销集中在查询组织、候选结果主机侧重组和阶段同步等待；优化后，系统性能不再首先被路径开销拖住，其决定因素开始更多落在 GPU 侧核心阶段，主要耗时更多地集中到设备侧检索与评分本身。

考虑到本节结果展示需要兼顾版面与代表性，图 4-4 与图 4-5 选取 1M 与 10M 两个具有代表性的合成规模展示阶段耗时分解，图 4-6 汇总比较四种方案在 QPS、p99 延迟、PCIe 传输次数与 GPU 利用率上的综合表现。

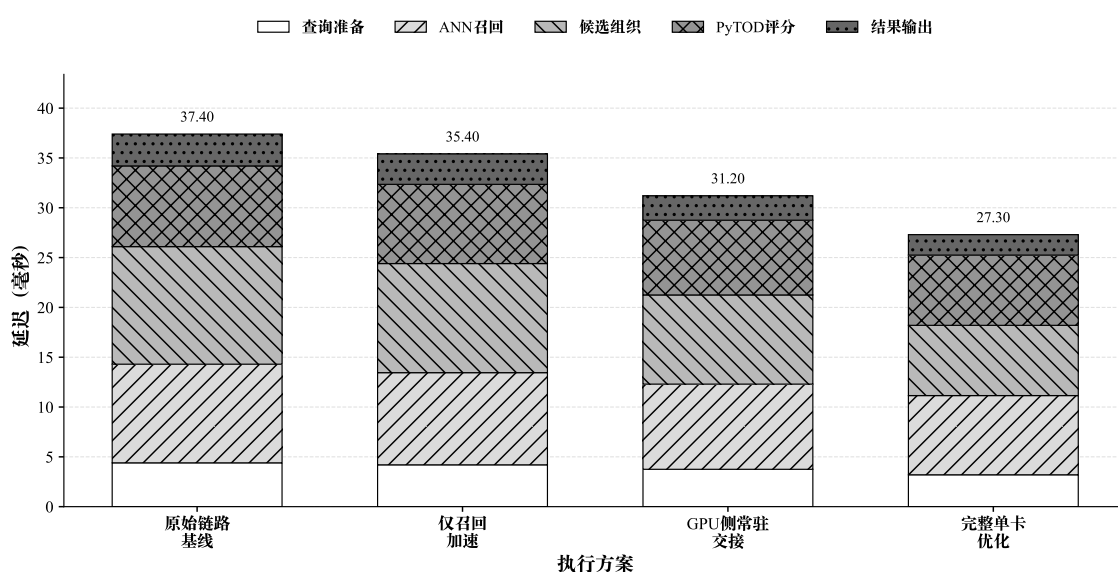


图 4-4 1M 数据规模下单卡各阶段耗时分解（实验数据绘制）

图 4-6 展示的是递进式消融结果，而不是完全独立的单因素消融。其目的在于从原始分段链路出发，逐步加入候选召回加速、GPU 主导执行链路和完整后端优化，以观察主要系统优化项对端到端吞吐、尾延迟、PCIe 传输次数和 GPU 利用率的累积影响。由于设备侧候选缓冲、异步 I/O—计算流水线、后端资源管理和评分张量组织之间存在耦合关系，本文不将其解释为严格单因素实验，而是将其作为单卡链路优化收益来源的递进式验证。

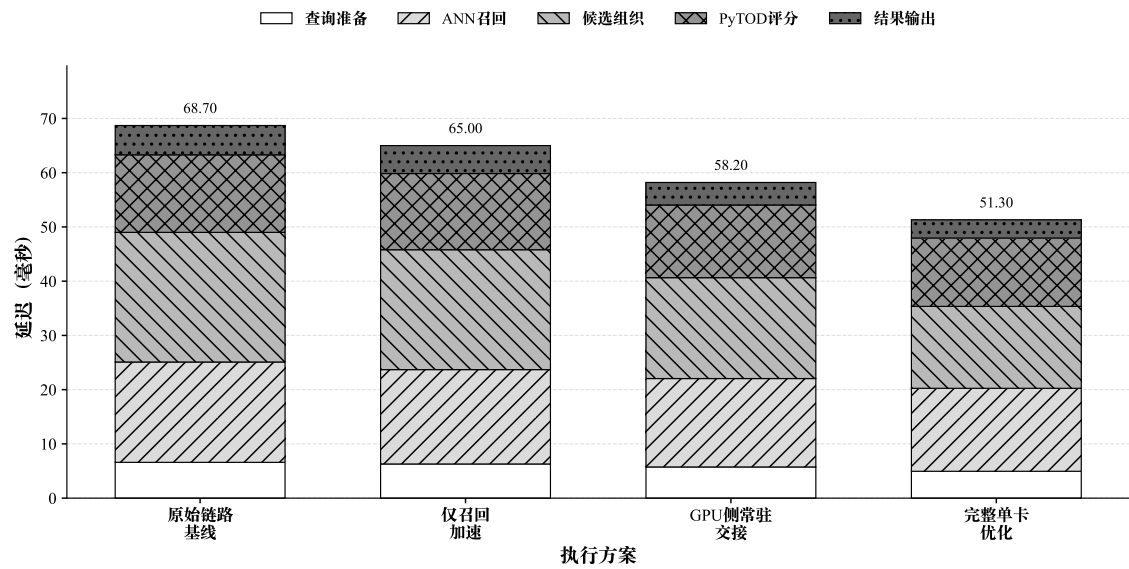


图 4-5 10M 数据规模下单卡各阶段耗时分解（实验数据绘制）

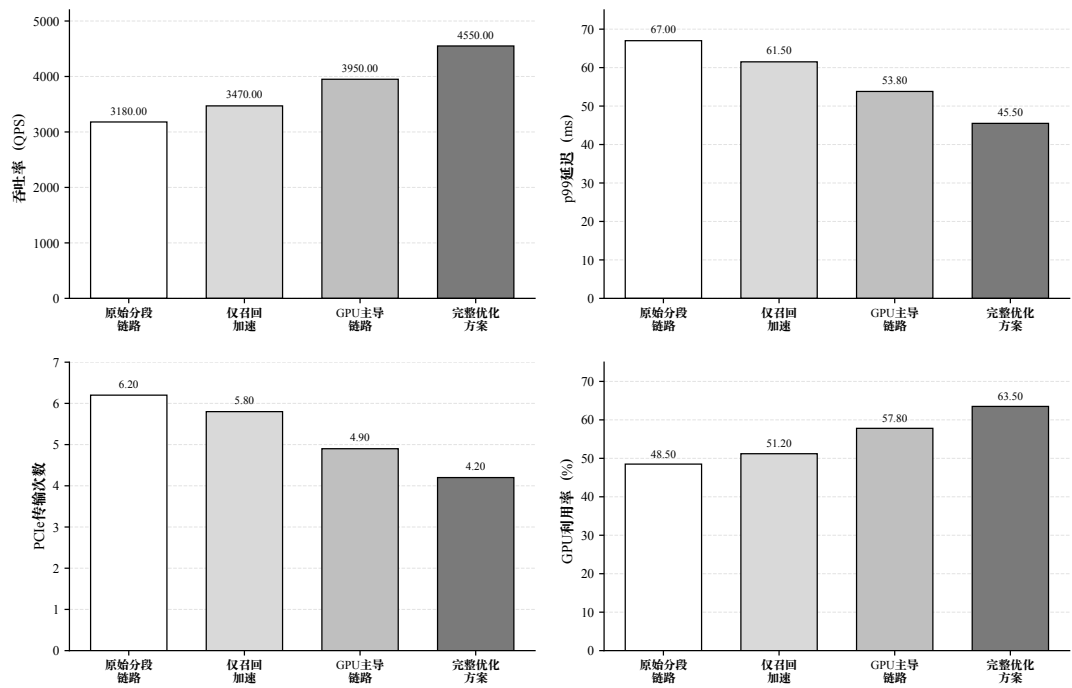


图 4-6 单卡执行链路递进式消融结果（实验数据绘制；该图为递进式消融，非完全独立的单因素消融）

4.7 本章小结

本章围绕单 GPU 场景下的系统实现问题，分析了原始 FlashTOD 链路中存在的边界交换、候选重组、I/O 等待以及资源管理抖动等瓶颈，并在此基础上设计了 GPU 主导的端到端执行链路。本章进一步通过 RAPIDS 重构数据处理路径，基于 FAISS-GPU 完成候选召回接口的初步验证，并在完整链路中接入 FlashANNS 作为主要候选召回前端；相关工作重点是候选召回前端的数据路径组织、设备侧候选缓冲和 PyTOD 评分接口衔接，而不是对 FlashANNS 本身的重新提出。

单卡实验的结果说明，本章的主要收益并不来自某一个局部算子的极限加速，而来自候选召回到异常评分整条数据路径的压缩，以及设备侧连续执行能力的增强。经过这一轮优化后，系统的主要压力已经更多集中到 GPU 侧核心阶段，这也使下一章有必要转向多 GPU 场景：当单卡链路已经被尽量收紧之后，系统吞吐的进一步提升才真正转化为多 GPU 扩展问题。

第五章 多 GPU 并行扩展与归并优化

5.1 多 GPU 扩展需求分析

第四章已经从单 GPU 视角完成了系统级优化，并初步说明在 GPU 主导端到端执行链路、异步 I/O—计算流水线和后端重构协同作用下，单卡场景中的主要损耗已不再集中在主机—设备边界，而更多落在设备侧检索与评分过程本身。然而，这并不意味着单卡已经足以覆盖后续所有实验和部署需求。随着输入规模继续增大、查询并发进一步提升、候选索引逼近单卡显存上限，单卡系统会同时遭遇三类上限：计算上限、容量上限和调度上限。

第一类需求来自计算上限。当候选召回和异常评分都已稳定驻留在 GPU 侧时，单卡吞吐最终会受限于单设备的计算资源。此时，即便继续压缩局部路径开销，系统整体 QPS 仍然难以随业务峰值线性提升。第二类需求来自显存与索引容量上限。在更大索引或更大规模合成扩展数据集条件下，单 GPU 可能无法同时容纳完整索引、副本缓存以及评分中间张量，系统必须在缩小索引规模、降低候选预算和增加设备数量之间进行权衡^[9,38]。第三类需求则来自调度压力。金融近实时异常检测并不只追求平均吞吐，系统还需要在高峰时段保持稳定的延迟分位数和资源利用率；一旦请求在单设备上集中堆积，即使平均性能尚可，尾延迟也会迅速恶化。

因此，对本文系统而言，多 GPU 扩展的目标并不是抽象地提高并行度，而是在保持检测效果、延迟分位数和实现复杂度可控的前提下，解决三个问题：如何把更多查询稳定分摊到多张 GPU 上，如何在容量受限时继续维持索引可用性，以及如何避免调度与共享路径重新成为系统上限^[9]。这些问题一旦进入多卡场景，局部结果如何归并、CPU 是否重新回到关键路径、共享存储路径是否发生拥塞，都会直接影响扩展收益是否能够兑现。

基于这一判断，本文后续优先采用索引复制（replicated）架构主方案，使每张 GPU 尽可能独立完成候选召回与异常评分；只有在索引容量或业务约束明确表明无法复制完整索引时，才进入设备侧跨卡归并与设备间通信的兜底路径。换言之，本章的关注点是在金融近实时异常检测场景下找到更符合吞吐优先和工程可控性的扩展方式，避免发散到构造复杂的多 GPU 检索系统。

5.2 多 GPU 扩展策略设计原则

多 GPU 扩展方案的设计必须建立在系统目标与瓶颈分析的基础上。结合第四章单卡结论与第二章相关工作梳理, 本文将多 GPU 扩展方案归纳为四项基本原则: 查询级并行优于单查询跨卡并行; 数据并行与索引复制优先于直接采用索引分片; CPU 保持在控制面; 当跨卡归并不可避免时, 优先采用 GPU 侧归并。这四项原则并非并列罗列, 其中前两项决定主方案形态, 后两项负责约束兜底路径和运行边界。

首先, 查询级并行优于单查询跨卡并行。若一个查询必须在多张 GPU 上分别完成局部搜索并在末端归并, 系统会额外承担局部结果表达、跨卡通信和同步等待等成本; 而当不同查询被均衡分发到不同 GPU 时, 每张设备就更容易复用单卡阶段已经压缩好的“候选召回—异常评分”闭环。cuVS 多 GPU 文档将索引复制 (replicated) 模式描述为通过在各 GPU 间分发查询以获得更高查询吞吐 (query throughput), 这一点与本文的吞吐目标一致^[9]。

在此前提下, 第二项原则进一步限定了主方案选择: 数据并行与索引复制优先于一开始即采用索引分片。cuVS 的分布模式定义表明, 索引复制 (replicated) 更偏向吞吐优先, 分片 (sharded) 更偏向容量扩展^[9,26]。因此, 只要索引仍可复制, 完整副本在每张 GPU 上独立服务本地查询, 通常比从一开始就引入跨卡依赖更符合本文的系统目标。

第三项原则主要约束控制边界, 即 CPU 保持在控制面而不重新回到结果处理主路径。cuVS 文档提示, 多 GPU ANN 的部分实现要求数据集、查询与输出数组位于主机内存 (host memory), 这意味着若直接沿用主机中心式 (host-centric) 默认路径, 则 CPU 与主机内存会重新进入关键路径^[38]。基于这一认识, 本文在多 GPU 方案中将 CPU 定位为控制面调度者: 它负责请求队列管理、负载均衡、有界批处理 (bounded batching)、运行时监控和最终小规模结果汇总, 但不承担大规模候选集合或评分中间张量的重新组织。

最后一项原则只在主方案不能直接成立时才发挥作用, 即当必须跨卡归并时, 归并逻辑尽量下沉到 GPU 侧, 并通过 NCCL 等设备间通信机制完成。NCCL 文档指出其通信操作会异步入队到 CUDA stream 上执行, 但也警告在同一 ncclGroupStart/End() 中混用多个 stream 会形成全局同步点^[27]。这意味着 GPU 侧归并虽然优于 CPU 聚合, 却仍然带来额外通信组织成本, 因此它在本文中是明确的兜底路径, 并不作为第二主线考虑。

表 5-1 对 FlashTOD 多 GPU 部署中的索引复制主方案、设备侧归并兜底路径以及主机侧聚合对比路径的适用边界进行了归纳。

表 5-1 FlashTOD 多 GPU 部署适用边界

条件	replicated 主方案	设备侧归并	主机侧聚合	主要风险
索引可完整复制，吞吐优先	每卡保留完整索引，按查询或微批分发	通常不需要启用，仅保留为容量受限兜底	仅用于调试或小规模对照	显存占用高，索引规模继续增大后可能失效
单卡无法容纳完整索引，需要容量扩展	不再直接适用	索引按卡分片，局部结果在 GPU 侧组织和归并	可作为实现简单的对照路径	跨卡通信、同步点和归并复杂度上升
原型验证、调试或离线分析	可用于简化路径验证	可与主机侧聚合形成对照	各 GPU 输出局部结果后回传 CPU 排序合并	CPU 重新进入数据平面，D2H/H2D 与排序开销放大

该表用于说明不同多 GPU 路径的适用条件，而不是把三类方案视为并列的最终选择。本文当前结论限定在单机多 GPU、固定候选预算和固定硬件拓扑条件下：replicated 路径是吞吐优先主方案，设备侧归并是容量受限时的兜底路径，主机侧聚合主要用于形成可解释的对照基线。

5.3 数据并行与索引复制的多 GPU 执行架构

基于前述设计原则，本文将多 GPU 场景下的主扩展方案定义为数据并行与索引复制（replicated）架构。索引复制架构适用于索引仍可在单张 GPU 显存内复制、查询吞吐优先且希望复用完整单卡链路的场景；该路径主要提升吞吐，不提升单卡可容纳的索引容量。这意味着，只要完整索引仍能复制到每张 GPU，本文就更倾向于用显存换取更简单的数据路径和更稳定的查询闭环^[9]。在该方案中，完整索引被复制到每张 GPU 上，每张 GPU 都具备完整的候选召回与异常评分能力；查询按批次或微批次被分发到不同 GPU，各设备在本地独立执行“查询向量组织—候选召回—异常评分—局部结果输出”全链路流程；CPU 主要承担请求级调度、批次分配和最终小规模结果汇总职责。

在执行流程上，数据并行与索引复制架构可以形式化表示为：给定总查询集合 $Q = \{q_1, q_2, \dots, q_m\}$ ，调度器将其划分为若干子批次 $\{B_1, B_2, \dots, B_g\}$ ，并将这些子批次分配给 g 张 GPU。每张 GPU G_j 本地维护一份完整索引 $I_j = I$ ，对其收到的批次 B_j 独立执行

$$C_j = \mathcal{R}(B_j, I_j), \quad S_j = \mathcal{F}(B_j, C_j), \quad (5-1)$$

式 (5-1) 中， $\mathcal{R}$ 表示候选召回， $\mathcal{F}$ 表示异常评分。由于每张 GPU 都能本地完成完整闭环，因此系统不需要在查询处理中途引入设备间依赖。

从系统工程角度看，索引复制虽然会增加显存占用，但换来了更低的执行复杂度 and 更清晰的数据路径。其优势主要体现在三个方面：查询之间天然独立，便于按请求或微批次做吞吐扩展；每张 GPU 都能沿用第四章已经优化好的本地执行链路，不需要为多卡场景重新拆解候选召回与评分接口；系统只需在极小规模结果层面做收口，而不需要在处理过程中持续维护跨卡依赖。综合以上原因考虑，索引复制在本文中始终被定义为主方案，而不是若干方案中的普通选项。

图 5-1 给出了索引复制主方案的总体结构，直观展示了 CPU 控制面与 GPU 本地闭环在多卡场景下的分工关系。

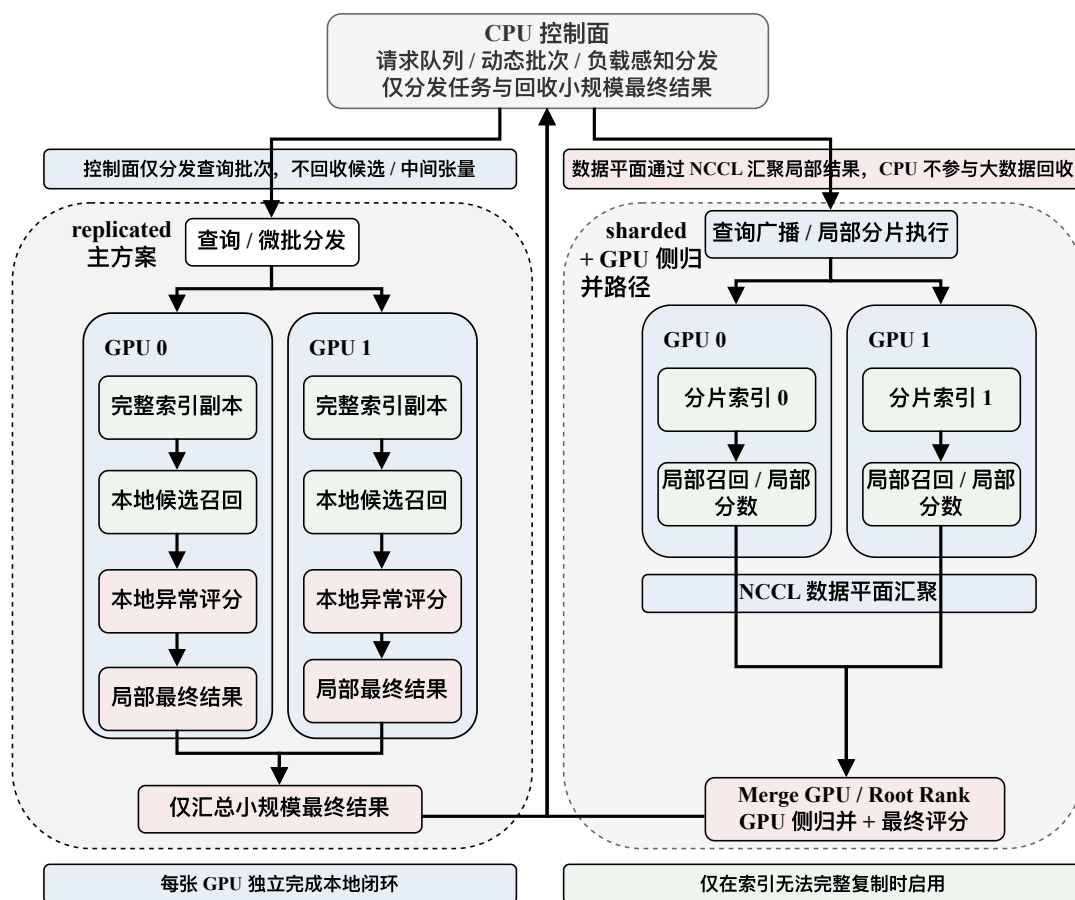


图 5-1 数据并行与索引复制多 GPU 架构（本文绘制）

5.4 CPU 控制面调度与负载均衡

在索引复制（replicated）架构下，CPU 的职责需要被严格限定。本节所讨论的控制面工作主要包括请求队列维护、微批形成、按 GPU 负载分发批次、控制 bounded batching 策略，以及在各 GPU 完成本地闭环之后接收最终小规模输出。

请求队列组织方面，系统采用统一入口队列接收查询请求，并根据队列长度、GPU 空闲度和微批策略动态形成批次。低时延推理系统和可预测深度学习服务系统的研究都表明，面向吞吐与尾延迟的批处理策略，需要在设备利用率和排队时间之间做持续折中；cuVS 动态批处理文档则为本文的多 GPU 实现提供了更接近工程接口层的参考^[39-41]。对本文而言，微批并不是单纯为了扩大批量规模，而是为了在吞吐和尾延迟之间建立一个可调的控制旋钮：批次过小会浪费设备并行能力，批次过大又会推高排队等待。

负载均衡方面，本文在索引复制架构下优先采用查询级动态调度而非静态平均分配。cuVS 在索引复制模式下提供了 LOAD_BALANCER 与 ROUND_ROBIN 两类搜索模式，其中前者用于保持各 GPU 负载均衡^[26]。这类调度能够防止某张 GPU 因瞬时队列堆积而拉高整机的第 99 百分位延迟（p99 延迟），避免只在形式上平均分配请求。

最终结果汇总方面，CPU 仅接收每张 GPU 已完成的最终小规模结果，如异常分数、Top-k 排序结果和必要日志元信息。局部候选集合、评分中间张量以及归并前的设备侧结构都不应回流到 CPU。

图 5-2 展示了 CPU 控制面的调度与回收流程，控制面与数据平面的职责边界将在正文中结合各组件的关键参数分别说明。

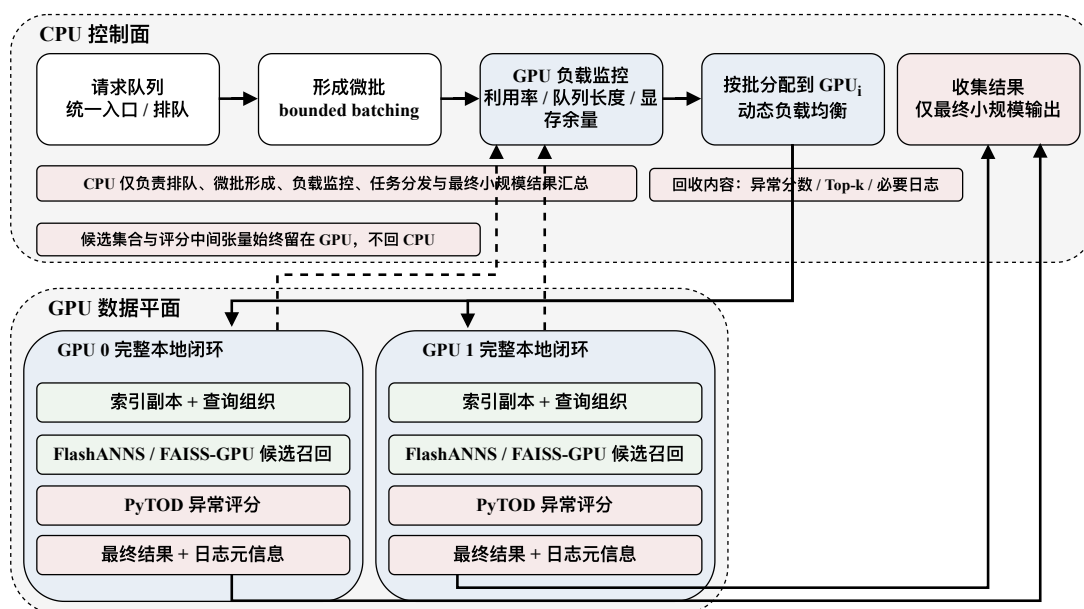


图 5-2 CPU 控制面调度与微批示意（本文绘制）

图中 FAISS-GPU 表示前期候选召回接口验证路径；多 GPU 完整路径下的本地候选生成以后续接入的 FlashANNs 候选召回前端为主。

5.5 跨卡归并场景下的 GPU 侧归并与 NCCL 通信

当索引容量超过单卡显存可容纳范围，或业务约束要求单一查询必须访问分布在多张 GPU 上的局部索引时，索引复制主方案就不再直接适用，系统必须进入跨卡归并场景。本文把这一场景明确界定为兜底路径：只有在主方案因容量或部署条件失效时，才引入分片索引与跨卡结果合并。因为一旦进入归并路径，系统就必须额外承担局部结果表达、设备间通信和流协调等成本。

在此条件下，本文仍坚持一个约束：局部结果尽量在 GPU 侧组织与合并，CPU 只接收最终极小规模结果。cuVS 在分片模式下给出了 MERGE_ON_ROOT_RANK 与 TREE_MERGE 两类归并方式，为本文设计设备侧归并路径提供了参考^[26]。与主机侧聚合路径相比，设备侧归并路径的主要收益在于减少大规模结果回传和主机侧排序开销，使跨卡合并仍尽可能延续设备侧处理逻辑；但它并不会像索引复制主方案那样天然简单，因为额外的通信与同步组织无法完全省去。

在设备间通信实现方面，本文采用 NCCL 作为主要通信机制，并将通信流与局部计算流明确分离。NCCL 文档指出其通信操作异步入队到指定 CUDA stream，但在同一 ncclGroupStart/End() 组内混用多个 stream 会形成全局同步点^[27]。因此，GPU 侧归并的收益建立在一个前提上：通信必须被控制在必要范围内，并与本地计算流合理解耦。若通信组织不当，兜底方案很容易因同步点过多而侵蚀掉原本希望保留的扩展收益。

5.6 拓扑感知 I/O 通道绑定与准入控制

在多 GPU 异常检测系统中，扩展上限不仅由 GPU 数量决定，还受制于存储路径和 PCIe/NUMA 拓扑。单卡阶段被压缩掉的路径开销，在多卡环境下可能以另一种形式重新出现：如果多张 GPU 共享同一 SSD 通道或 PCIe 路径，局部检索和候选读取虽然仍在设备侧执行，但共享链路上的排队会直接反映到吞吐下降和 p99 延迟上升。GDS 概览文档指出，GDS 通过在 GPU 显存与存储设备之间建立直接内存访问（Direct Memory Access, DMA）路径来避免 CPU bounce buffer，并强调拓扑路径会影响传输效率^[28]。基于这一认识，本文引入拓扑感知 I/O 通道（I/O lane）绑定与准入控制机制：运行前分析 GPU、SSD、PCIe 根复合体（PCIe root complex）、PCIe 交换机（PCIe switch）与 NUMA 节点之间的关系；运行时尽量将 GPU 与 SSD 通道进行绑定，并对每条 I/O 通道的在途请求（in-flight requests）数做有界控制，以避免过度并发导致尾延迟失控。

图 5-3 以左右对比方式展示了拓扑感知 I/O 通道在合理绑定与共享冲突两种条件下的 GPU、SSD、PCIe 与 NUMA 关系，为后续扩展结果中的路径差异提供解

释依据。

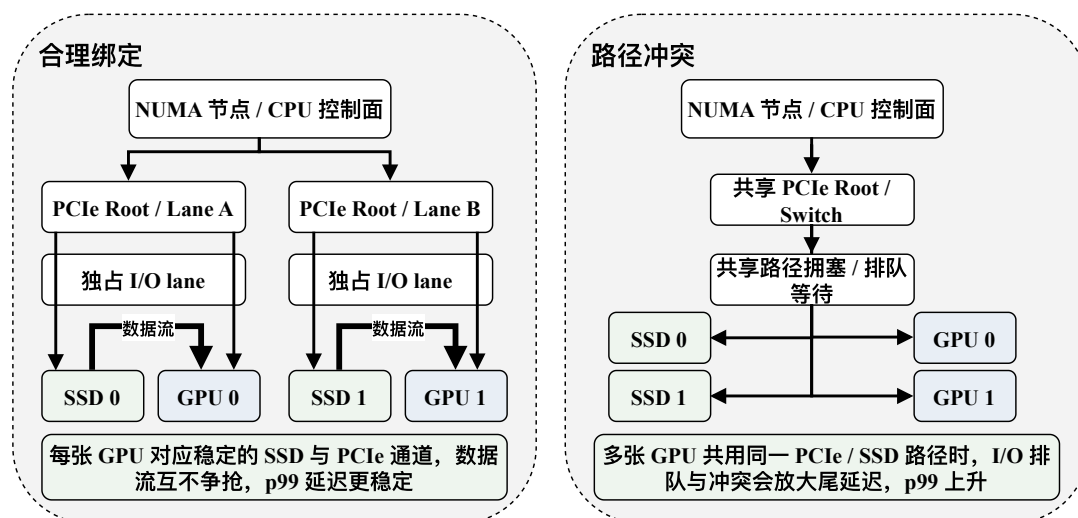


图 5-3 拓扑感知 I/O 通道示意（本文绘制）

5.7 多 GPU 扩展实验与结果分析

多 GPU 实验沿第三章定义的扩展规模链路逐级组织，并在单卡口径基础上延伸到 100M 样本压力场景；实验围绕索引复制、设备侧归并、主机侧聚合与共享路径冲突（shared-path conflict）四类路径展开，对应配置列入附录 B 补充表 B-2。图 5-4 汇总展示索引复制路径的扩展曲线与共享路径下的 p99 延迟变化，图 5-5 展示不同归并路径下的延迟变化，图 5-6 用于比较 I/O 通道配置对扩展性的影响，表 5-2 对总体结果进行了汇总。

从结果上看，索引复制方案在 1/2/4 GPU 条件下保持了较为稳定的 QPS 增长和相对平稳的延迟变化，说明在索引可复制的前提下，查询级并行和本地闭环执行更符合本文的吞吐优先目标。与之对应，本地评分 + 设备侧归并和本地评分 + 主机侧聚合的对比主要用于回答兜底路径如何选的问题：两者都建立在索引分片的前提下，但在本文实验口径下，设备侧归并的平均延迟、p99 延迟和 CPU 利用率低于主机侧聚合，说明一旦必须跨卡归并，把局部结果留在设备侧处理更有利于控制路径开销。

另一方面，拓扑感知绑定（Topology-aware binding）与共享路径冲突（Shared-path conflict）的对比表明，即便计算资源足够，当多张 GPU 共享同一 PCIe/SSD 路径时，QPS 仍会明显下降，p99 延迟也会迅速抬升。这意味着，多 GPU 扩展的实际表现并不只由归并方式决定，还受到共享路径组织方式的直接限制。

图 5-4 左图给出了索引复制路径下 QPS 随 GPU 数量增加的变化，并加入理想

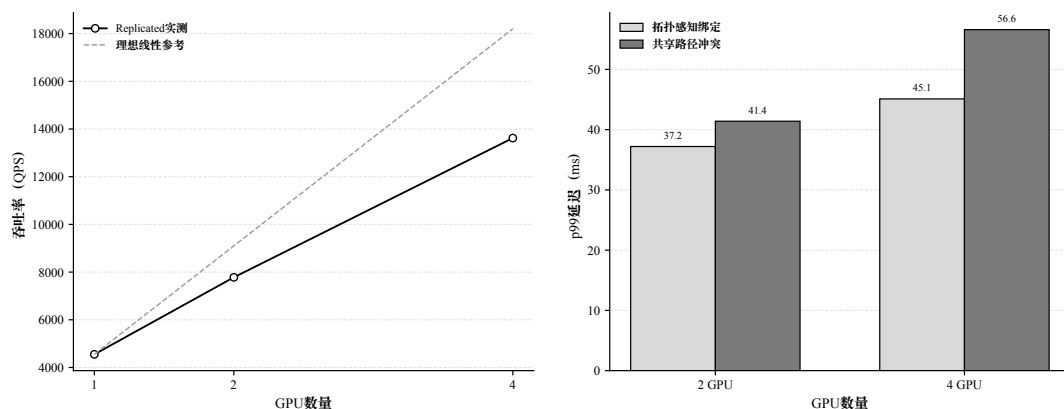


图 5-4 多 GPU 扩展趋势与共享路径瓶颈分析（实验数据绘制）

线性扩展参考线；该参考线仅由 1 GPU QPS 线性外推得到，是理论参考而非实测结果。实际曲线与理想线性之间存在差距，说明 CPU 控制面调度、查询分发、设备侧资源竞争和 I/O 路径都会削弱扩展效率。右图比较了拓扑感知绑定与共享路径冲突下的 p99 延迟，结果显示共享 PCIe/SSD 路径会放大尾延迟，因此拓扑感知 I/O 通道绑定并非附属优化，而是在多 GPU 场景下维持稳定扩展的重要条件。对于索引无法完整复制的容量受限场景，仍需结合图 5-5 比较设备侧归并与主机侧聚合：前者把局部结果尽量留在设备侧，后者需要回到主机侧排序与聚合，因此会引入更高的路径开销。

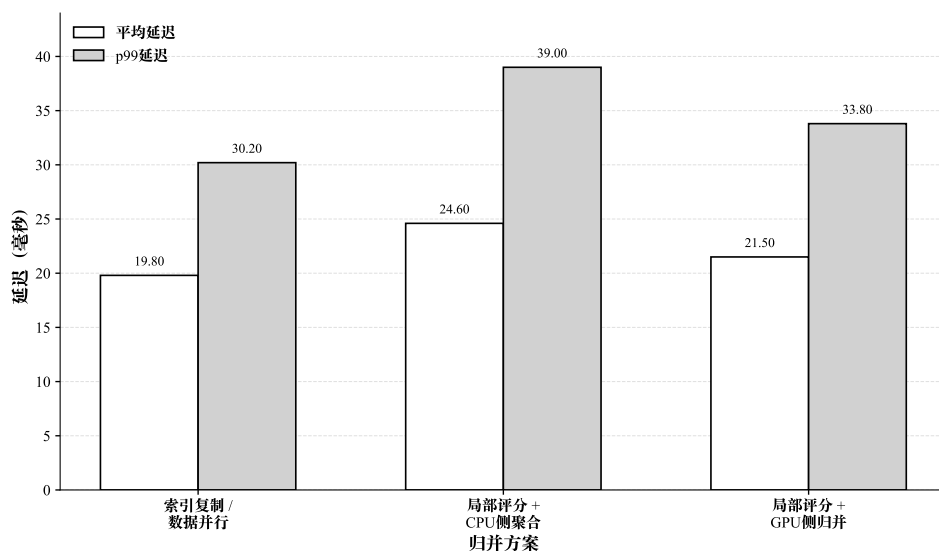


图 5-5 平均延迟与 p99 延迟变化（实验数据绘制）

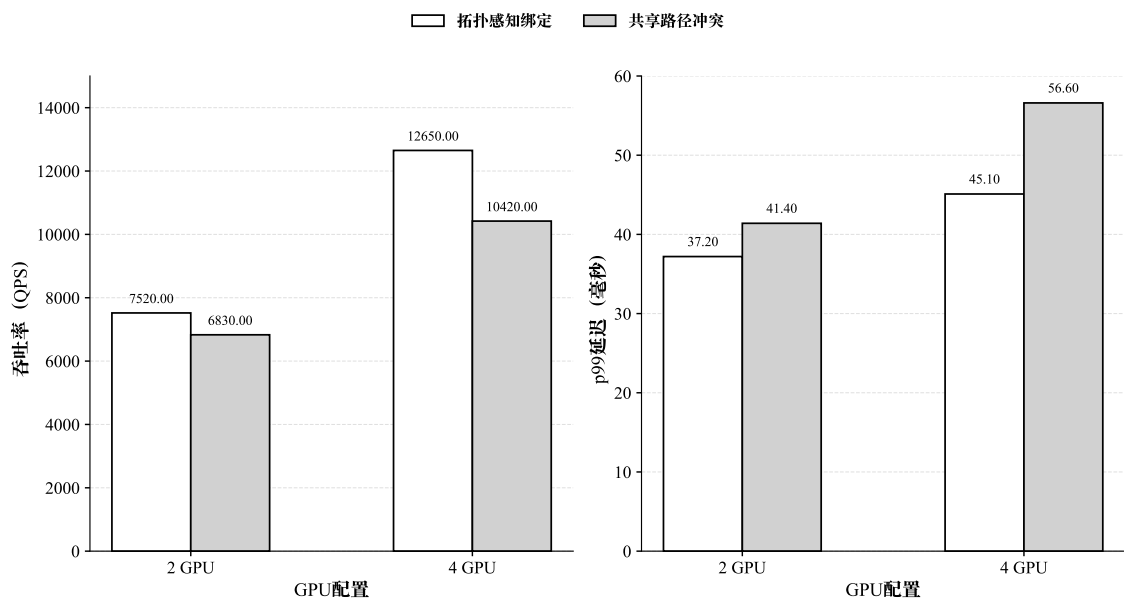


图 5-6 I/O 通道配置对扩展性的影响（实验数据绘制）

表 5-2 多 GPU 扩展结果汇总

GPU 方案数		数据规模	QPS	avg latency (ms)	p99 latency (ms)	CPU 利用率 (%)	GPU 利用率 (%)
1	Replicated	10M	4550.00	22.80	34.80	47.00	63.50
2	Replicated	10M	7780.00	21.00	31.90	42.00	68.80
4	Replicated	10M	13620.00	19.80	30.20	37.80	73.50
4	本地评分 + 设备侧归并	100M	11380.00	21.50	33.80	41.50	69.80
4	本地评分 + 主机侧聚合	100M	9580.00	24.60	39.00	48.50	62.50
2	Topology-aware binding	100M	7520.00	23.40	37.20	43.80	67.50
4	Topology-aware binding	100M	12650.00	22.10	45.10	39.80	72.20
2	Shared-path conflict	100M	6830.00	25.10	41.40	45.80	61.80
4	Shared-path conflict	100M	10420.00	28.90	56.60	44.50	56.50

注：结果展示遵循与附录 B 中补充表 B-2 相同的多 GPU 统一规模口径。受版面与结果表达重点限制，表中列出的是 10M 索引复制扩展结果和 100M 压力场景结果，用于回答主方案、兜底路径与共享路径冲突三类设计问题。

5.8 本章小结

本章的工作主要形成了三项进入后续综合实验的结构性结论。第一，在索引可复制的前提下，索引复制主方案能够最直接地复用第四章已经优化好的单卡执行链路，因此适合吞吐优先场景。第二，当容量约束迫使系统进入分片路径时，设备侧归并相比主机侧聚合更能保留设备侧处理优势，但其收益仍然受通信与同步组织方式影响。第三，多 GPU 扩展的上限不仅取决于计算资源，还取决于 I/O 路径与拓扑关系能否支撑多卡并发。

这些结论在本章内完成了初步验证，但其适用边界还需要在下一章与单卡优化结果、不同数据规模表现和整体检测效果一起统一考察。因此，第六章将不再分别讨论单卡与多卡的局部机制，而是把本章得到的主方案、兜底路径和路径冲突基线放回到综合实验框架中，继续验证其在整体系统层面的有效性与边界。

第六章 综合实验设计与结果分析

6.1 实验环境与软硬件配置

本章的作用是把前文提出的系统路线放到统一实验框架中检验：FlashTOD 是否在真实或半真实金融数据上保持可用，单卡优化是否确实压缩了数据路径，多 GPU 扩展是否把吞吐提升建立在合理的结构选择之上，以及 I/O 路径与归并方式在何种条件下开始成为新的限制因素。围绕这些问题，本节先交代实验所依赖的硬件条件、软件栈和实验分类口径。

硬件环境方面，实验平台由多张同型号 GPU、主机 CPU、内存、SSD 以及对应 PCIe 拓扑组成。单卡实验使用 1 张 GPU 运行完整的候选召回与异常评分链路，主要观察第四章提出的数据路径优化、异步流组织、RAPIDS 后端重构、FAISS-GPU 前期接口验证经验和 FlashANNS 候选召回前端接入在单设备上的综合表现；多卡实验使用 2 张及以上 GPU，主要对应第五章中的索引复制（replicated）扩展、控制面调度、设备侧归并和 I/O 通道控制。由于本文后续还需要讨论拓扑差异对扩展结果的影响，GPU 型号、显存容量、CPU 型号、主机内存、SSD 容量、PCIe 代际和 NUMA 关系都需要作为实验前提显式给出。

软件环境方面，实验系统由 PyTOD 评分实现、FlashANNS 候选召回前端、FAISS-GPU 初步 ANN 接口验证组件、RAPIDS 数据处理与资源管理组件、多 GPU 通信和调度运行时共同组成。FAISS-GPU 主要用于验证候选召回接口、设备侧输入输出路径和候选结果格式，完整 FlashTOD 路径中的大规模候选生成以后续接入的 FlashANNS 为主。为了保证结果可复核，本文固定 Python、PyTorch、PyTOD、FAISS-GPU、CUDA、NCCL 以及操作系统版本；CuPy、cuDF、cuML、RMM 等 GPU 数据处理与资源管理组件的版本也一并记录为统一软件环境快照。

在实验分类上，本文将综合实验划分为三组：方法有效性验证实验，对应真实或半真实数据及 1M 合成桥接口；单卡系统性能实验，沿第三章定义的单卡规模链路组织；多 GPU 扩展实验则在此基础上延伸至 100M 样本压力场景，用于比较索引复制主方案、设备侧归并兜底路径以及 I/O 路径组织方式。三类实验共享一套基础代码框架，但在数据规模、硬件使用方式和评价重点上各有分工。

表 6-1 给出了实验平台与软件环境配置快照，图 6-1 则补充展示了实验平台中 CPU、GPU、SSD 与 PCIe/NUMA 的连接关系，为后续性能差异解释提供硬件背景。

表 6-1 实验平台与软件环境配置

项目	规格	说明
硬件平台配置		
CPU 平台	2 × Intel Xeon Gold 6348 @ 2.60GHz	56 核 112 线程，双 NUMA
主存容量	503 GiB	系统总内存
GPU 平台	8 × NVIDIA GeForce RTX 3090	24 GB/卡，总显存 192 GB
PCIe 能力	PCIe Gen4 ×16	8 张 GPU 均为 Gen4 ×16
本地存储	3 × NVMe SSD	894.3 GB × 2, 3.5 TB × 1, 总约 5.29 TB
NUMA 拓扑	2 个 NUMA 节点	NUMA 0 对应 GPU0–GPU3, NUMA 1 对应 GPU4–GPU7
GPU 互连特征	同侧 PIX/PXB, 跨侧 SYS	用于解释多 GPU 与 I/O 通道实验
网络接口	2 × Mellanox NIC	mlx5_0, mlx5_1
软件环境配置		
操作系统	Ubuntu 20.04.4 LTS, Linux 5.13.0-30-generic	实验运行环境
Python / CUDA	Python 3.11.14; CUDA Runtime 12.4	基础运行时环境
深度学习框架	PyTorch 2.5.0+cu124; NCCL 2.21.5	GPU 张量计算与通信环境
驱动环境	NVIDIA Driver 550.120	与 CUDA 12.4 对应
异常检测框架	PyTOD (yzhao062/pytod)	GitHub 主仓库默认分支 main
ANN 检索组件	FlashANNS / ann_search (Tinker/ann_search)	GitHub 主仓库默认分支 main
其他核心库	FAISS 1.13.2; PyOD 2.0.6; NumPy 2.2.6; SciPy 1.13.1; scikit-learn 1.6.1	实验所用主要 Python 依赖; FAISS 用于初步 ANN 接口验证和对照实现
RAPIDS 相关	CuPy 13.3.0; cuDF 25.02.00; cuML 25.02.00; RMM 25.02.00	用于 GPU 侧表格处理、中间结果组织、数组计算和内存资源管理, 不单独归因整体性能收益

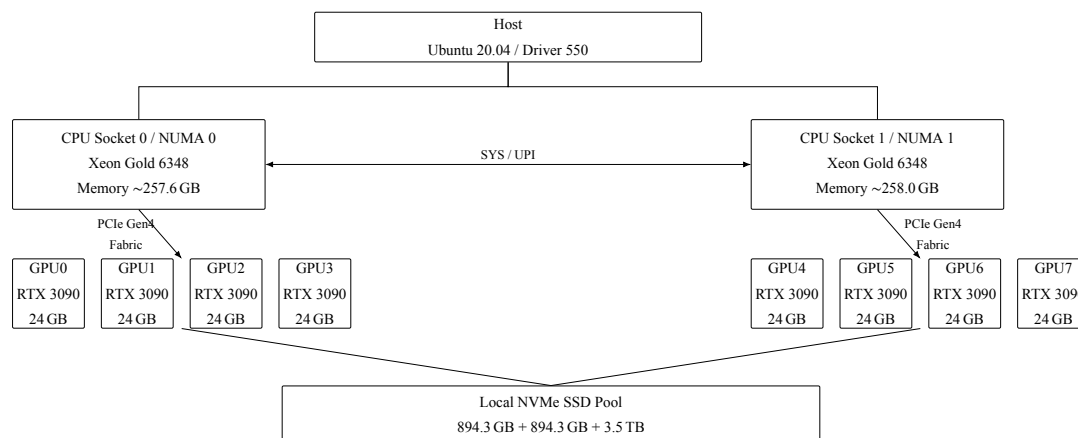


图 6-1 实验平台拓扑示意图（本文根据实验平台配置绘制）

6.2 数据集与任务设置

本章沿用第三章的数据安排，但实验目的已经从建立系统对象转向统一验证系统结论，不同规模数据在这里承担的角色也更明确。真实或半真实金融数据对应全文的现实口径，1M 合成数据承担桥接层，单卡实验沿前述规模链路逐层展开，多 GPU 实验则继续延伸到 100M 样本压力场景。

需要说明的是，本章中的真实或半真实金融数据并不等同于金融机构生产系统中的在线原始流量，而是指经过公开发布、脱敏、匿名化、采样或特征变换后的金融相关任务数据。它们适合用于检验异常检测链路在真实业务语境下的可用性；10M、50M、100M 等 AMLSim / IBM AML 合成规模则主要服务于系统压力、吞吐扩展和 I/O 路径验证，相关结论应理解为在本文实验平台和生成口径下的工程趋势。

在有效性验证部分，本文继续使用 IEEE-CIS Fraud Detection、Credit Card Fraud Detection 和 DGraphFin 这三类具有代表性的公开金融数据，分别覆盖结构化支付风控、经典信用卡欺诈以及图金融数据场景。具体数据来源和字段特征已在第三章中给出，这里不再重复展开。

在系统性能与扩展实验部分，本文采用第三章已经介绍过的高保真合成金融交易数据。通过规模分层让不同规模分别承担不同验证任务，其中，1M 数据用于连接真实数据验证与系统实验，使检测效果和端到端实现处于同一实验口径；10M 数据是单卡综合性能与多 GPU 扩展的共同起点；50M 数据主要用于观察规模继续增大后单卡链路与多 GPU 扩展的中间趋势；100M 数据则主要服务于多 GPU 扩展、跨卡归并和共享路径冲突实验。

在任务组织上，本章实验可分为四类：候选召回与异常评分有效性任务、单卡性能任务、多 GPU 索引复制扩展任务以及跨卡归并与 I/O 路径任务。这些任务共

享同一套核心实现，但比较对象和观测重点并不相同。有效性任务更关注检测结果与候选排序质量；单卡性能任务更关注数据路径压缩后的吞吐、延迟和利用率；多 GPU 任务则进一步考察索引复制主方案、设备侧归并兜底路径以及共享路径冲突对扩展收益的影响。

在数据使用方式上，有效性验证任务按训练集、验证集和测试集划分，以保证 PR-AUC、Recall@ k 等指标可比；性能与扩展实验则采用固定索引集与固定查询集，将查询批量持续注入系统，以更准确地衡量 QPS、延迟分位数与资源利用率。这样区分能够避免把检测效果评估和系统压测混在同一套数据口径下讨论。

图 6-2 展示了规模层级与实验类型的映射。

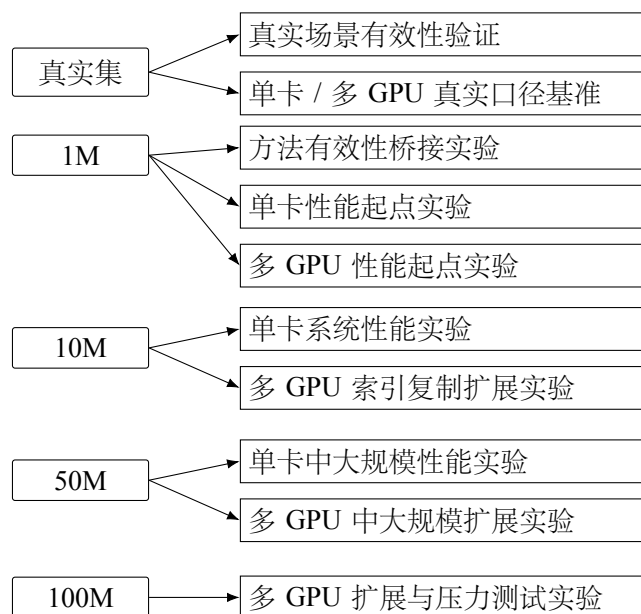


图 6-2 数据规模层级与实验类型关系图（本文绘制）

本章据此区分效果评估和系统压测：有效性口径沿用固定随机种子的训练/验证/测试划分，用于比较 PR-AUC、Recall@ k 与 Top- k overlap；更大规模性能实验则固定索引集和查询集，持续注入请求以观察 QPS、p99 延迟和资源利用率。

6.3 评价指标与比较原则

本章采用的指标体系服务于全文的四条核心判断：FlashTOD 是否在效果上可接受，单卡优化是否主要压缩了数据路径，多 GPU 索引复制是否真正提升了吞吐，以及 I/O 路径是否决定扩展上限。围绕这四类判断，指标被组织为四组：异常检测效果指标、候选检索质量指标、系统吞吐与延迟指标以及资源利用率指标。

其中，受试者工作特征曲线下面积 (Receiver Operating Characteristic Area Under Curve, ROC-AUC)、PR-AUC 与 Recall@ k 用于评价候选召回与张量化评分组合是

否仍保持合理的检测能力；Recall@ k 与 Top- k overlap 用于观察候选召回是否改变了最终排序口径；QPS、平均延迟与 p95/p99 延迟直接对应系统在单卡和多卡场景中的端到端收益；GPU 利用率、CPU 占用率、显存使用率、I/O 带宽利用率以及 CPU↔GPU 传输开销占比，则用于解释这些收益来自哪里。

比较时不将所有方案统称为“现有方法”或“基线系统”。同一类实验共享数据源、向量表示、硬件和软件环境；候选召回比较保持候选预算与评分主干可比；多 GPU 路径则固定查询到达方式，以便将差异定位到复制、归并或共享 I/O 策略。不同数据规模的结果仅用于各自实验问题，不拼接为伪装的同条件横向比较。

表 6-2 按四种性质组织比较对象。PyTOD 全量评分是已有评分方法参考；FAISS-GPU + PyTOD 是本文为控制“简单增加 ANN 前端”这一变量而构造的工程组合基线，不是已有文献提出的独立异常检测方法；FlashTOD 是本文完整方案；仅召回、CPU aggregation、GPU merge、replicated 和 shared-path conflict 均是内部执行路径对照。

表 6-2 对比方案、工程基线与内部执行路径设置

方案	性质	候选召回	异常评分	GPU 外存	GPU 主导	多 GPU	比较目的
				ANNS	路径		
PyTOD 全量评分	已有评分方法参考	不使用	PyTOD full scoring	是	否	评分内部	否 观察不使用候选压缩时的效果与开销
仅召回方案	内部功能对照	FAISS-GPU	无完整评分	是	否	部分	否 检验候选召回是否能替代异常评分
FAISS-GPU + PyTOD 初步融合	本文构造的工程组合基线	FAISS-GPU	候选集 PyTOD	是	否	否	否 观察简单 ANN + 评分串接的表现
FlashTOD	本文完整方案	FlashANNS	候选集 PyTOD	是	是	是	是 验证候选、评分与数据路径协同的端到端收益
CPU aggregation	多 GPU 内部路径对照	本地 FlashANNS	本地 PyTOD	是	是	本地链路	是 检验主机归并的搬运和同步代价
GPU merge	多 GPU 内部路径对照	本地 FlashANNS	本地 PyTOD	是	是	是	是 比较设备侧归并的收益与通信代价
replicated	多 GPU 内部路径对照	本地 FlashANNS	本地 PyTOD	是	是	是	是 观察索引可复制时的吞吐扩展
shared-path conflict	多 GPU 内部压力对照	本地 FlashANNS	本地 PyTOD	是	是	受共享路径限制	是 观察 PCIe/SSD 竞争对 p99 和吞吐的影响

端到端方案比较使用本文已有的 1M 桥接口径，在同一 PyTOD 评分主干下观察 PR-AUC 和 QPS。单 GPU 递进路径继续使用第四章已归档的 QPS、p99 延迟、PCIe 往返次数和 GPU 利用率；多 GPU 路径只在各自的 10M 或 100M 现有压测口径内比较。这三类数据承担的问题不同，因此正文不对未记录的延迟、利用率或召回指标作推算。

在统计口径上，当前实验结果基于固定硬件平台、固定索引集、固定查询集、

固定候选预算和固定批大小下的稳定运行配置。由于送审前实验时间有限，本文尚未对所有配置给出完整误差棒、标准差和显著性检验，因此正文图表主要展示代表性结果和趋势性比较，相关结论也限定在本文设定的数据规模、硬件平台和实现路径内。后续若进一步完善实验，应在附录或实验日志中补充各配置的重复运行记录、标准差范围和必要的统计检验；在此之前，本文不将当前差异解释为统计显著结论。

本章图表中的数值均来自论文现有实验记录和绘图数据。理论线性扩展线仅作参考，已在相应图注中与实测结果区分；本轮没有新增实验、占位数据或外部 ANNS 替换结果。

图 6-3 给出了指标体系结构。

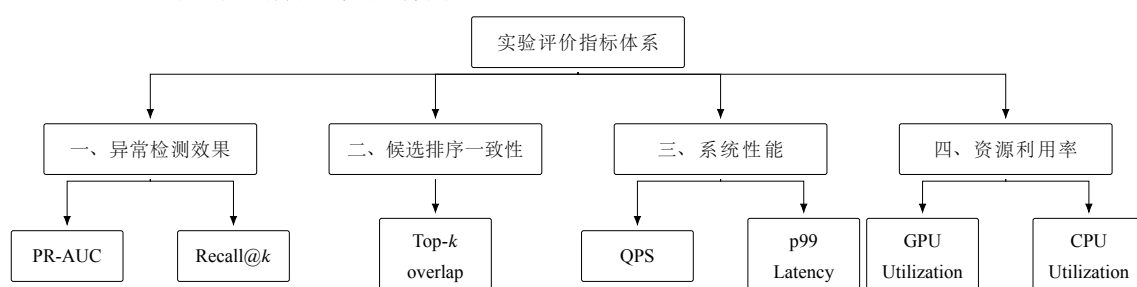


图 6-3 指标体系结构图（本文绘制）

比较时，同类实验固定数据源、标签口径、有效性划分、索引集和查询集；不同方案共享候选预算、批大小、硬件平台以及 CUDA、PyTorch、NCCL 等基础软件栈。其中，FAISS-GPU 主要用于初步 ANN 召回和接口验证方案，完整 FlashTOD 方案以 FlashANNS 作为主要候选召回前端；RAPIDS 主要用于 GPU 侧数据管理和资源管理支撑。单卡与多 GPU 仅在相同规模下比较，并统一统计口径；部分配置尚未覆盖完整重复运行和显著性检验，相关差异主要用于趋势性分析。

6.4 核心结果展示

本节围绕四个需要被验证的问题展开：FlashTOD 是否总体有效；单卡优化和多 GPU 扩展分别带来了什么收益；多 GPU 主方案与兜底路径的边界如何理解；规模继续增大后系统是否仍保持可用。各小节先给出比较对象和待回答的问题，再引出对应图表。

6.4.1 FlashTOD 与端到端比较方案

第一组结果使用 PyTOD 全量评分作为已有评分方法参考，并设置仅召回内部对照、FAISS-GPU + PyTOD 工程组合基线和 FlashTOD 完整方案。这组比较不把

四者说成四种已有异常检测方法，而是分别观察全量评分、单独召回、简单 ANN + 评分串接以及完整系统协同的效果和吞吐。

6.4.2 单 GPU 优化与多 GPU 扩展性能对比

第二组结果用于分析单卡优化与多 GPU 扩展各自改变的系统环节。比较对象包括原始分段链路、完整单卡优化、2 GPU 索引复制扩展以及 4 GPU 索引复制扩展。这里希望区分两类收益来源：单卡优化主要压缩候选召回到异常评分之间的数据路径和尾延迟，多 GPU 扩展则在此前基础上继续抬高系统吞吐上限，并缓解 CPU 压力。

6.4.3 多 GPU 扩展结果对比

第三组结果用于检验第五章中的结构选择是否成立。比较对象不再只是 GPU 数量不同的索引复制方案，还包括在容量受限场景下的设备侧归并与主机侧聚合。需要验证的是：索引复制是否可以作为优先主方案；当主方案不能直接成立时，设备侧归并是否比主机侧聚合带来更低的路径开销；这些差异是否反映在延迟分位数、CPU 利用率和吞吐变化上。

6.4.4 不同数据规模下的系统表现对比

第四组结果用于分析规模继续增大后系统能否保持前述判断。比较对象覆盖 1M、10M、50M、100M 四个规模层级，观察 QPS、第 99 百分位延迟（p99 延迟）和 PR-AUC 如何随规模变化。需要说明的是，PyTOD 全量评分可以作为 1M 量级下的效果上界参考，但并不适合作为 10M 以上规模的统一对照，因此这一组跨规模结果采用初步融合方案和完整融合方案作为可扩展口径下的比较对象。这里更关注的是，在外存驻留路径和多 GPU 扩展逐渐进入关键位置后，系统性能与检测效果是否同时出现可解释的退化：吞吐是否按预期下降，p99 延迟是否继续抬升，PR-AUC 和候选覆盖类指标是否在 10M 之后出现更明显回落，以及 I/O 路径是否开始主导上限。

表 6-3 集中列出端到端方案的现有 PR-AUC 和 QPS，表 6-4 整理第四章已有单 GPU 递进式消融数据。前者回答候选压缩带来什么效果-效率取舍，后者用于定位 FlashTOD 性能收益的来源。图 6-4-图 6-6 保留原有的可视化表达，图 6-7 和图 6-8 继续展示多 GPU 与跨规模趋势。

图 6-4 与表 6-3 使用相同的现有数据。仅召回方案吞吐最高，但 PR-AUC 为 0.844，这个内部对照只能说明召回阶段的速度上限，不能当作完整异常检测系统。FAISS-GPU + PyTOD 初步融合用于控制简单候选前端这一变量；FlashTOD 则在

表 6-3 端到端方案的检测效果与吞吐比较

方案	方案性质	PR-AUC	QPS	主要作用
PyTOD 全量评分	已有评分方法参考	0.932	2700	观察全量评分的检测效果与计算开销
仅召回方案	内部功能对照	0.844	4580	说明只保留候选召回不能替代完整异常评分
FAISS-GPU + PyTOD	本文构造的工程组合基线	0.883	3490	观察简单 ANN 候选前端与 PyTOD 串接的表现
FlashTOD	本文完整方案	0.906	4280	检验候选召回、异常评分和数据路径协同

表 6-4 FlashTOD 单 GPU 性能收益的路径来源

执行路径	QPS	p99 延迟 (ms)	PCIe 往返 次数/查询	GPU 利用率
原始分段链路	3180	67.0	6.2	48.5%
仅召回加速	3470	61.5	5.8	51.2%
GPU 主导路径	3950	53.8	4.9	57.8%
完整优化	4550	45.5	4.2	63.5%

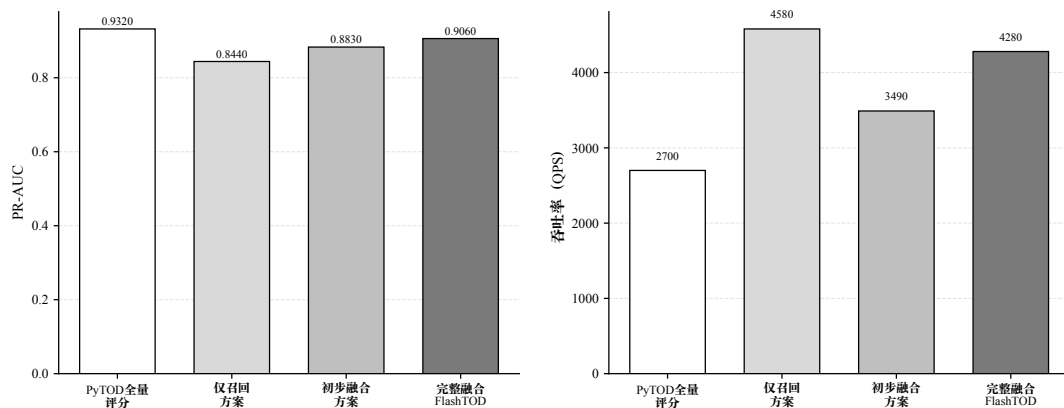


图 6-4 端到端比较方案的检测效果与吞吐率（实验数据绘制）

FlashANNS 候选召回之外，继续组织设备侧候选和 GPU 主导数据路径。具体差异在第 6.5 节结合路径消融结果分析。

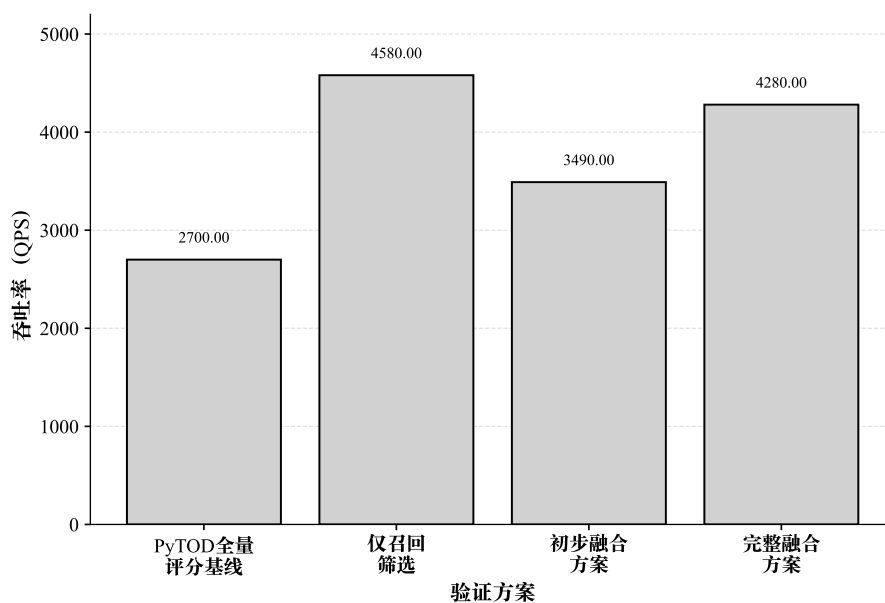


图 6-5 端到端比较方案的吞吐量（实验数据绘制）

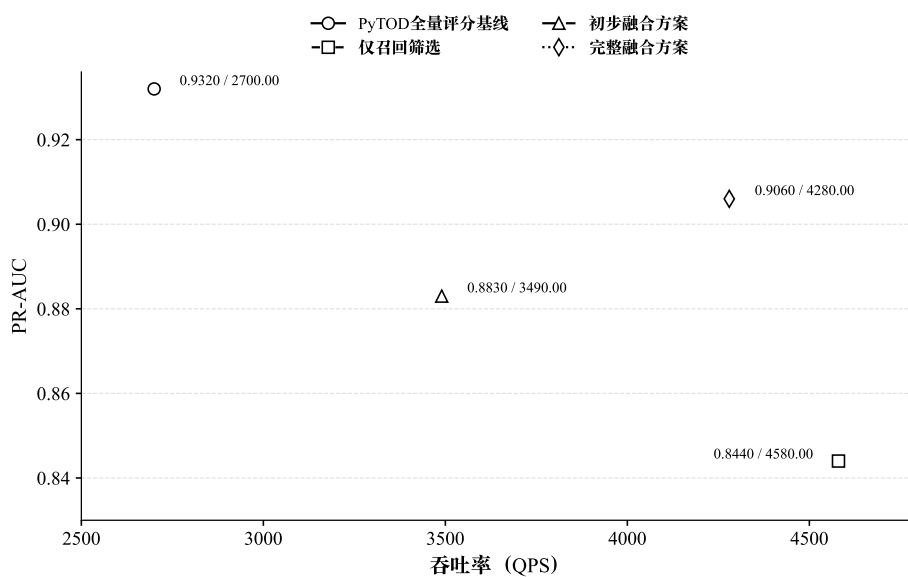


图 6-6 FlashTOD 效果与性能折中（实验数据绘制）

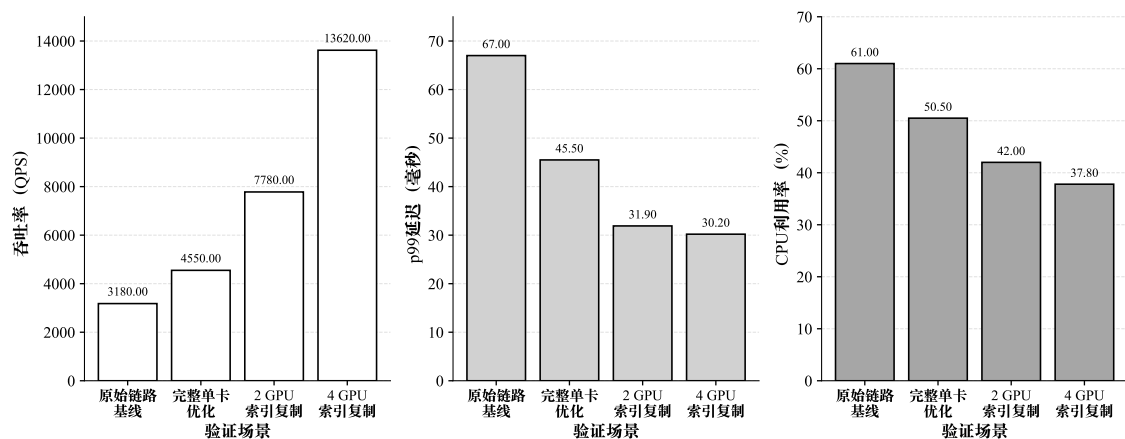


图 6-7 单卡优化与多 GPU 扩展收益对比 (实验数据绘制)

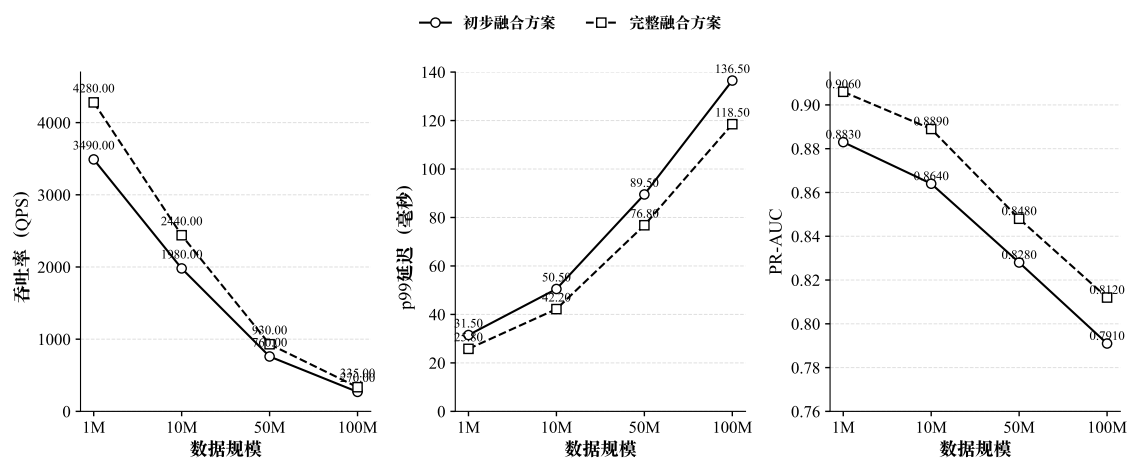


图 6-8 不同规模下系统表现趋势 (实验数据绘制)

6.5 结果讨论

与 PyTOD 全量评分相比, FlashTOD 改变的不是评分函数本身, 而是进入高代价评分阶段的样本范围。现有结果中, PyTOD 全量评分的 PR-AUC 为 0.932、QPS 为 2700; FlashTOD 的 PR-AUC 为 0.906、QPS 为 4280。后者的吞吐率约提高 58.5%, 但 PR-AUC 下降了 0.026。因此, FlashTOD 并没有在检测效果上超过 PyTOD 全量评分。这组结果表明的是效果-效率折中: 候选召回用有限的效果损失换取了更高的端到端吞吐。

FAISS-GPU + PyTOD 初步融合的 PR-AUC 为 0.883、QPS 为 3490, FlashTOD 对应为 0.906 和 4280。这一比较不能用来证明 FlashANNs 在所有条件下都优于 FAISS, 因为它们面向的索引和系统问题不同。该工程组合基线只回答一个更具体的问题: 把成熟 GPU ANN 接口放到 PyTOD 前面, 是否已经足够。结果显示, 简单串接仍未形成完整 FlashTOD 的效果与性能。

差异出现在阶段边界。候选 ID 和局部 distance 如果返回 CPU 重新打包, 评分前会额外经历 D2H、CPU candidate repacking 和 H2D。后续还包含候选张量构造、GPU 中间张量驻留、阶段同步以及外存 I/O 与计算的重叠。ANN 检索本身变快, 这些开销不会自动消失。FlashTOD 对设备侧候选组织和执行路径继续优化, 所以该结果应解释为系统协同的收益, 而不是单一 ANNS 算法的胜负。

候选召回不是全部答案。表 6-4 显示, 从原始分段链路到仅召回加速, QPS 由 3180 增加到 3470, p99 延迟只由 67.0 ms 降至 61.5 ms。当候选到评分的交接改为 GPU 主导路径后, QPS 进一步达到 3950, p99 降至 53.8 ms。完整优化最终将 PCIe 往返次数从每查询 6.2 降到 4.2, GPU 利用率从 48.5% 提高到 63.5%, p99 降至 45.5 ms, QPS 达到 4550。这是一条递进链路: 候选召回解决评分范围问题, GPU 主导路径解决阶段边界问题, 两者对应不同的瓶颈。

为了更清楚地解释不同优化项的收益来源, 可以将端到端时间分解为

$$T_{\text{total}} = T_{\text{prep}} + T_{\text{retrieve}} + T_{\text{pack}} + T_{\text{score}} + T_{\text{merge}} + T_{\text{io}}. \quad (6-1)$$

式 (6-1) 中, T_{prep} 表示查询准备时间, T_{retrieve} 表示候选召回时间, T_{pack} 表示候选组织与张量化时间, T_{score} 表示 PyTOD 评分时间, T_{merge} 表示多 GPU 归并时间, T_{io} 表示外存访问与主机-设备传输。候选预算主要改变 T_{score} 的输入规模, 设备侧候选组织直接压缩 T_{pack} , 异步 I/O 与计算重叠减少 T_{retrieve} 中的等待, 而多 GPU 归并会额外引入 T_{merge} 。这个分解与表 6-4 中 PCIe 往返、GPU 利用率和尾延迟的同向变化一致。

多 GPU 结果回答的是另一类问题。索引可复制时, replicated 路径能直接复用单卡闭环, CPU 只需调度请求和汇总小规模结果。容量约束迫使系统分片时, GPU

merge 在本文 100M 实验中的平均延迟和 p99 低于 CPU aggregation，但它仍需通信和流协调。多卡共享 PCIe/SSD 路径还会增加 I/O 排队，因此卡数增加并不等于线性扩展。

这些结果有四个明确边界。FlashTOD 的 PR-AUC 0.906 低于 PyTOD 全量评分的 0.932，效率收益并非没有代价。FAISS-GPU + PyTOD 是本文构造的工程组合基线，不是已有文献中的独立 ANN + TOD 异常检测系统。当前数据不支持将 FlashTOD 说成全面优于 HNSW、IVF/IVF-PQ 或 DiskANN。部分配置尚未覆盖完整重复运行和统计显著性检验，所以章内数值支撑的是当前实验口径下的趋势和工程取舍，不是无条件的普遍性结论。

6.6 局限性分析

超大规模实验主要依赖 IBM AML / AMLSim 等高保真合成数据。这类数据可用于施加吞吐、存储路径和多 GPU 扩展压力，却不能完全替代真实金融机构的在线负载、噪声结构和概念漂移。关于 10M、50M 和 100M 规模的结论因而限定为当前平台上的工程趋势。

PyTOD 全量评分的时间和中间结果存储开销限制了它在大规模压测中的覆盖范围。本文在 1M 桥接口径保留全量评分参考，10M 以上主要观察工程组合方案、FlashTOD 与多 GPU 内部路径。这能回答系统扩展问题，但不等同于所有规模下的全量评分横向实验。

现有对比已覆盖技术能力边界、PyTOD 全量评分参考、FAISS-GPU + PyTOD 工程组合基线以及 FlashTOD 内部路径消融。比较范围仍然有限：本文没有在相同候选预算和 PyTOD 评分后端下穷尽 HNSW、IVF/IVF-PQ 与 DiskANN 等可替换前端，也未新增 PyOD CPU 实验。所以，结论不包含对所有 ANNS 方法或异常检测系统的全面优势声称。

统计口径同样需要保留。当前数值主要来自固定硬件、查询集、候选预算和批处理参数下的已有稳定运行记录。部分配置尚未提供完整重复轮次、标准差、误差棒和显著性检验，因此章内差异用于趋势性比较和工程可行性分析，不解释为统计显著结论。

多 GPU 主方案也有适用前提。replicated 依赖索引可复制；索引超出单卡容量后必须转向 sharded 路径，并承担通信与归并代价。外存和拓扑感知 I/O 收益还取决于 GPU-SSD-PCIe-NUMA 拓扑、驱动与 GDS 支持。本文只验证单机多 GPU，多机通信、分布式索引与集群调度尚未进入实验范围。

6.7 本章小结

本章的综合实验把全文前几章分散建立的判断收束到同一验证框架中。在本文设定的数据规模、硬件平台和候选预算条件下，FlashTOD 在保持合理检测效果的同时，体现出面向大规模金融异常检测的系统组织价值；单卡优化的主要收益来自候选召回到异常评分之间的数据路径压缩；多 GPU 条件下，索引复制可作为吞吐优先场景的主方案，而设备侧归并与拓扑感知 I/O 控制则主要用于容量受限或路径受限场景下维持扩展效果。

第七章 全文总结与展望

7.1 全文总结

本文围绕金融大规模数据异常检测系统的实现与优化展开研究。面对传统异常检测系统在大规模场景下暴露出的异常评分复杂度高、候选召回吞吐不足、主机—设备边界交换频繁以及多 GPU 扩展效率受限等问题，论文把研究重点放在系统组织方式上，而不是新的异常检测理论模型上。围绕这一边界，全文尝试将候选召回、异常评分、数据路径优化与多 GPU 扩展放入同一框架中考察，并据此验证其在金融近实时异常检测场景中的工程可行性。

在系统架构层面，论文以 PyTOD 为异常评分模块、以 FlashANNS 为候选召回前端，构建了 FlashTOD。这样组织的目的，是把候选召回和异常评分纳入同一条执行链路，使系统能够先在大规模样本中缩小评分范围，再在候选集合上执行张量化异常评分，从而缓解全量评分带来的时间与空间压力。

围绕这一执行链路，论文进一步完成了数据准备与任务建模工作。针对金融交易数据、账户行为数据和图金融数据的差异，本文对输入形式、向量化表示、候选召回目标和异常评分目标进行了统一建模，并形成了按规模逐层推进的数据组织方案。借助这一方案，全文把真实有效性验证、桥接实验、单卡性能口径和多 GPU 压力场景纳入同一实验框架。

在系统实现层面，单 GPU 场景的工作重点放在数据路径压缩上。论文围绕 GPU 主导的端到端执行链路，对查询特征组织、候选结果传递和评分中间张量的驻留方式进行了重新安排，并通过页锁定内存、RAPIDS 后端重构和 FAISS-GPU 前期接口验证降低阶段边界上的重复搬运与同步等待；完整路径中的主要候选召回前端则以 FlashANNS 为主。在这一基础上，多 GPU 部分继续沿用吞吐优先的思路，选择数据并行与索引复制作为主扩展方案，使每张 GPU 尽可能独立完成本地候选召回与异常评分；当索引复制不可行时，再以设备侧局部结果组织、NCCL 通信和设备侧最终归并作为兜底路径，并结合拓扑感知的 I/O 通道绑定与准入控制减轻共享路径冲突。

本文前期利用 FAISS-GPU 较完善的 ANN API 完成候选召回接口和 PyTOD 评分衔接验证，并在完整 FlashTOD 路径中以 FlashANNS 作为主要候选召回前端。该处理使 FAISS-GPU 的作用限定在工程验证和接口参照层面，避免将其与 FlashANNS 在大规模外存 ANNS 路径中的职责混同。

综合实验部分则把前述系统设计放回统一验证框架中。在本文设定的数据规模、硬件平台和候选预算条件下，结果显示 **FlashTOD** 在真实数据口径和 **1M** 合成桥接口径上能够保持合理的检测效果；在更大规模的高保真合成数据上，单卡优化的主要收益来自候选召回到异常评分之间的数据路径压缩，而多 **GPU** 数据并行与索引复制方案表现出一定的吞吐扩展能力。随着规模继续增大，**I/O** 路径与拓扑关系对扩展上限的影响也变得更加明显。

本文的比较证据由三个层次构成：第二章分析已有评分、检索与数据路径技术的能力边界；第六章在统一 **PyTOD** 评分主干下比较全量评分参考、**FAISS-GPU** + **PyTOD** 工程组合基线和 **FlashTOD**；单与多 **GPU** 路径对照用于解释性能收益的来源。这些结果能回答本文设计与代表性比较方案之间的差异，但不支持对所有 **ANNS** 前端或异常检测系统声称全面优势。

综合来看，本文的主要研究结论与技术贡献可以归纳为三点：

1. **FlashTOD** 将高吞吐候选召回与异常评分组织到同一执行链路中，使系统从全量高复杂度评分转向候选缩减后的精排评分路径，并在本文实验口径下取得更稳妥的效果与效率平衡。
2. 单 **GPU** 场景下，系统性能的主要限制并不来自个别算子的绝对速度，而更多来自数据路径与阶段同步成本；通过 **GPU** 主导执行链路、**RAPIDS** 后端重构、**FAISS-GPU** 初步验证经验和 **FlashANNS** 候选召回前端接入，可以更稳定地改善端到端吞吐与延迟表现。
3. 多 **GPU** 场景中，数据并行与索引复制可作为吞吐优先条件下的主扩展方案；当索引无法复制时，**GPU** 侧归并相比 **CPU** 聚合更有利于减少主机侧路径开销，而扩展收益能否真正保留下来，还取决于存储拓扑与 **I/O** 路径组织方式。

7.2 未来工作展望

本文的工作主要集中在单机环境下金融大规模异常检测系统的实现与优化。沿着这一主线继续向前推进，仍有若干方向值得进一步深入。它们并非对现有工作的简单延伸，而是源于本文已经触及但尚未展开的研究边界。

当前工作已从异常评分、候选召回和系统执行路径三个层面完成代表性比较：**PyTOD** 全量评分提供效果参考，**FAISS-GPU** + **PyTOD** 工程组合基线控制简单 **ANN** 前端变量，**FlashTOD** 内部路径消融解释数据搬运、候选组织和多 **GPU** 归并开销。受实验周期和系统范围限制，本文没有穷尽所有 **ANNS** 前端的替换实验。后续可在统一数据、候选预算、查询集和 **PyTOD** 评分后端下，扩展比较 **HNSW**、**IVF/IVF-PQ** 与 **DiskANN** 等候选检索路径，以分析前端实现对端到端异常检测系统的影响。

7.2.1 面向更真实金融场景的数据验证

本文已经在真实或半真实数据上验证了方法可用性，也借助高保真合成数据完成了大规模系统实验，但更接近生产环境的金融业务数据仍然有限。在严格遵守隐私与合规要求的前提下，后续若能接入更复杂分布、更强业务噪声和更明显概念漂移条件下的数据，将有助于进一步检验 FlashTOD 在真实业务约束下的稳定性，也能更准确地评估系统结论向生产场景外推的边界。

7.2.2 面向多机场景的分布式扩展

本文重点解决的是单机多 GPU 条件下的系统路径组织，多机场景中的跨节点通信、远程结果归并和分布式索引管理尚未进入论文范围。如果继续沿用控制面与数据平面分离的思路，把它扩展到多节点环境，就有机会进一步回答更大规模集群中的查询分发、跨节点索引一致性以及层次化归并问题，这将决定本文方案能否从单机扩展走向更大规模部署。

7.2.3 面向更深层存储路径的优化

本文已经讨论了外存驻留路径、拓扑感知 I/O 控制和 GDS 相关约束，但对更深层的存储访问优化仍然展开得不够。后续若进一步引入 GPUDirect Storage、RDMA 等技术，并把查询调度与存储访问策略做更细粒度的联动，就有可能继续减少主机参与和拷贝代价，从而在更大规模数据和更复杂路径条件下保留更多系统收益。

7.2.4 面向图异常检测与时序异常检测的统一扩展

本文已经把图金融数据纳入统一向量接口，但现有系统主干仍主要围绕表格型异常检测与候选召回—评分链路展开。后续若在保持总体架构稳定的前提下，引入面向图异常检测和时序异常检测的表示方式，例如图嵌入、图神经网络、时序编码或基于事件流的特征聚合机制，就有机会把当前系统从可兼容多类输入进一步推进到对不同异常类型具备更强适配性的统一平台。

7.2.5 面向国产 GPU 平台的验证与部署

本文当前实验与实现主要建立在现有 NVIDIA GPU 软件栈之上，而国产 GPU 平台在算子支持、向量检索实现、设备间通信机制和存储接口适配上往往存在不同约束。围绕这些差异进一步验证和移植本文方案，不仅能够检验系统设计中哪些部分具有较强的平台无关性，也有助于探索等价或近似等价的工程实现路径，从而提升方案在不同计算平台上的实际部署价值。

参考文献

- [1] Aggarwal C C. Outlier analysis[M]. 2nd ed. Springer, 2017.
- [2] 宋凌云, 马卓源, 李战怀, 等. 面向金融风险预测的时序图神经网络综述 [J]. 软件学报, 2024, 35(8):3897-3922.
- [3] 裴威, 李战怀, 潘巍. GPU 数据库核心技术综述 [J]. 软件学报, 2021, 32(3):859-885.
- [4] 张延松, 刘专, 韩瑞琛, 等. GPU 数据库 OLAP 优化技术研究 [J]. 软件学报, 2023, 34(11): 5205-5229.
- [5] 宋子文, 王斌, 张喜瑞, 等. 向量数据库中近似最近邻搜索关键技术综述 [J]. 软件学报, 2026, 37(3):971-1005.
- [6] 李晨, 刘畅, 葛一漩, 等. 多 GPU 系统非一致存储访问优化: 研究进展与展望 [J]. 电子学报, 2024, 52(5):1783-1800.
- [7] Zhao Y, Chen G, Jia Z. Tod: Gpu-accelerated outlier detection via tensor operations[J]. Proceedings of the VLDB Endowment, 2022.
- [8] Xiao Y, Sun M, Song Z, et al. Breaking the storage-compute bottleneck in billion-scale anns: A gpu-driven asynchronous i/o framework[Z]. 2025.
- [9] NVIDIA. cuvs documentation: Multi-gpu nearest neighbors[EB/OL]. 2026. https://docs.rapid.s.ai/api/cuvs/stable/python_api/neighbors_multi_gpu/.
- [10] Chandola V, Banerjee A, Kumar V. Anomaly detection: A survey[J/OL]. ACM Computing Surveys, 2009, 41(3):15:1-15:58. DOI: 10.1145/1541880.1541882.
- [11] Markou M, Singh S. Novelty detection: A review—part 1: Statistical approaches[J/OL]. Signal Processing, 2003, 83(12):2481-2497. DOI: 10.1016/j.sigpro.2003.07.018.
- [12] Phua C, Lee V, Smith-Miles K, et al. A comprehensive survey of data mining-based fraud detection research[J/OL]. arXiv preprint arXiv:1009.6119, 2010. DOI: 10.48550/arXiv.1009.6119.
- [13] Akoglu L, Tong H, Koutra D. Graph based anomaly detection and description: A survey[J/OL]. Data Mining and Knowledge Discovery, 2015, 29(3):626-688. DOI: 10.1007/s10618-014-0365-y.
- [14] Breunig M M, Kriegel H P, Ng R T, et al. Lof: Identifying density-based local outliers[C]// Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data. 2000.
- [15] Angiulli F, Pizzuti C. Fast outlier detection in high dimensional spaces[C]//Proceedings of the 6th European Conference on Principles of Data Mining and Knowledge Discovery (PKDD). 2002.
- [16] 陈波冯, 李靖东, 卢兴见, 等. 基于深度学习的图异常检测技术综述 [J]. 计算机研究与发展, 2021, 58(7):1436-1455.

- [17] 李康和, 黄震华. 基于噪声过滤与特征增强的图神经网络欺诈检测方法 [J]. 电子学报, 2023, 51(11):3053-3060.
- [18] 于浩淼, 刘炜, 孟流畅, 等. RWK-GNN: 基于特征增强与子核分解的非平衡图欺诈检测算法 [J]. 电子学报, 2024, 52(10):3382-3391.
- [19] Angiulli F, Basta S, Lodi S, et al. Fast outlier detection using a gpu[C/OL]//2013 International Conference on High Performance Computing & Simulation (HPCS). 2013:143-150. DOI: 10.1109/HPCSim.2013.6641405.
- [20] Azmandian F, Yilmazer A, Dy J G, et al. Harnessing the power of gpus to speed up feature selection for outlier detection[J/OL]. Journal of Computer Science and Technology, 2014, 29(3):408-422. DOI: 10.1007/s11390-014-1439-4.
- [21] Paul J, Lu S, He B. Database systems on gpus[J/OL]. Foundations and Trends in Databases, 2021, 11(1):1-108. DOI: 10.1561/19000000076.
- [22] RAPIDS Developers. cudf documentation: How it works / cudf.pandas[EB/OL]. 2026. https://docs.rapids.ai/api/cudf/stable/cudf_pandas/how-it-works/.
- [23] RAPIDS Developers. Rmm documentation: Pinnedhostmemoryresource[EB/OL]. 2026. <https://docs.rapids.ai/api/rmm/stable/>.
- [24] Johnson J, Douze M, Jégou H. Billion-scale similarity search with gpus[J]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2019.
- [25] FAISS Contributors. Faiss on the gpu[EB/OL]. 2024. <https://github.com/facebookresearch/faiss/wiki/Faiss-on-the-GPU>.
- [26] NVIDIA. cuvs documentation: Distribution mode / search mode / merge mode (multi-gpu ann)[EB/OL]. 2026. <https://docs.rapids.ai/api/cuvs/stable/>.
- [27] NVIDIA. Nccl documentation: Cuda stream semantics[EB/OL]. 2024. <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/streams.html>.
- [28] NVIDIA. Gpudirect storage documentation: Overview guide[EB/OL]. 2024. <https://docs.nvidia.com/gpudirect-storage/overview-guide/>.
- [29] NVIDIA. Gpudirect storage documentation: Design guide[EB/OL]. 2024. <https://docs.nvidia.com/gpudirect-storage/design-guide/>.
- [30] Liu Z, Xiao Z, Li L, et al. Dgraph: A large-scale financial dataset for graph anomaly detection[EB/OL]. 2022. <https://arxiv.org/abs/2207.03579>.
- [31] IBM Research. Ibm aml-data: Synthetic data for anti-money laundering research[EB/OL]. 2022. <https://github.com/IBM/AML-Data>.
- [32] IBM Research. Amlsim: A synthetic financial transaction generator for anti-money laundering research[EB/OL]. 2021. <https://github.com/IBM/AMLSim>.
- [33] IEEE-CIS. Fraud detection (ieee-cis) dataset[EB/OL]. 2019. <https://www.kaggle.com/c/ieee-fraud-detection>.
- [34] Dal Pozzolo A, Caelen O, Johnson R A, et al. Calibrating probability with undersampling for unbalanced classification[C/OL]//2015 IEEE Symposium Series on Computational Intelligence (SSCI). 2015. DOI: 10.1109/SSCI.2015.33.

-
- [35] Machine Learning Group – ULB, Kaggle. Credit card fraud detection (kaggle dataset)[EB/OL]. 2013. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>.
 - [36] DGraph Team. Dgraphfin dataset (official website)[EB/OL]. 2022. <https://dgraph.xinye.com/dataset>.
 - [37] DGraph Team. Dgraphfin_baseline (github repository)[EB/OL]. 2022. https://github.com/DGraphXinye/DGraphFin_baseline.
 - [38] NVIDIA. cuvs documentation: Multi-gpu ivf-pq / ivf-flat (host memory requirement)[EB/OL]. 2026. <https://docs.rapids.ai/api/cuvs/stable/>.
 - [39] Crankshaw D, Wang X, Zhou G, et al. Clipper: A low-latency online prediction serving system[C]//Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation. 2017.
 - [40] Gu J, Lee Y, Zhang C, et al. Clockwork: Predictable and efficient deep learning serving for cloud[C]//Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation. 2020.
 - [41] NVIDIA. cuvs documentation: Dynamic batching[EB/OL]. 2026. <https://docs.rapids.ai/api/cuvs/stable/>.

附录 A 实验脚本与日志归档说明

本附录结合现有实验脚本与历史运行记录，对论文中各验证阶段所对应的脚本入口、日志归档方式和参数组织口径作统一说明。目的不是列出全部内部文件，而是将正文中需要复核的单卡、多 GPU、真实集有效性和论文出图阶段，整理为可复用的一套归档规范。

A.1 实验阶段与材料对应关系

论文验证阶段	代表性脚本或日志前缀	主要用途
真实集路径检查与冒烟验证	<code>freeze_last_tune_<ts>/01_*</code>	对真实集输入路径、基础运行链路和候选召回—评分接口进行快速联调，对应正文中的真实集有效性验证起点。
真实集校准与配置筛选	<code>freeze_last_tune_<ts>/02_*</code>	保存校准轮次的标准输出、排序摘要和配置对比结果，用于冻结论文定稿前的候选规模与评分配置。
单卡链路对比与阶段耗时分析	<code>bench_flash_<scale>_<scene>_gpu<id>.log、compare_<topic>_<ts>.md</code>	对应第四章单卡路径优化实验，重点记录原始链路、召回加速、GPU 主导链路和完整优化方案之间的耗时与吞吐差异。
多 GPU 扩展、归并与冲突对比	<code>bench_flash_<scale>_replay_*、*_shard_*、gpu_<topic>_gpu<id>.csv</code>	对应第五章索引复制主方案、设备侧归并兜底路径、主机侧聚合对比基线以及共享路径冲突实验。
论文图表整理与结果汇总	<code>latest_<topic>_paper/、chart_ready_*.csv、session_metadata.txt</code>	将多轮实验结果整理为论文图表输入数据，保留轮次汇总、出图口径与会话元信息。

A.2 核心实验日志与复现材料归档

表 A-1 给出正文核心实验与归档材料之间的对应关系。该表只说明复现材料的组织口径，不引入新的实验数值；若后续补充重复运行统计，应在相同目录结构下继续追加原始日志、汇总 CSV 和绘图输入文件。

表 A-1 核心实验日志与复现材料归档说明

实验类别	代表性日志或数据前缀	对应正文位置	归档说明
有效性验证	freeze_last_tune_<ts>/、 baseline_effect_*.csv	第三章基础实验、第六章 基线效果对比	保留真实或半真实数据、1M 桥接规模下的候选召回、候选评分和绘图输入。
单卡递进式消融	bench_flash_<scale>_<scene>_gpu<id>.log、 single_gpu_progressive_*.csv	第四章单卡实验、第六章 综合对比	保留原始分段链路、候选召回加速、GPU 主导链路和完整优化方案的运行记录。
多 GPU 扩展	bench_flash_<scale>_replica_*.csv、 multigpu_scaling_*.csv	第五章多 GPU 扩展、第六章 扩展趋势	保留 replicated、GPU merge、CPU aggregation 和 shared-path conflict 相关运行记录。
绘图与结果汇总	plot_assets/、figures/exp/、 data/*.csv	第四章至第六章结果图	保留论文图表使用的数据源、出图脚本和导出 PDF/SVG。

A.3 统一文件命名风格

- 会话目录统一采用“任务名 + 时间戳”形式，例如 freeze_last_tune_<YYYYmmdd_HHMMSS>，用于标识一轮冻结前复核或专题实验。
- 单次运行日志统一采用 bench_<route>_<scale>_<scene>_gpu<id>.log，其中 route 表示单卡或多卡执行路径，scale 表示数据规模，scene 表示验证场景或配置标签。
- 对比结果统一采用 compare_<topic>_<timestamp>.md 与 compare_<topic>_<timestamp>.csv 成对保存，其中 Markdown 文件用于保留人工可读结论，CSV 文件用于后续绘图与表格汇总。
- 过程检查类文件统一采用 0N_<step>.prompt.txt、0N_<step>.stdout.log 和 checklist.log 的组合形式，用于保留命令入口、标准输出和阶段性检查记录。
- 出图目录统一保留 all_rounds.csv、chart_ready_*.csv 和 session_meta.txt 三类文件，分别对应原始轮次结果、绘图输入和会话说明。

A.4 统一参数设置风格

- 数据与划分参数统一使用 real-root、real-datasets、synthetic-root、synthetic-scales、input-npz、out-dir、seed 等名称，明确区分真实集入口、合成集规模和数据切分位置。
- 单卡热路径参数统一使用 device、batch-size、score-chunk-size、warmup、repeats、max-run-seconds，分别控制设备绑定、批大小、评分块大小、预热次数和单轮运行上限。

- 候选召回与索引参数统一使用 `flash-search-k`、`flash-rerank-exact`、`flash-indices-only`、`faiss-index`、`faiss-nlist`、`faiss-nprobe`、`faiss-pq-m`、`faiss-pq-bits` 等名称，避免在不同脚本中混用语义相近的别名。
- 多 GPU 相关参数统一使用 `gpu-list`、`flash-multi-gpu-route`、`flash-service-topology`、`flash-service-protocol`、`flash-multi-gpu-devices`、`flash-multi-gpu-gather-device`、`flash-shards`、`flash-service-batch-size`，明确区分索引复制主路径、分片兜底路径和归并位置。
- 监控与分析参数统一使用 `gpu-log-path`、`gpu-log-interval-ms`、`chain-profile`、`trace-output-dir` 等名称；Shell 包装脚本中对应使用全大写环境变量，如 `BATCH_SIZE`、`FLASH_SEARCH_K`，冻结配置则统一使用 `FREEZE_*` 前缀。

A.5 RAPIDS 相关组件使用位置说明

RAPIDS 相关组件在本文中作为 GPU 数据处理与内存资源管理支撑，不单独作为异常检测算法或候选召回算法，也不将整体性能收益单独归因于 RAPIDS。表 A-2 对相关组件在本文系统中的使用位置作统一说明。

表 A-2 RAPIDS 相关组件使用位置说明

组件	使用位置	对应脚本/阶段口径	作用	是否单独归因性能
cuDF	数据预处理与候选前后表格组织	预处理脚本 / <code>query_prep</code> 阶段	GPU <code>DataFrame</code> 形式的字段筛选、类型转换、窗口统计和批量结果组织	否
CuPy	候选数组与中间张量表达	候选组织 / <code>pack_candidates</code> 阶段	用于设备侧数组计算、候选 ID/距离缓冲和张量接口衔接	否
RMM	内存池与页锁定主机内存资源	资源初始化 / 运行时配置阶段	管理临时内存、pinned host 资源和内存分配抖动	否
cuML	环境组件与后续可扩展接口	软件环境快照 / 非核心热路径	作为 RAPIDS 生态组成记录，本文不将其作为核心评分或召回路径依赖	否

附录 B 图表补充说明

本附录对正文图表中反复出现的变量命名、单位和比较口径作统一补充，便于阅读时快速核对不同章节之间的含义是否一致。

B.1 图表口径说明

- 图中的 QPS、平均时延和 p99 时延均指对应实验场景下的端到端统计结果；若未特别说明，时延单位统一为 ms。
- 单卡结果图主要用于比较数据路径优化前后的链路差异，多 GPU 结果图主要用于比较索引复制主方案、设备侧归并兜底路径以及共享路径冲突条件下的扩展差异。
- 真实集结果用于有效性验证，1M 及以上合成数据结果用于系统性能与扩展分析，二者不直接比较绝对吞吐数值。
- 正文结果图中的方案名称均对应具体验证场景或系统路径，不再使用含义不明确的编号标签。

B.2 特征构造字段与派生特征补充说明

表 B-1 特征构造明细

原始字段/信息源	派生特征	特征类型	预处理方式	在系统中的作用
交易金额	标准化金额、对数金额、 时间窗金额均值/波动统计	连续型	缺失填补、异常值截断、标准化/归一化	刻画交易规模及金额异常波动
交易时间戳	小时段、星期信息、夜间交易标记、平均交易间隔	时间型/统计型	时间规范化、周期化表达、滑动时间窗统计	捕捉异常时段行为与时序突变
账户标识及账户历史	账户时间窗交易次数、平均金额、活跃度统计	类别索引 + 统计型	安全编码、滑动窗口聚合	表征账户级长期与短期行为模式
设备标识	设备使用频次、设备共享度、账户—设备切换统计	类别型/统计型	缺失标识、频次编码、窗口统计	识别设备复用、盗刷与团伙风险
商户标识/商户类别	商户类别编码、商户频次、商户局部风险统计	类别型/统计型	one-hot、频次编码或安全统计编码	增强交易场景相似性与商户异常识别能力
地理位置字段	地域编码、跨地域切换次数、地理跳变标记	类别型/统计型	地域粒度统一、编码、窗口切换统计	捕捉异地交易、短时跨区域异常
支付渠道/终端类型	渠道编码、终端类别向量、渠道频次统计	类别型	one-hot 或频次编码	丰富候选召回的相似度基础，提升异常区分度
图节点原始属性	节点属性规范化向量	混合型	数值归一化、类别编码	形成图节点输入的基础表示
图边关系与邻域信息	入边/出边计数、邻域金额统计、方向比例、时间分布	关系统计型	邻域聚合、时间窗统计、向量拼接	将图结构信息映射为可检索、可评分的节点向量
融合后统一表示	固定维度最终向量 ($v_i \in \mathbb{R}^d$, d 由最终预处理脚本确定)	向量型	维度检查、类型检查、异常值截断	完整路径作为 FlashANNs 候选召回输入，前期可用于 FAISS-GPU 接口验证，并直接供 PyTOD 评分使用

B.3 多 GPU 实验配置补充表

表 B-2 多 GPU 实验配置补充表

(a) 固定参数与实验口径						
GPU 数	方案	数据规模	批大小	候选数	k	实验目的
1	Replicated	10M	4096	128	50	单卡吞吐扩展基线
2	索引复制	10M	4096	128	50	2 GPU 索引复制扩展基线
4	索引复制	10M	4096	128	50	四卡索引复制扩展基线
4	本地评分 + 设备侧归并	100M	3072	128	50	显存受限时的设备侧归并兜底方案
4	本地评分 + 主机侧聚合	100M	3072	128	50	主机侧聚合对比基线
2	拓扑感知绑定	100M	4096	128	50	I/O 通道绑定对比实验
4	拓扑感知绑定	100M	4096	128	50	I/O 通道绑定对比实验
2	Shared-path conflict	100M	4096	128	50	共享路径冲突对比基线
4	Shared-path conflict	100M	4096	128	50	共享路径冲突对比基线

(b) 方案配置说明				
方案	索引组织	归并路径	I/O 绑定	定位
Replicated	Full replica	None	Topology-aware	主扩展方案，验证随 GPU 数量增加的吞吐收益
本地评分 + 设备侧归并	分片索引	NCCL all-gather + 设备侧归并	拓扑感知	显存受限时的设备侧归并兜底路径
本地评分 + 主机侧聚合	分片索引	主机侧聚合	拓扑感知	用于对比 CPU 回到数据平面的额外代价
拓扑感知绑定	完整副本	无	拓扑感知	验证 I/O 通道绑定对扩展效率的影响
Shared-path conflict	Full replica	None	Shared PCIe/SSD path	作为共享路径冲突下的对比基线

注：多 GPU 实验沿正文定义的扩展规模链路组织，即由真实或半真实数据和 1M 桥接规模逐步延伸到 100M 样本压力场景。10M 主要服务于索引复制主方案扩展曲线，100M 主要服务于兜底归并与路径冲突对比。

附录 C 缩写、变量与配置对照

本附录列出正文中频繁出现的关键变量、配置项及其含义，便于后续统一实验脚本和结果复核。

变量或配置项	含义
FAISS-GPU	FAISS 的 GPU 检索实现；本文中主要用于前期 ANN 接口与候选召回链路验证，包括设备侧输入输出路径、候选 ID/距离返回格式以及 PyTOD 评分衔接。
FlashANNS	完整 FlashTOD 路径中的主要候选召回前端，用于大规模外存驻留或超显存索引条件下的高吞吐候选生成。
初步融合方案	FAISS-GPU ANN API 与 PyTOD 的简单串接，用于验证候选召回—候选评分链路可行性。
完整融合方案	以 FlashANNS 为主要候选召回前端、以 PyTOD 为异常评分主干，并结合 GPU 主导数据路径和设备侧候选组织后的完整 FlashTOD 实现。
batch_size	单次提交到候选召回或精排路径中的批处理大小。
rerank_budget	进入高精度精排阶段的候选数量预算。
nlist	索引聚类桶数量或粗排分区规模参数。
nprobe	查询阶段实际访问的粗排桶数量。
device_id	当前任务绑定的 GPU 设备标识。
replica_mode	多 GPU 复制式多实例执行模式开关。
bounded_batch_size	在时延约束下允许形成的微批上限。

在学期间完成的相关学术成果

致 谢